

sábado, 09 de mayo de 2026

UNIVERSIDAD DE SONORA

Facultad Interdisciplinaria de Ingenierías



4to Semestre

Ingeniería en Sistemas de Información

ACT 7 -Proyecto final: API REST -Manga Store

Maestro: Meza Ibarra Ivan Dostoyewsky

Meza Diaz María de los Ángeles

García Horta Abril Melissa

Córdova Montiel Axel

Contenido

Contenido	2
Introducción	5
Contexto del Proyecto	5
Objetivos	5
Estructura del Proyecto.....	6
Organización por Carpetas (Módulos de Interfaz).....	6
Archivos de Soporte y Backend (Raíz)	6
Base de Datos (mangaStore1.db).....	6
Diagrama de esquema relacional	7
API REST (MangaStore.py)	8
Librerías	8
Inicializar la base de datos y crear las tablas	9
Método GET en mangas	11
GET por id en mangas	12
Método GET por Nombre mangas	13
Método GET Mangas por autor.....	14
Método POST en mangas	15
Metodo PUT en mangas	16
Método DELETE en mangas.....	17
Capturas de postman para mangas	17
Método GET en editoriales	26
Método GET en editoriales por id	27
Método GET en editoriales por Nombre	27
Prueba en Postman para editoriales get por nombre:	28
Metodo GET en editoriales por Pais.....	30
Método POST en editoriales	35
Método PUT en editoriales	36
Método DELETE en editoriales.....	36

Capturas de postman para editoriales	37
METODO GET en ventas	42
Metodo GET por id en ventas	44
Metodo POST en ventas	45
Método DELETE en ventas	47
Método PUT en ventas	48
Capturas de postman para ventas	50
Método GET para las estadísticas del panel.....	59
Prueba en postman de stats.....	59
Método Get para la tabla manga_imagenes	60
Método POST para manga_imagenes	61
Método DELETE para manga_imagenes.....	62
Prueba en Postman para manga_Imagenes	62
Carpeta Catalogo Mangas.....	62
Manga.html:	62
Manga.css:	62
Manga.js :	64
Carpeta PP (Página Principal)	71
pp.html:	71
pp.css:	71
Pp.js.....	71
Carpeta historialVentas	80
Ventas.html	80
Ventas.css.....	80
Ventas.js	83
Mapeo y visualización	83
Gestión del Carrito (Modal de Actualización)	90
Operaciones CRUD y Persistencia	93
Búsqueda y Filtrado.....	96

Carpeta editorial	97
Editorial.html.....	97
styleEditorial.css.....	98
Editorial.js	100
Metodos CRUD	100
CONCLUSIONES.....	107
1. ESTRUCTURA	107
2. DISEÑO DE DATOS	108
3. CRUD	108
4. INTERFAZ VISUAL Y EXPERIENCIA DEL USUARIO	109
CONCLUSION GENERAL	109

Introducción

En la actualidad, la transformación digital es un pilar fundamental para optimizar la eficiencia operativa en cualquier modelo de negocio. El presente proyecto consiste en el desarrollo de un **Sistema Integral de Gestión para "MangaStore"**, una plataforma diseñada específicamente para la administración de inventario y control de ventas de una tienda especializada en mangas japoneses.

Contexto del Proyecto

El sistema nace como una solución a la necesidad de centralizar procesos administrativos que, de forma manual, son propensos a errores, tales como la duplicidad de registros y la falta de sincronización entre los datos lógicos de un producto y sus recursos visuales. La temática principal gira en torno a la **digitalización de flujos de trabajo**, permitiendo una gestión precisa de títulos, volúmenes, autores y niveles de stock.

Para abordar esta problemática, se ha implementado una arquitectura cliente-servidor robusta:

- **Frontend:** Desarrollado con tecnologías web modernas (HTML5, CSS3, JavaScript) para ofrecer una interfaz de usuario minimalista y reactiva, facilitando la interacción del administrador con el catálogo.
- **Backend:** Una API desarrollada en **Flask (Python)** que gestiona la lógica de negocio y la comunicación con la base de datos.
- **Base de Datos Relacional:** Utiliza un modelo relacional (sql3Lite) que almacena todos los registros y permite la relacion de varios datos de manera organizada.

Objetivos

El propósito fundamental de esta herramienta es proporcionar una plataforma donde la administración de inventario y el proceso de venta sea ágil y visualmente intuitiva,

resolviendo retos técnicos como la persistencia de datos complejos en una estructura de base de datos dividida.

Estructura del Proyecto

El proyecto sigue una organización modular basada en vistas, donde cada carpeta en la raíz representa un módulo funcional o página de la aplicación.

Organización por Carpetas (Módulos de Interfaz)

Cada carpeta principal (**catalogomangas**, **editorial**, **historialVentas**, **pp**) contiene los tres procesos.

Archivos .html: Definen la estructura semántica y el contenido de la página (formularios, contenedores de tarjetas, modales).

Archivos .css: Gestionan la identidad visual, aplicando estilos modernos, paletas de colores y el diseño responsivo de los elementos.

Archivos .js: Controlan la lógica del lado del cliente, gestionando el consumo de la API mediante fetch, la manipulación del DOM y la interactividad de los modales.

Archivos de Soporte y Backend

Fuera de las carpetas de interfaz, se encuentran el api y la base de datos del sistema:

mangaStore.py: El servidor principal desarrollado en Flask. Actúa como el puente entre la base de datos y la interfaz de usuario.

mangaStore1.db: El archivo de la base de datos SQLite, que garantiza la persistencia de toda la información de manera local y eficiente.

Base de Datos (mangaStore1.db)

La estructura se divide en tablas especializadas:

editoriales: Funciona como un catálogo que contiene el nombre y el país de las empresas editoras. Su llave primaria es el id.

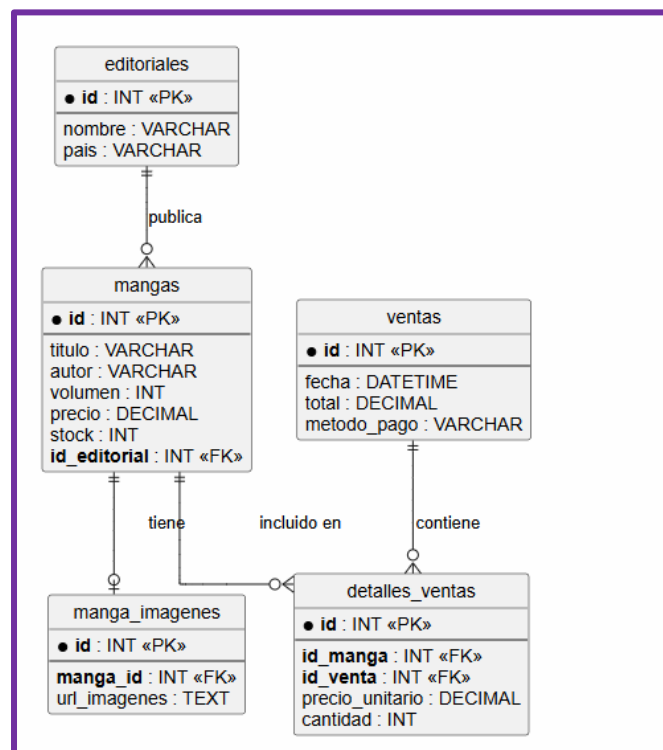
mangas: Almacena los datos maestros del producto, incluyendo título, autor, volumen, precio y stock. Esta tabla se categoriza mediante la llave foránea **id_editorial**, que vincula cada tomo con su respectiva editorial de forma limpia.

manga_imagenes: Es una tabla especializada en recursos visuales. Vincula el **manga_id (llave foránea)** con **url_imagenes**, permitiendo que cada producto tenga su ruta de imagen asociada sin sobrecargar la tabla principal de mangas.

ventas: Registra la información general de cada transacción realizada, capturando la fecha, el total acumulado y el **metodo_pago**.

detalles_ventas: Debido a que la relación entre mangas y ventas es de muchos a muchos (una venta puede tener varios mangas y un manga puede estar en varias ventas), surge esta tabla intermedia. Aquí se descomponen los movimientos, almacenando las llaves forneas **id_manga** e **id_venta**, junto con datos específicos del momento de la compra: **precio_unitario** y **cantidad**.

Diagrama de esquema relacional



API REST (MangaStore.py)

Construido con Flask (Python), funciona como una API RESTful. Su propósito es recibir las solicitudes del Frontend (como "quiero crear un manga" o "muéstrame las ventas") y procesarlas contra la base de datos.

Librerías

```
#librerías necesarias para el funcionamiento del backend
from flask import Flask, jsonify, request
from flask_cors import CORS #permite que el frontend pueda hacer peticiones a este backend sin problemas de CORS
import sqlite3
from datetime import datetime#importamos la libreria datetime para poder guardar la fecha y hora de las ventas en la base de datos

app = Flask(__name__)

CORS(app)#habilitamos CORS para permitir que el frontend pueda hacer peticiones a este backend sin problemas de CORS
#para que no ordene por alfabeto los datos del json y mantenga el orden que designamos
app.json.sort_keys = False
database = "mangaStore1.db" #nombre de la base de datos

def conexionDB(): #definimos un metodo para conectarnos a la base de datos
    return sqlite3.connect(database) #en caso de que la base de datos no exista sqlite3 la creara
```

En la parte superior tenemos la importación de las librerías a utilizar, como anteriormente habíamos estado trabajando, se utilizaron la librería **Flask** y la librería **sqlite3** para manipular la base de datos, jsonify para la implementación de json, request para obtener los datos que inserte el usuario y CORS para no haya problemas con la comunicación del código y de la interfaz. Posteriormente se crea la instancia de la aplicación, también especificamos que los datos que se visualizaran al utilizar el método get, se muestren en el orden que se le está designando.

Se le asigna la ruta de la base de datos a una variable, para un mejor manejo dentro del código, evitando errores al escribir varias veces la ruta.

Se crea una función para la conexión con la base de datos estableciendo que en caso de que esta no exista, sqlite3 la creara.

Inicializar la base de datos y crear las tablas

```
# metodo para la creacion de los campos de la base de datos
def inicializarDB():
    try:
        conn = conexionDB() #abre la conexion a la base de datos
        cursor = conn.cursor() #crea un cursor para ejecutar las consultas

        # Habilitar llaves foráneas a nivel de base de datos
        cursor.execute("PRAGMA foreign_keys = ON;")
```

Se implementó una función para inicializar la base de datos mediante el manejo de excepciones. Esto establece la conexión inicial y define la estructura de las tablas para el inventario de mangas. Además, también activamos las llaves foráneas para que haya concordancia entre las tablas y el funcionamiento de la base de datos. Al finalizar la creación, se debe cerrar la conexión para liberar recursos. En caso de fallo, el sistema captura la excepción y despliega un mensaje informativo indicando el error en la inicialización.

Querys para crear las tablas

```
# ejecutamos la consulta que creara los campos de nuestra base de datos

#crear tabla editoriales
cursor.execute("""
    CREATE TABLE IF NOT EXISTS editoriales (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nombre TEXT NOT NULL,
        pais TEXT NOT NULL
    )
""")
```

```
#crear tabla mangas
cursor.execute("""
    CREATE TABLE IF NOT EXISTS mangas (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        titulo TEXT NOT NULL,
        autor TEXT NOT NULL,
        volumen INTEGER NOT NULL,
        precio REAL NOT NULL,
        stock INTEGER NOT NULL,
        id_editorial INTEGER,
        FOREIGN KEY(id_editorial) REFERENCES editoriales(id)
    )
""")
```

```
#crear tabla detalle_ventas
```

```
cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS detalle_ventas (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        venta_id INTEGER NOT NULL,  
        manga_id INTEGER NOT NULL,  
        cantidad INTEGER NOT NULL,  
        precio_unitario REAL NOT NULL,  
        FOREIGN KEY(venta_id) REFERENCES ventas(id),  
        FOREIGN KEY(manga_id) REFERENCES mangas(id)
```

```
)
```

```
""")
```

```
# Crear tabla de imágenes relacionada
```

```
cursor.execute("""
```

```
    CREATE TABLE IF NOT EXISTS manga_imagenes (  
        id INTEGER PRIMARY KEY AUTOINCREMENT,  
        manga_id INTEGER NOT NULL,  
        url_imagen TEXT NOT NULL,  
        FOREIGN KEY(manga_id) REFERENCES mangas(id) ON DELETE CASCADE
```

Método GET en mangas

```
#VISUALIZAR TODOS LOS MANGAS, METODO GET
@app.route("/mangas", methods=["GET"])
def obtenerTodos():
    try:
        conn = conexionDB()
        cursor = conn.cursor()
        cursor.execute("""
            SELECT m.id, m.titulo, m.autor, m.volumen, m.precio, m.stock, e.nombre, i.url_imagen
            FROM mangas m
            LEFT JOIN manga_imagenes i ON m.id = i.manga_id
            LEFT JOIN editoriales e ON m.id_editorial = e.id
        """) #ejecuta la consulta MySQL para seleccionar todos los registros de la base de datos
        filas = cursor.fetchall() #guarda todos los datos obtenidos de la consulta

        #convertimos cada fila de la base de datos (la cual es una tupla) en un diccionario de python
        mangas = []
        for f in filas:
            mangas.append({
                "id": f[0],
                "titulo": f[1],
                "autor": f[2],
                "volumen": f[3],
                "precio": f[4],
                "stock": f[5],
                "id_editorial": f[6],
                "url_imagen": f[7]
            })

        conn.close() #cierra la conexion a la base de datos
        return jsonify(mangas), 200 #envia la lista al usuario en formato JSON
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Para el **endpoint get** también se hizo uso del manejo de excepciones, dentro del try nuevamente se inicia abriendo una conexión, por medio de **cursor.execute** se ejecuta la consulta la cual es un **join** que concatena la tabla mangas y la tabla **manga_imagenes** para traerse la información completa con respecto al producto, dicho **join** nos facilitara la visualización del producto en la interfaz. Posteriormente estos se colocan dentro el formato establecido de **json** utilizando un **for** y una vez todos los datos han sido colocados correctamente se debe cerrar la conexión con la bd para liberar recursos, si la operación es exitosa se desplegará un mensaje de éxito junto con su respectivo código. En dado caso que ocurra un error, ya sea al ejecutar la consulta o intentar acceder al archivo de la base de datos, el sistema lanzara un mensaje de error acompañado de su respectivo código.

GET por id en mangas

```
#VISUALIZAR MANGA POR ID, METODO GET
@app.route("/mangas/<int:id>", methods=["GET"])
def obtenerUno(id):
    try:
        conn = conexionDB()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT m.id, m.titulo, m.autor, m.volumen, m.precio, m.stock, e.nombre, i.url_imagen
            FROM mangas m
            LEFT JOIN manga_imagenes i ON m.id = i.manga_id
            LEFT JOIN editoriales e ON m.id_editorial = e.id
            WHERE m.id = ?
        """, (id,)) #ejecuta la consulta mysql para seleccionar el registro que coincida con el id ingresado por el usuario
        fila = cursor.fetchone() # fetchone guardara el primer datos que coincida con la consulta
        conn.close()

        # Si "fila" tiene contenido, crearemos el JSON
        if fila:
            manga = {
                "id": fila[0],
                "titulo": fila[1],
                "autor": fila[2],
                "volumen": fila[3],
                "precio": fila[4],
                "stock": fila[5],
                "id_editorial": fila[6],
                "url_imagen": fila[7]
            }
            return jsonify(manga), 200 #consulta exitosa

        # Si no se encuentra el id devolvera un mensaje de error
        return jsonify({"error": "Manga no encontrado"}), 404

    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Trabaja de la misma manera que el **endpoint** de **get general** con la diferencia de que mediante la consulta que se ingresa en el **cursor.execute**, filtramos los mangas por medio de su id y pedimos mediante el fetchone que se nos traiga un único dato.

Método GET por Nombre mangas

```
@app.route("/mangas/nombre/<string:nombre>", methods=["GET"])
def MangaporNombre(nombre):
    try:
        conn = conexionDB()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT m.id, m.titulo, m.autor, m.volumen, m.precio, m.stock, e.nombre, i.url_imagen
            FROM mangas m
            LEFT JOIN manga_imagenes i ON m.id = i.manga_id
            LEFT JOIN editoriales e ON m.id_editorial = e.id
            WHERE m.titulo LIKE ?
        """, (f"%{nombre}%",))

        filas = cursor.fetchall() # Usamos fetchall para obtener TODOS los que coincidan
        conn.close()

        # Transformamos todas las filas en una lista de diccionarios
        mangas = []
        for fila in filas:
            mangas.append({
                "id": fila[0],
                "titulo": fila[1],
                "autor": fila[2],
                "volumen": fila[3],
                "precio": fila[4],
                "stock": fila[5],
                "id_editorial": fila[6],
                "url_imagen": fila[7]
            })

        # Siempre devolvemos la lista, incluso si está vacía []
        return jsonify(mangas), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Este a diferencia del get por id, usa el filtro mediante el nombre del manga

Método GET Mangas por autor

```
#VISUALIZAR MANGA POR AUTOR, METODO GET

@app.route("/mangas/autor/<string:autor>", methods=["GET"])
def MangaporAutor(autor):
    try:
        conn = conexionDB()
        cursor = conn.cursor()

        cursor.execute("""
            SELECT m.id, m.titulo, m.autor, m.volumen, m.precio, m.stock, e.nombre, i.url_imagen
            FROM mangas m
            LEFT JOIN manga_imagenes i ON m.id = i.manga_id
            LEFT JOIN editoriales e ON m.id_editorial = e.id
            WHERE m.autor LIKE ?
        """, (f"%{autor}%",))

        filas = cursor.fetchall() # Usamos fetchall para obtener TODOS los que coincidan
        conn.close()

        # Transformamos todas las filas en una lista de diccionarios
        mangas = []
        for fila in filas:
            mangas.append({
                "id": fila[0],
                "titulo": fila[1],
                "autor": fila[2],
                "volumen": fila[3],
                "precio": fila[4],
                "stock": fila[5],
                "id_editorial": fila[6],
                "url_imagen": fila[7]
            })

        # Siempre devolvemos la lista, incluso si está vacía []
        return jsonify(mangas), 200

    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Funciona de la misma manera que los get por id y por nombre, solo que en este caso el parámetro de búsqueda es el campo "autor"

Método POST en mangas

```
34 #INSERTAR NUEVO MANGA, METODO POST
35 @app.route("/mangas", methods=["POST"])
36 def insertar():
37     try:
38         data = request.json #trae los datos insertados por el usuario
39
40         #validaciones basicas
41         campos_obligatorios = ["titulo", "autor", "volumen", "precio", "stock", "id_editorial"]
42
43         # Revisar que todos los campos existan en el JSON y no sean None o esten vacios
44         for campo in campos_obligatorios:
45             if campo not in data or data[campo] is None or data[campo] == "":
46                 return jsonify({"error": f"El campo '{campo}' es obligatorio y no puede ser nulo"}), 400 #mensaje de error de que falto llenar un campo
47
48         # Validación de tipos y valores numéricos
49         if not isinstance(data["precio"], (int, float)) or data["precio"] < 0:
50             return jsonify({"error": "El precio debe ser un número positivo"}), 400
51         if not isinstance(data["stock"], int) or data["stock"] < 0:
52             return jsonify({"error": "El stock debe ser un número entero positivo"}), 400
53         if not isinstance(data["volumen"], (int, float)):
54             return jsonify({"error": "El volumen debe ser un número "}), 400
55
56         conn = conexionDB()
57         cursor = conn.cursor()
58
59         # Evitar duplicados por título
60         cursor.execute("SELECT id FROM mangas WHERE titulo = ? AND volumen = ?", (data["titulo"], data["volumen"]))
61         if cursor.fetchone():
62             conn.close()
63             return jsonify({"error": "Este manga y volumen ya están registrados"}), 409
64
65         # Verificamos la existencia de editorial
66         cursor.execute("SELECT id FROM editoriales WHERE id = ?", (data["id_editorial"],))
67         if not cursor.fetchone():
68             conn.close()
69             return jsonify({"error": "La editorial no existe"}), 404
70
71         cursor.execute("""
72             INSERT INTO mangas (titulo, autor, volumen, precio, stock, id_editorial)
73             VALUES (?, ?, ?, ?, ?, ?)
74             """, (data["titulo"], data["autor"], data["volumen"], data["precio"], data["stock"], data["id_editorial"]))
75
76         conn.commit() #guarda los ccambios
77         nuevo_id = cursor.lastrowid #nos permite saber el nuevo id que le asigno la base de datos al nuevo producto que agregamos
78         conn.close()
79         return jsonify({"mensaje": "Manga registrado correctamente", "id": nuevo_id}), 201 #mensaje de que el manga se agrego correctamente
80     except Exception as e:
```

Para el método POST se utiliza un bloque de manejo de excepciones y varias validaciones, aquí se valida que el usuario haya ingresado todos los campos obligatorios y que estos no estén en null o vacíos, también se valida que no se ingresen títulos duplicados, que el stock, precio y volumen sean números y no letras y que si exista el id del editorial ingresado. Una vez validados los datos, se establece la conexión con la base de datos para ejecutar la sentencia de inserción. Tras confirmar la transacción y recuperar el id asignado, se liberan los recursos del sistema cerrando la conexión y se devuelve una respuesta exitosa. Cualquier falla en el acceso al archivo o en la ejecución de la consulta es capturada por una excepción que retorna un mensaje error y su respectivo código de estado HTTP.

Método PUT en mangas

```
@app.route("/mangas/<int:id>", methods=["PUT"])
def actualizar(id):
    try:
        data = request.json #obtiene los datos insertados por el usuario

        #validaciones basicas para actualizar
        campos_obligatorios = ["titulo", "autor", "volumen", "precio", "stock", "id_editorial"]
        # Revisar que todos los campos existan en el JSON y no sean None o esten vacios
        for campo in campos_obligatorios:
            if campo not in data or data[campo] is None or data[campo] == "":
                return jsonify({"error": f"Para actualizar, el campo '{campo}' no puede ser nulo"}), 400

        # Validación de tipos y valores numéricos
        if not isinstance(data["precio"], (int, float)) or data["precio"] < 0:
            return jsonify({"error": "El precio debe ser un número positivo"}), 400
        if not isinstance(data["stock"], int) or data["stock"] < 0:
            return jsonify({"error": "El stock debe ser un número entero positivo"}), 400
        if not isinstance(data["volumen"], (int, float)):
            return jsonify({"error": "El volumen debe ser un número "}), 400

        conn = conexionDB()
        cursor = conn.cursor()

        # Evitar duplicados por título
        cursor.execute("SELECT id FROM mangas WHERE titulo = ? AND volumen = ? AND id != ?", (data["titulo"], data["volumen"], id))
        if cursor.fetchone():
            conn.close()
            return jsonify({"error": "Este manga y volumen ya están registrados"}), 409

        # Verificamos la existencia de editorial
        cursor.execute("SELECT id FROM editoriales WHERE id = ?", (data["id_editorial"],))
        if not cursor.fetchone():
            conn.close()
            return jsonify({"error": "La editorial no existe"}), 404

        cursor.execute("SELECT * FROM mangas WHERE id = ?", (id,))
        if not cursor.fetchone(): #si no existe manda un mensaje de error
            conn.close()
            return jsonify({"error": "Manga no encontrado"}), 404

        #si si existe actualizara la informacion de los campos por los datos que haya ingresado el usuario
        cursor.execute("""
            UPDATE mangas
            SET titulo = ?, autor = ?, volumen = ?, precio = ?, stock = ?, id_editorial = ?
            WHERE id = ?
            """, (data["titulo"], data["autor"], data["volumen"], data["precio"], data["stock"], data["id_editorial"], id))
```

En el PUT, así como en los anteriores se utilizan las excepciones y las mismas validaciones del POST. Aquí también se debe especificar el id del registro que se desea modificar, posteriormente se captura la nueva información que el usuario está enviando en formato JSON, se valida mediante iteración que ningún campo este vacío o sea nulo, ante cualquier omisión, se devuelve un mensaje de error seguido de un código de estado. A continuación, se valida que el id del registro que deseas modificar exista, si no es el caso, se devuelve un mensaje de error especificando que el dato no fue encontrado. Una vez validado tanto la existencia del id y los datos que se buscan modificar del registro, se ejecuta la consulta del update para modificar la información solicitado y finalmente se cierra la conexión, posteriormente se despliega un mensaje de éxito. Como en anteriores casos, si llega a existir un problema para acceder al archivo de la base de datos o en la

ejecución de la consulta, por medio de una excepción se retornará un mensaje de error junto con un código de estado.

Método DELETE en mangas

```
#ELIMINAR MANGA, METODO DELETE
@app.route("/mangas/<int:id>", methods=["DELETE"])
def eliminar(id):
    try:
        conn = conexionDB()
        cursor = conn.cursor()
        #ejecuta la consulta SQL para eliminar el producto que coincida con el id ingresado por el usuario
        cursor.execute("DELETE FROM mangas WHERE id = ?", (id,))
        conn.commit() #guarda los cambios

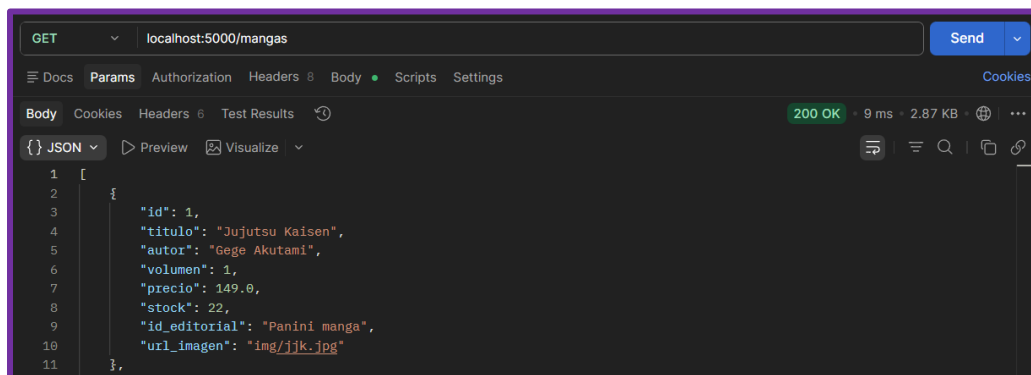
        if cursor.rowcount == 0: #cuenta el numero de fila afectada por la ultima consulta, si es cero el id no se encontro
            conn.close()
            return jsonify({"error": "ID no encontrado"}), 404

        conn.close()
        return jsonify({"mensaje": "Manga eliminado"}), 200 #si no, el manga fue eliminado
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

El delete que al igual que el put, se debe buscar el registro a eliminar mediante el id. Al igual que en el resto de los métodos explicados anteriormente, se hizo uso del manejo de excepciones. Dentro del try se ejecuta la consulta de delete para eliminar el registro del id solicitado, en caso de que este no exista se retornara un mensaje de error en conjunto con un código de estado donde se detalle el registro no fue encontrado. Si no hay ningún inconveniente, la base de datos se actualizara eliminando el registro, retornando un mensaje de éxito y un código de estado.

Capturas de postman para mangas

Get



```

12     {
13         "id": 2,
14         "titulo": "Spy x Family",
15         "autor": "Tatsuya Endo",
16         "volumen": 1,
17         "precio": 139.0,
18         "stock": 17,
19         "id_editorial": "Kamite",
20         "url_imagen": "img/spy x family.jpg"
21     },
22     {
23         "id": 3,
24         "titulo": "Berserk",
25         "autor": "Kentaro Miura",
26         "volumen": 1,
27         "precio": 299.0,
28         "stock": 2,
29         "id_editorial": "Panini manga",
30         "url_imagen": "img/berserk'.jpg"

```

GET por id

GET localhost:5000/mangas/2

Send

Docs Params Authorization Headers 8 Body • Scripts Settings Cookies

Body Cookies Headers 6 Test Results 200 OK • 9 ms • 385 B •

{ } JSON Preview Visualize

```

1  {
2      "id": 2,
3      "titulo": "Spy x Family",
4      "autor": "Tatsuya Endo",
5      "volumen": 1,
6      "precio": 139.0,
7      "stock": 17,
8      "id_editorial": "Kamite",
9      "url_imagen": "img/spy x family.jpg"

```

Get Por Nombre

GET localhost:5000/mangas/Jujutsu Kaisen

Send

Docs Params Authorization Headers 8 Body • Scripts Settings Cookies

Body Cookies Headers 6 Test Results 200 OK • 10 ms • 408 B •

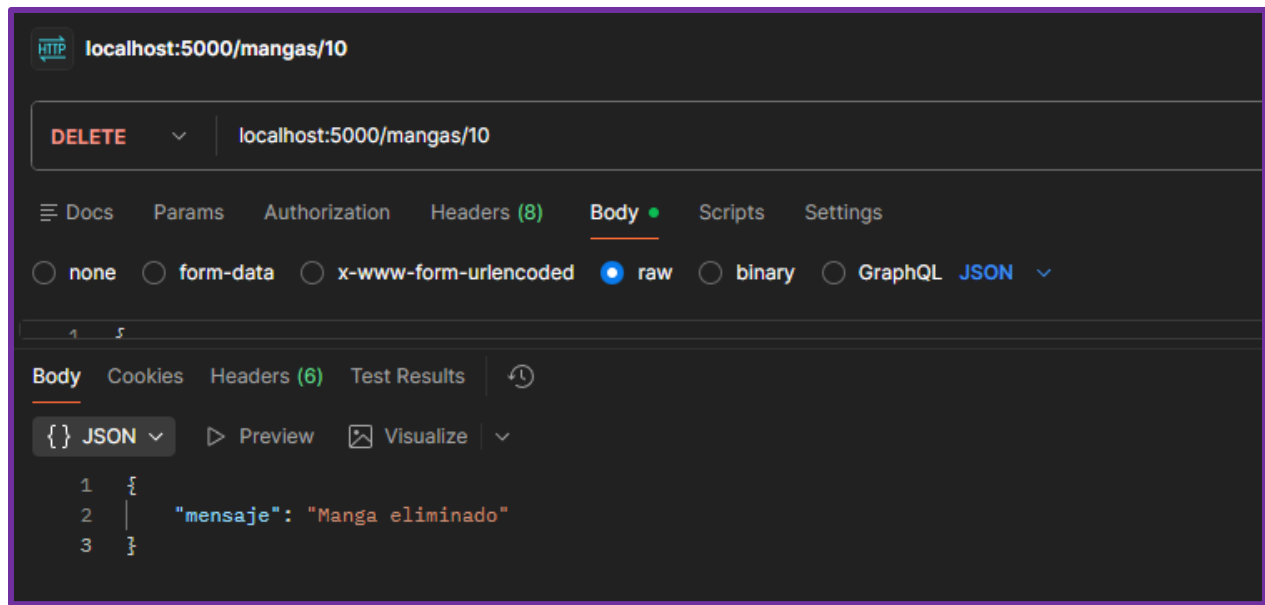
{ } JSON Preview Visualize

```

1  [
2      {
3          "id": 1,
4          "titulo": "Jujutsu Kaisen",
5          "autor": "Gege Akutami",
6          "volumen": 1,
7          "precio": 149.0,
8          "stock": 22,
9          "id_editorial": "Panini manga",
10         "url_imagen": "img/jjk.jpg"
11     }
12 ]

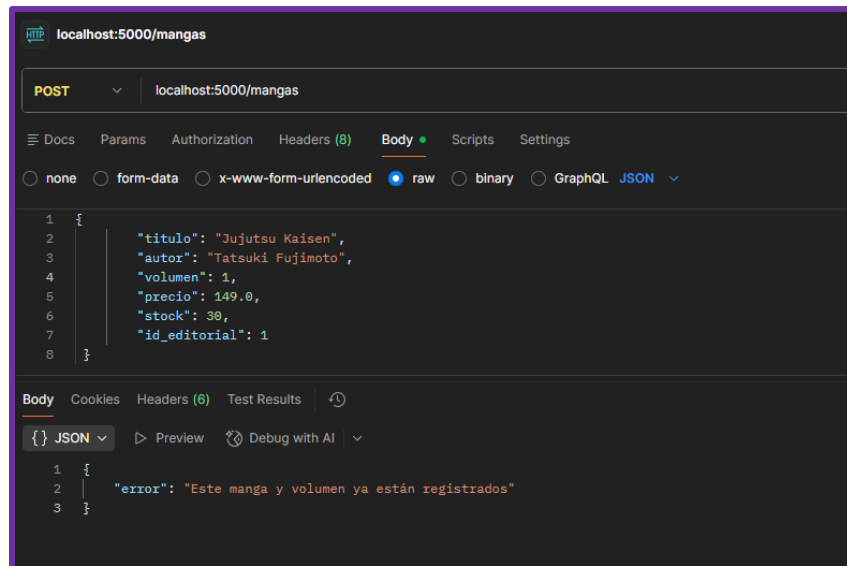
```

DELETE

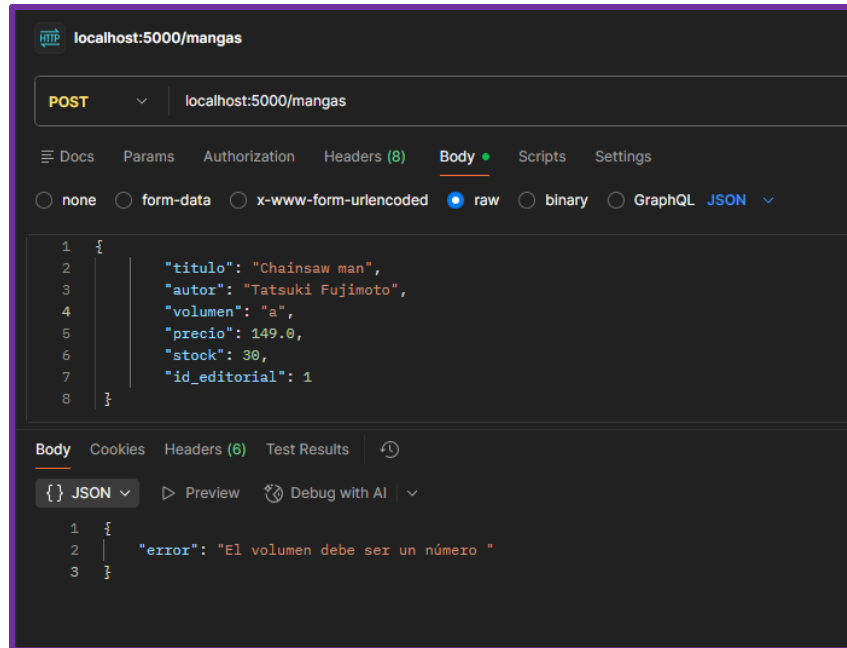


POST

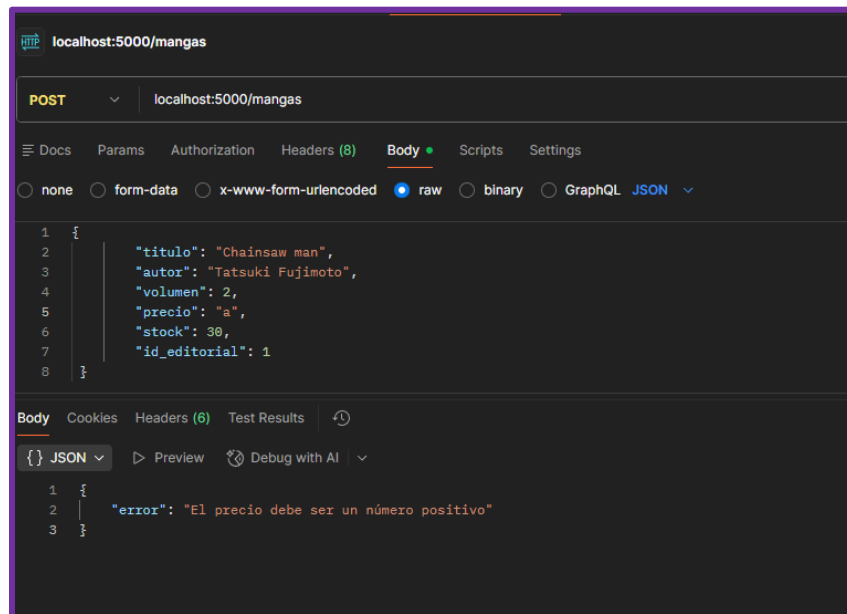
Validación de que el manga no este repetido



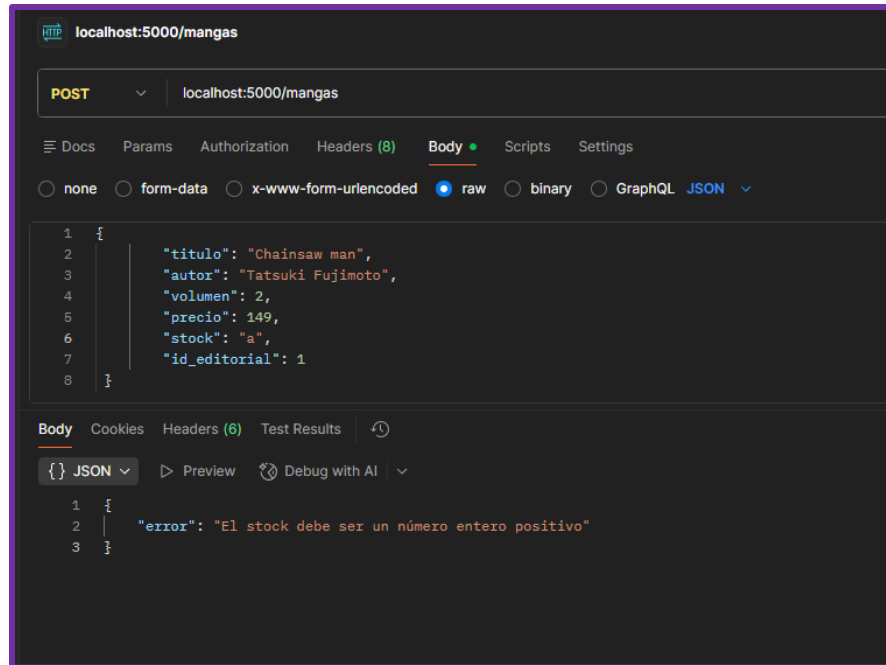
Validación de que el volumen debe ser un numero



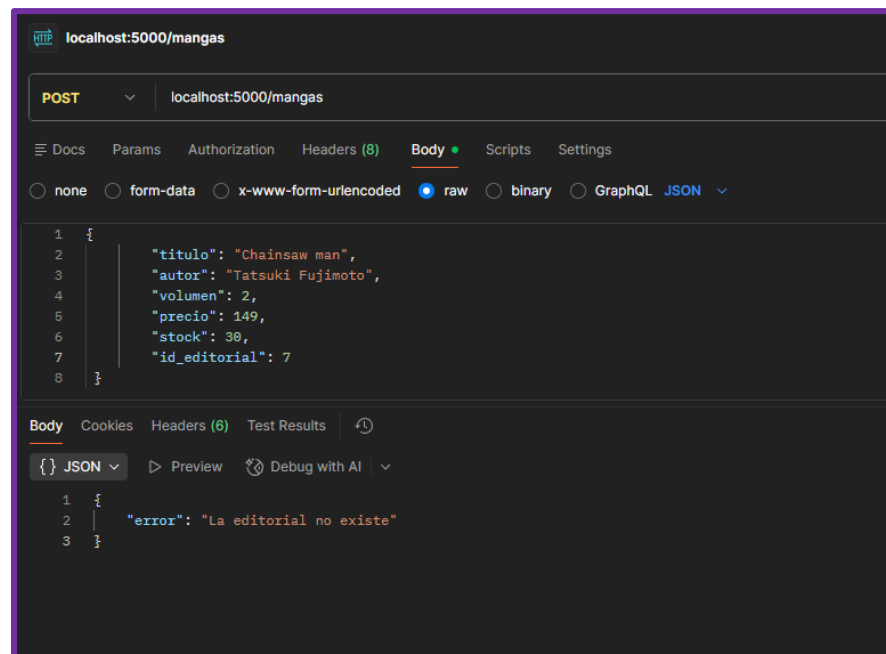
Validación de que el precio debe ser un numero positivo



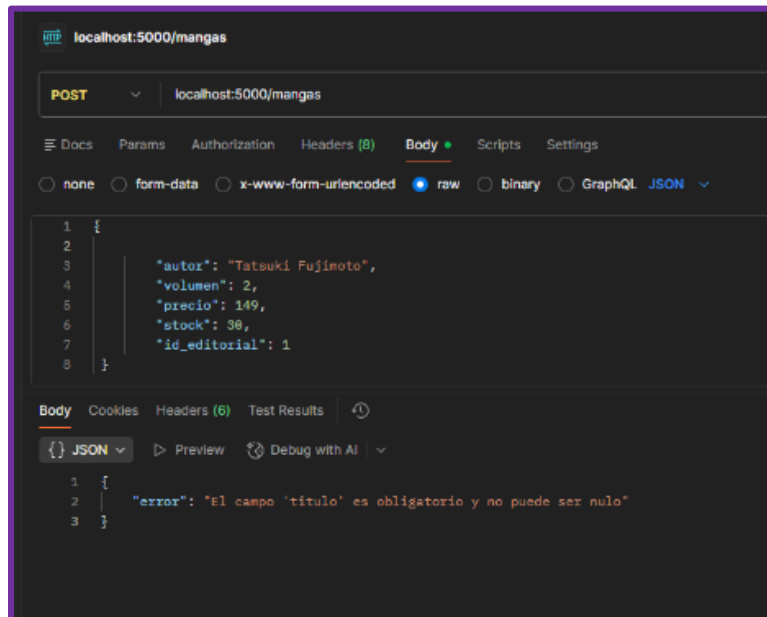
Validación de que el stock debe ser un numero positivo



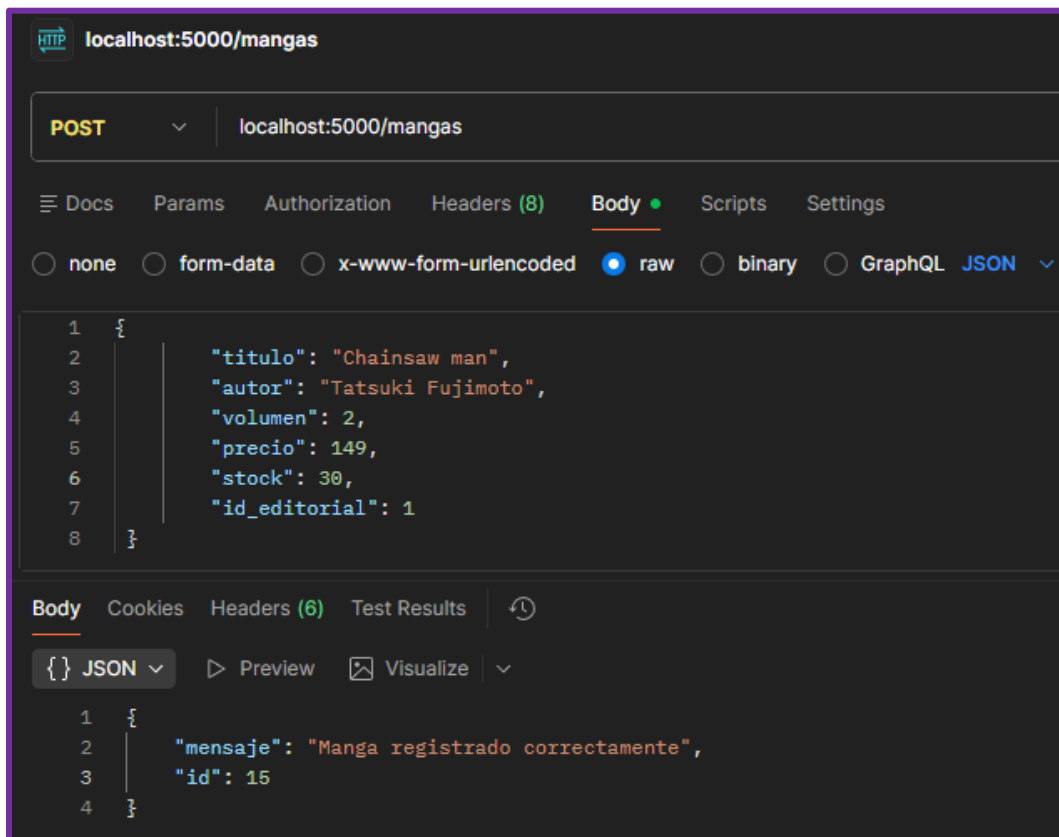
Validación de que debe existir la editorial



Validación de que cierto campo es obligatorio

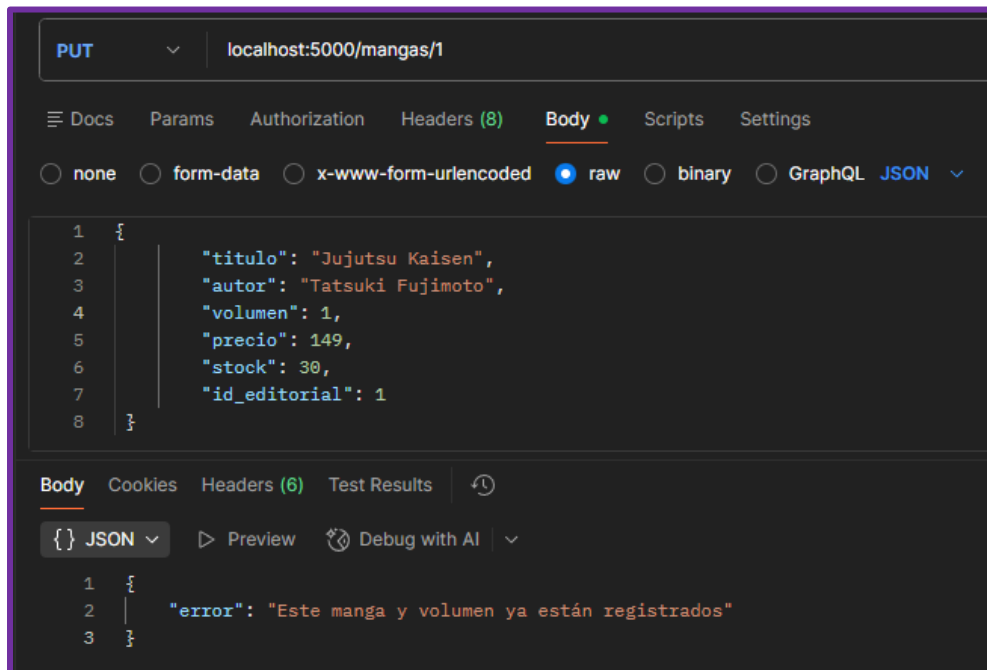


Post exitoso

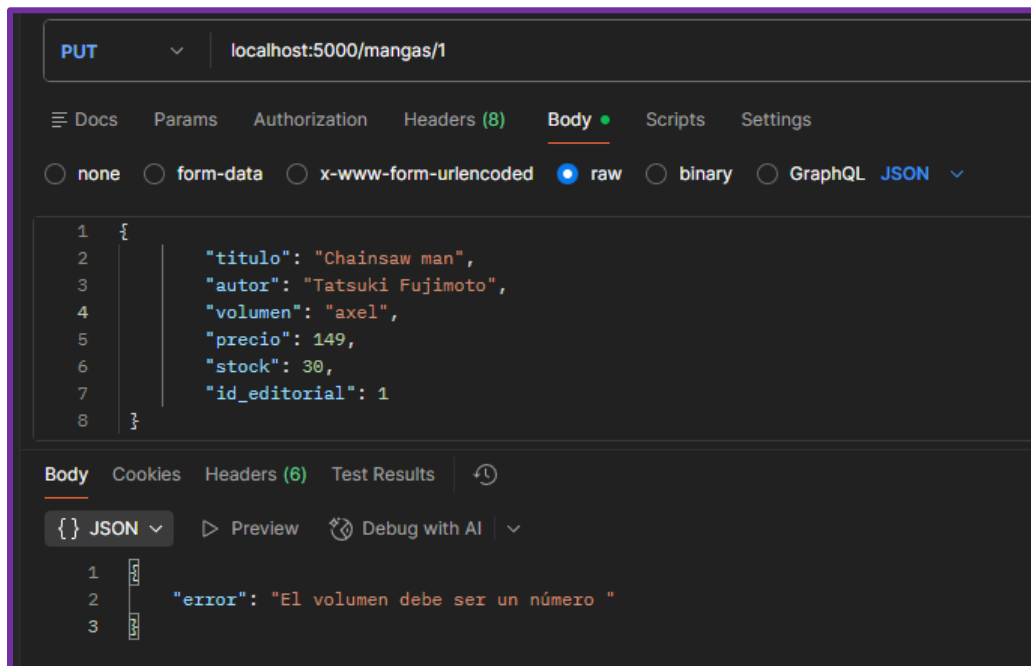


Put

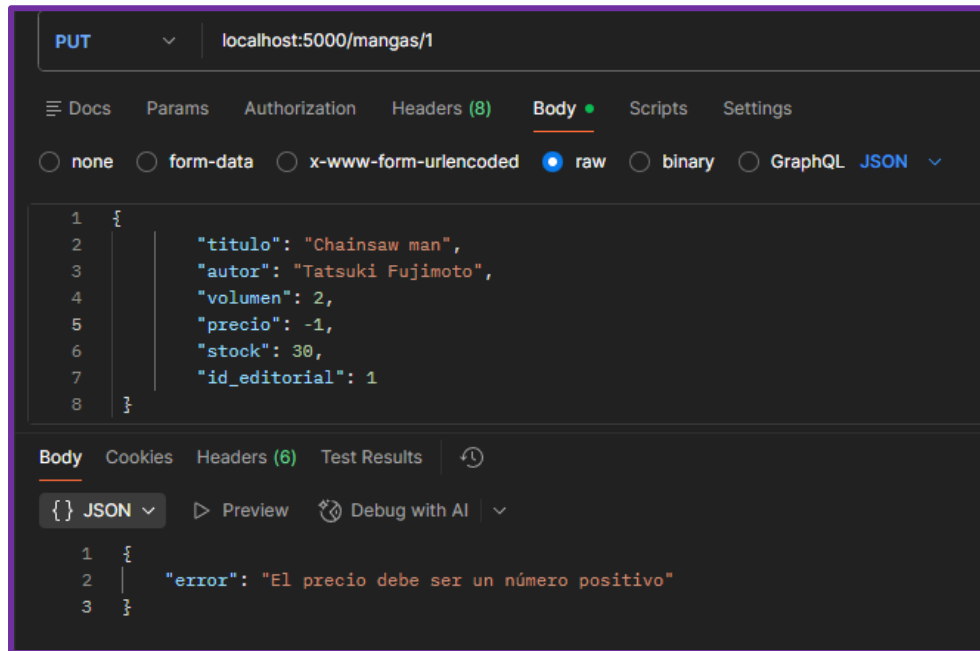
Validación de que el manga no este repetido



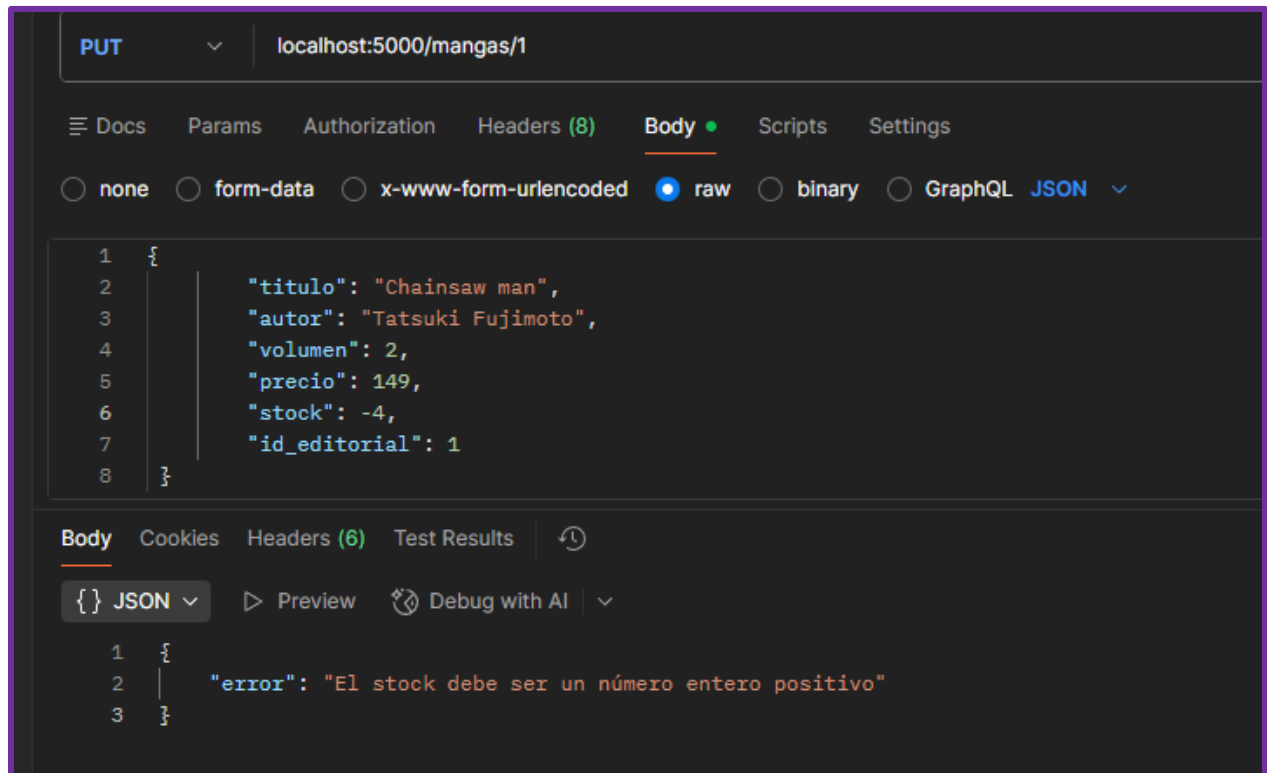
Validación de que el volumen debe ser un numero



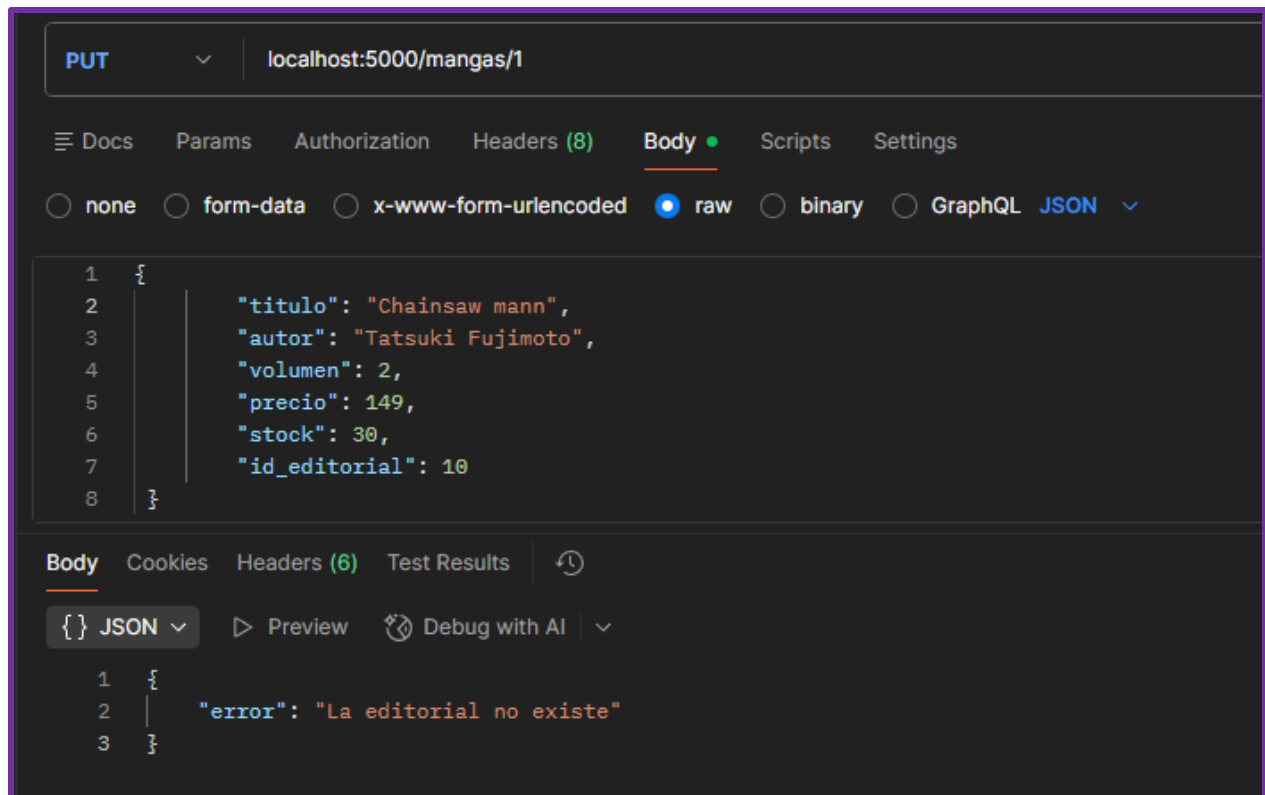
Validación de que el precio debe ser un numero positivo



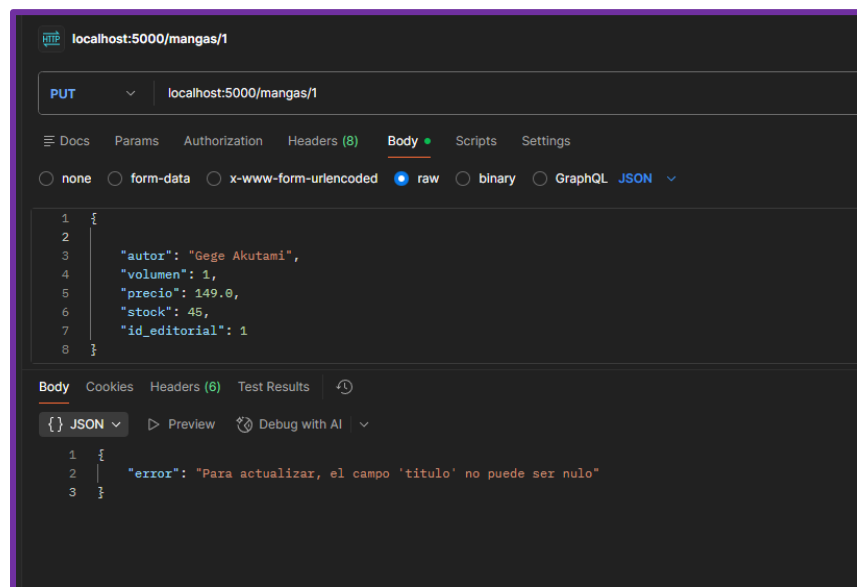
Validación de que el stock debe ser un numero positivo



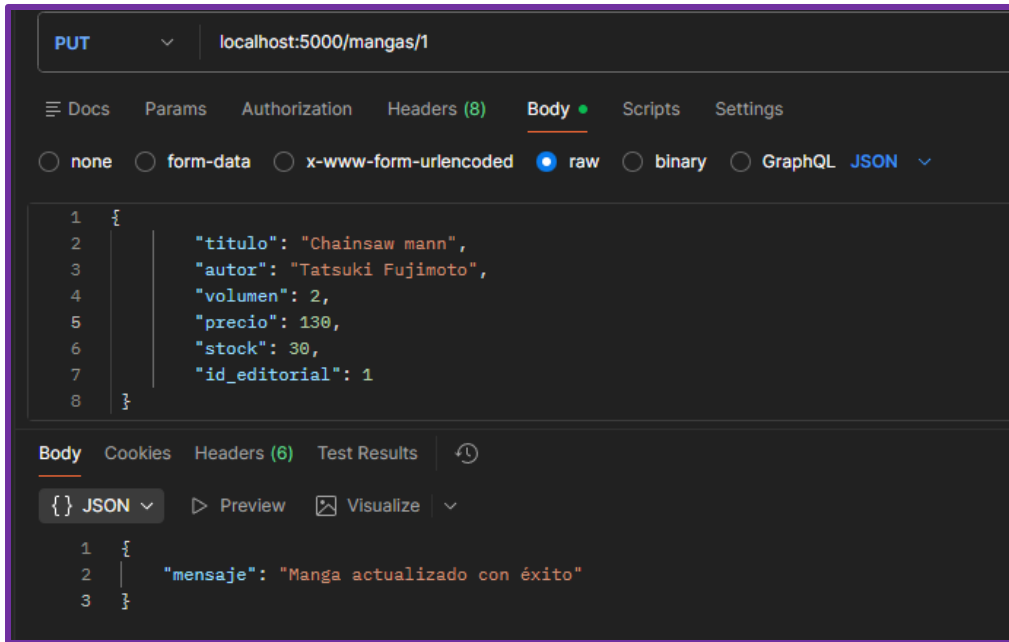
Validación de que existe la editorial



Validación de que un campo no puede ser nulo



Put exitoso



Método GET en editoriales

```
#Metodo GET editoriales
@app.route("/editoriales", methods=["GET"])
def obtenerEditoriales():
    try:
        conn = conexionDB()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM editoriales") #se seleccionan todos los registros de la tabla "editoriales"
        filas = cursor.fetchall()

        editoriales = [{ "id": f[0], "nombre": f[1], "pais": f[2] } for f in filas] #acomodamos los resultados en una lista de diccionarios para cada registro

        conn.close()
        return jsonify(editoriales), 200 #regresamos la lista en formato json
    except Exception as e:
        return jsonify({ "error": str(e) }), 500
```

Usando también el manejo de excepciones, dentro del bloque try abrimos una conexión a la base de datos y con el cursor ejecutamos la consulta que nos da todos los registros de la tabla editoriales, luego con un ciclo for los acomodamos en diccionarios dentro de una lista, cerramos la conexión a la base de datos y regresamos los resultados en formato json, si la operación no fue exitosa el sistema desplegará un mensaje de error.

Método GET en editoriales por id

```
#Metodo GET editoriales por id
@app.route("/editoriales/<int:id>", methods=["GET"])
def obtenerEditorialesId(id):
    try:
        conn = conexionDB()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM editoriales WHERE id = ?", (id,)) #se selecciona el registro que coincida con el id ingresado
        fila = cursor.fetchone()

        editoriales = {"id": fila[0], "nombre": fila[1], "pais": fila[2]} #acomodamos los resultados en un diccionario

        conn.close()
        return jsonify(editoriales), 200 #regresamos los resultados en formato json
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Este método funciona exactamente de la misma manera que el anterior, lo único que cambia es la consulta, la consulta busca el id que coincida con el que inserto el usuario

Método GET en editoriales por Nombre

```
@app.route("/editoriales/nombre/<string:nombre>", methods=["GET"])
def obtenerEditorialesNom(nombre):
    try:
        conn = conexionDB()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM editoriales WHERE LOWER(nombre) LIKE LOWER(?)", (f"%{nombre}%",))
        #se selecciona el registro que coincida con el id ingresado
        fila = cursor.fetchone()

        if not fila: #Verifica si encontro resultados
            return jsonify({#Si no encontro envia un mensaje de que no fueron encontradas las editoriales
                "mensaje": "No se encontro la editorial"
            }), 404

        editoriales = {"id": fila[0], "nombre": fila[1], "pais": fila[2]} #acomodamos los resultados en un diccionario

        conn.close()
        return jsonify(editoriales), 200 #regresamos los resultados en formato json
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Este método permite filtrar las búsquedas por medio del nombre, la query va buscar las editoriales mediante su nombre, al igual que en el resto de los métodos el proceso se coloca dentro de un try catch para asegurar que en caso de que no pueda establecer conexión con la base de datos o esta conexión se pierda durante el proceso.

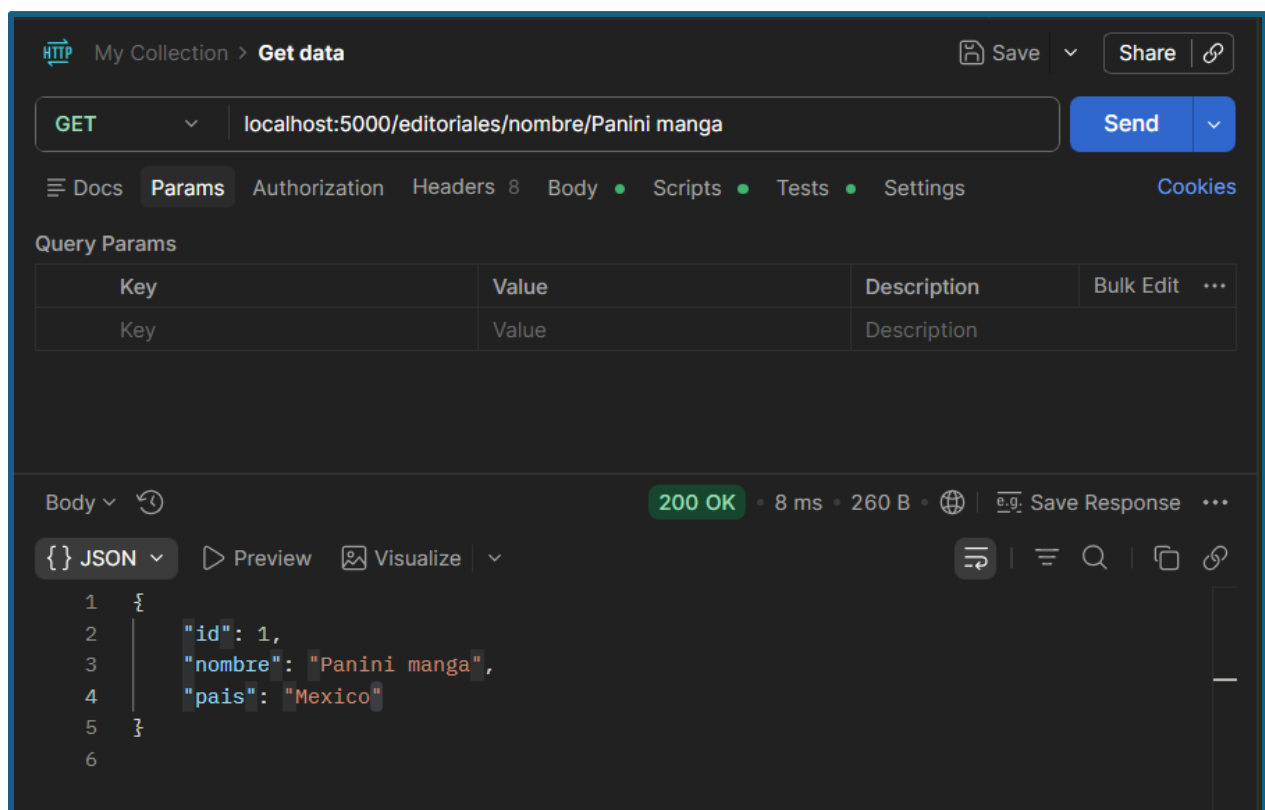
En la consulta SQL se utiliza la cláusula LIKE en combinación con la función LOWER(), lo que permite realizar una búsqueda flexible (coincidencias parciales) e insensible a mayúsculas o minúsculas.

En caso de que la editorial que se ingresa no este en la bd , se lanzara un mensaje por medio de la validación.

Los datos que recolecta el fetch se mapean en un diccionario de nombre editoriales y se cierra la conexión con la base de datos.

El edpoint devolverá los datos de la editorial que se buscó.

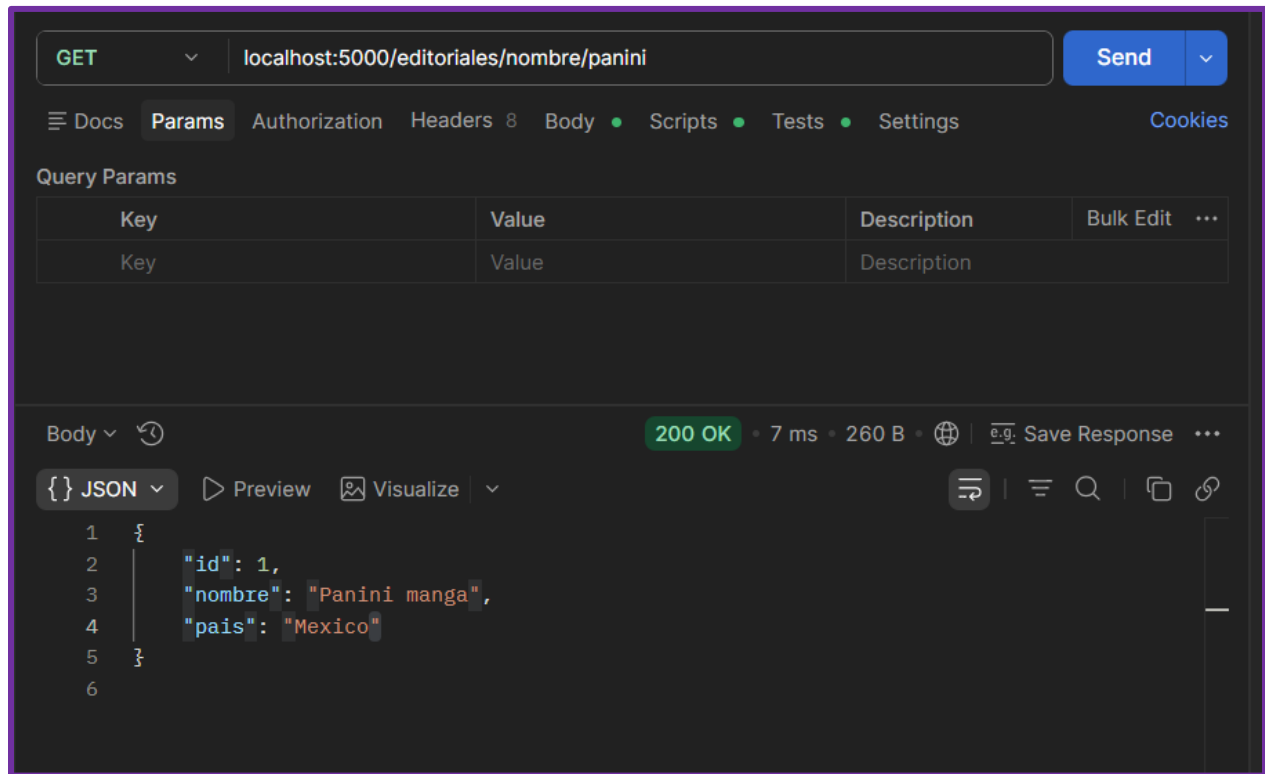
Prueba en Postman para editoriales get por nombre:



Ejemplo del uso de LIKE

Este ejemplo señala que como se mencionó, va a encontrar la editorial **panini** en la bd por medio de las coincidencias, es decir que no es necesario escribir el nombre tal completo tal cual como esta en la bd.

NOTA: Aunque se haga uso de la cláusula LIKE para facilitar las búsquedas, aun así, es necesario que el nombre se parezca en lo posible, ya que, de no ser así, no podrá encontrar la editorial que se busca.



Forma incorrecta de realizar la búsqueda:

GET localhost:5000/editoriales/nombre/paimi Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 404 NOT FOUND • 5 ms • 252 B Save Response

{ } JSON Preview Debug with AI

```
1 {
2   "mensaje": "No se encontro la editorial"
3 }
4
```

Método GET en editoriales por Pais

```
@app.route("/editoriales/pais/<string:pais>", methods=["GET"])
def obtenerEditorialesPais(pais):
    try:
        conn = conexionDB() #Se abre conexion con la base de datos
        cursor = conn.cursor() #Se crea un cursor que realice consultas sql

        cursor.execute("SELECT * FROM editoriales WHERE LOWER(pais) LIKE LOWER(?)", (f"%{pais}%",))
        #La query es para buscar las editoriales por pais ,el parametro de busqueda (pais) el lower lo convertira en minusculas
        #y el like , sirve para la busqueda de coincidencias parciales
        #f"%{pais}%", en esta parte se manda el parametro
        filas = cursor.fetchall() #Va traer todos las editoriales que tengan el mismo pais de origen

        if not filas: #Verifica si encontro resultados
            return jsonify({"mensaje": "No se encontraron editoriales"}), 404

        editoriales = [] #Crea una lista vacia para almacenar los datos que encuentre

        for fila in filas: #En este for se llenamos la lista con la informacion
            editoriales.append({
                "id": fila[0],
                "nombre": fila[1],
                "pais": fila[2]
            })

        conn.close() #Se cierra la conexion con la bd
        return jsonify(editoriales), 200 #regresamos los resultados en formato json
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Este método como el anterior permite filtrar las búsquedas por medio del país, la query va a buscar las editoriales mediante el país al igual que en el resto de los métodos el proceso se coloca dentro de un try catch para asegurar que en caso de que no pueda establecer conexión con la base de datos o esta conexión se pierda durante el proceso.

En la consulta SQL se utiliza la cláusula LIKE en combinación con la función LOWER (), lo que permite realizar una búsqueda flexible (coincidencias parciales) e insensible a mayúsculas o minúsculas.

En caso de que ninguna editorial este en el país que se busca, se lanzara un mensaje por medio una validación.

Los datos que recolecta el fetch se mapean en una lista de nombre editoriales y se cierra la conexión con la base de datos.

El endpoint devolverá los datos de las editoriales que tengan el mismo país de origen.

Prueba en Postman para editoriales por país:

GET localhost:5000/editoriales/pais/Mexico Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 8 ms • 342 B • Save Response

JSON Preview Visualize

```
1  [
2    {
3      "id": 1,
4      "nombre": "Panini manga",
5      "pais": "Mexico"
6    },
7    {
8      "id": 4,
9      "nombre": "Kamite",
10     "pais": "Mexico"
11   }
12 ]
13
```

Prueba haciendo uso del LIKE

GET

localhost:5000/editoriales/pais/mexico

Docs Params Authorization Headers 8 Body Scripts Tests Settings

Query Params

Key	Value	Description
Key	Value	Description

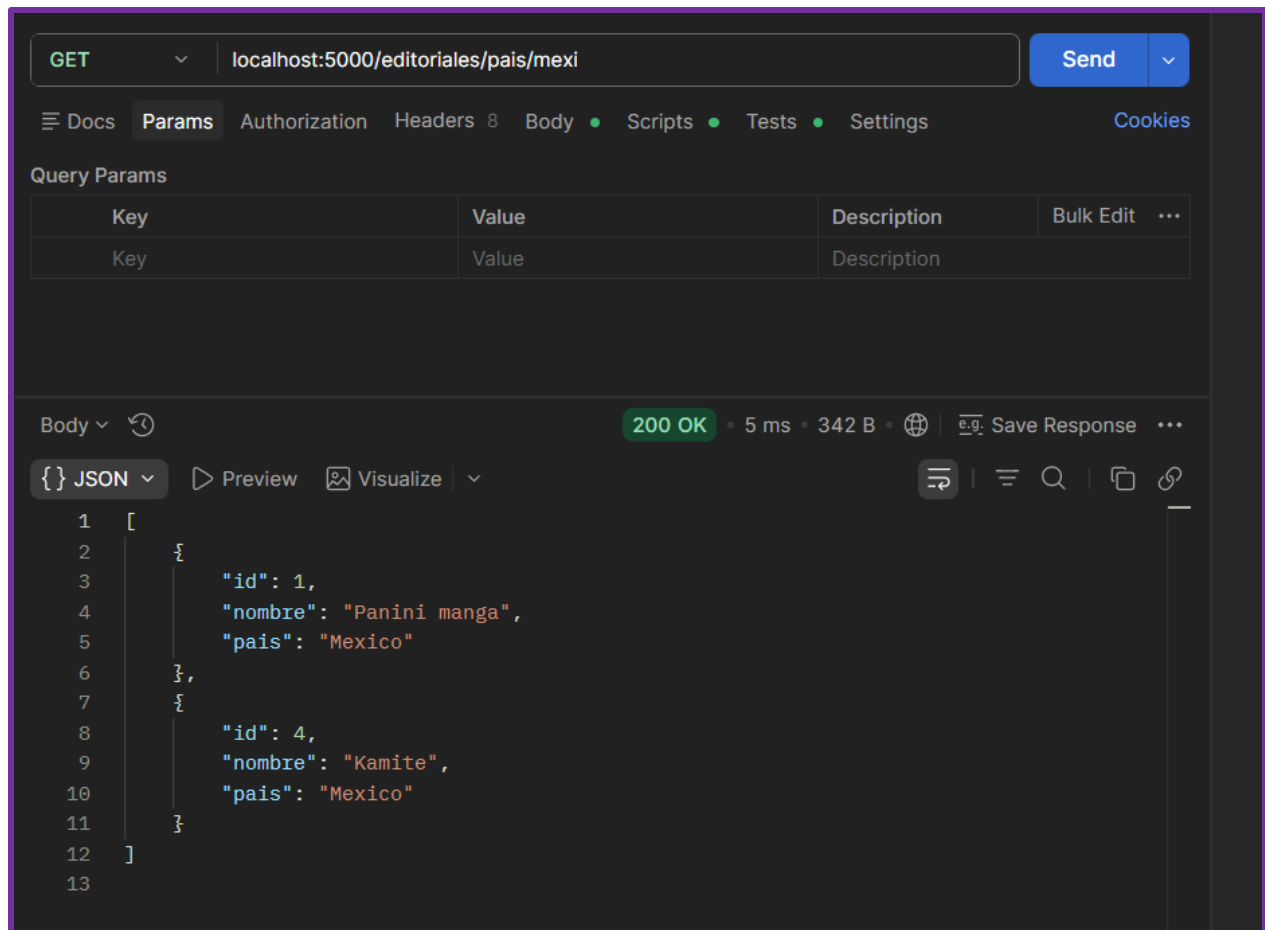
Body

200 OK • 7 ms • 342 B • e.g.

{ } JSON

Preview Visualize

```
1  [  
2    {  
3      "id": 1,  
4      "nombre": "Panini manga",  
5      "pais": "Mexico"  
6    },  
7    {  
8      "id": 4,  
9      "nombre": "Kamite",  
10     "pais": "Mexico"  
11   }  
12 ]  
13
```



NOTA: Para el endpoint de búsqueda por país, si algún país que estas buscando tiene acento; por ejemplo: Si en una editorial estableces que su lugar de origen es México, escrito de esta manera y posteriormente vuelves a ingresar otra editorial y le colocas Mexico, al realizar la búsqueda ingresando la palabra “**mexico**”, no te traerá la editorial cuyo país tenga el acento.

Método POST en editoriales

```
#Metodo POST editoriales
@app.route("/editoriales", methods=["POST"])
def insertarEditorial():
    try:
        data = request.json #obtenemos los datos insertados por el usuario
        if "nombre" not in data or "pais" not in data: #si el nombre o pais no se encuentran lanza un mensaje de error, estos campos son obligatorios
            return jsonify({"error": "Los campos 'nombre' y 'pais' son obligatorios"}), 400

        # Validación de tipos y valores
        if not isinstance(data["pais"], (str)):
            return jsonify({"error": "el pais no puede ser un numero"}), 400

        conn = conexionDB()
        cursor = conn.cursor()

        # Evitar duplicados por nombre y pais
        cursor.execute("SELECT id FROM editoriales WHERE nombre = ? AND pais = ?", (data["nombre"], data["pais"]))
        if cursor.fetchone():
            conn.close()
            return jsonify({"error": "Esta editorial ya esta repetida"}), 409

        cursor.execute("INSERT INTO editoriales (nombre, pais) VALUES (?, ?)", (data["nombre"], data["pais"])) #insertamos en la tabla la nueva editorial
        conn.commit()
        nuevo_id = cursor.lastrowid #obtenemos el id que la base de datos le asigno a la editorial que agregamos
        conn.close()

        return jsonify({"mensaje": "Editorial registrada", "id": nuevo_id}), 201 #mensaje de que se agrego exitosamente
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Para el método post, dentro del bloque de excepciones se obtiene los datos insertados por el usuario por medio del request y luego se valida que se encuentren los campos obligatorios, en caso de que no lo estén lanza un mensaje de error, luego de verificarlo abrimos una conexión a la base de datos y verificamos ahora que los datos no estén repetidos, si no están repetidos entonces insertamos dentro de la base de datos “editoriales” la nueva editorial , guardamos los cambios y lanzamos un mensaje de éxito en caso de que no haya habido ningún inconveniente, en caso de que si el sistema lanza un mensaje error por medio del bloque de excepción

Método PUT en editoriales

```
#Metodo PUT editoriales
@app.route("/editoriales/<int:id>", methods=["PUT"])
def actualizarEditorial(id):
    try:
        data = request.json
        if "nombre" not in data or "pais" not in data:
            return jsonify({"error": "Los campos 'nombre' y 'pais' son obligatorios"}), 400

        # Validación de tipos y valores
        if not isinstance(data["pais"], (str)):
            return jsonify({"error": "el pais no puede ser un numero"}), 400

        conn = conexionDB()
        cursor = conn.cursor()

        # Evitar duplicados por nombre y pais
        cursor.execute("SELECT id FROM editoriales WHERE nombre = ? AND pais = ?", (data["nombre"], data["pais"]))
        if cursor.fetchone():
            conn.close()
            return jsonify({"error": "Esta editorial ya esta repetida"}), 400

        cursor.execute("UPDATE editoriales SET nombre = ?, pais = ? WHERE id = ?", (data["nombre"], data["pais"], id)) #actualizamos la editorial con los datos ingresado por el usuario
        conn.commit()

        #si ninguna fila fue afectada significa que el id de la editorial no se encontro
        if cursor.rowcount == 0:
            conn.close()
            return jsonify({"error": "Editorial no encontrada"}), 404

        conn.close()
        return jsonify({"mensaje": "Editorial actualizada"}), 200 #mensaje de que si se pudo actualizar correctamente
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

En el método PUT se usa el mismo método de validación de campos y de repetición que en el post, luego actualizaremos la base de datos con los nuevos datos y veremos si alguna fila de la tabla editoriales fue afectada, esto nos ayuda a saber si el id de la editorial que ingreso el usuario si existe o no, lanzando un mensaje de éxito si la actualización fue exitosa o un mensaje de error en caso de una excepción.

Método DELETE en editoriales

```
#Metodo DELETE editoriales
@app.route("/editoriales/<int:id>", methods=["DELETE"])
def eliminarEditorial(id):
    try:
        conn = conexionDB()
        cursor = conn.cursor()

        # Integridad referencial: No borrar si tiene mangas
        cursor.execute("SELECT COUNT(*) FROM mangas WHERE id_editorial = ?", (id,))
        if cursor.fetchone()[0] > 0:
            conn.close()
            return jsonify({"error": "No se puede eliminar: existen mangas asociados a esta editorial"}), 400

        cursor.execute("DELETE FROM editoriales WHERE id = ?", (id,)) #elimina la editorial que coincida con el id ingresado
        conn.commit()

        #si ninguna fila fue afectada significa que el id de la editorial no se encontro
        if cursor.rowcount == 0:
            conn.close()
            return jsonify({"error": "Editorial no encontrada"}), 404

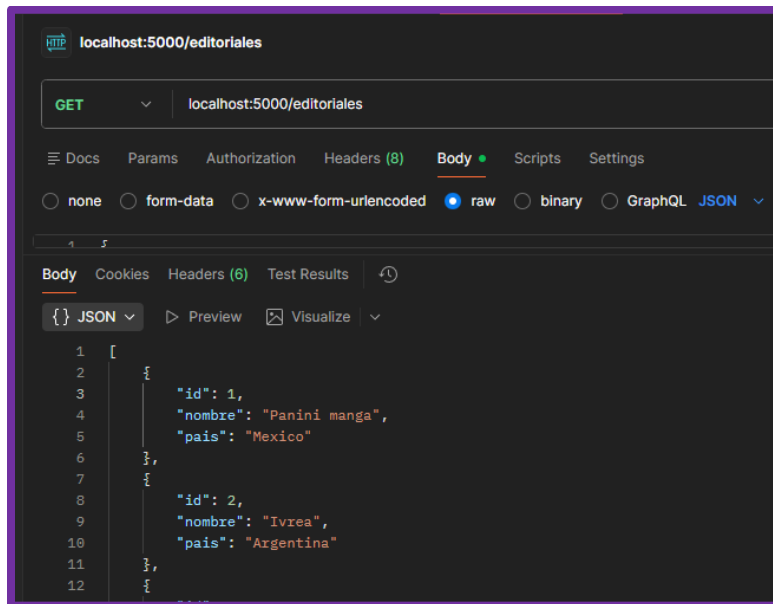
        conn.close()
        return jsonify({"mensaje": "Editorial eliminada"}), 200 #mensaje de que se elimino la editorial exitosamente
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Lo primero que hará el método para eliminar una editorial es verificar que otros mangas no estén vinculados a la editorial, esto con el fin de no dejar datos huérfanos, en caso de que un manga este vinculado a la editorial que se quiere eliminar se mostrara un mensaje de error diciendo que no se puede eliminar esa editorial, en caso de que la editorial este libre se ejecutara el scrip para eliminar de la base de datos la editorial que coincida con

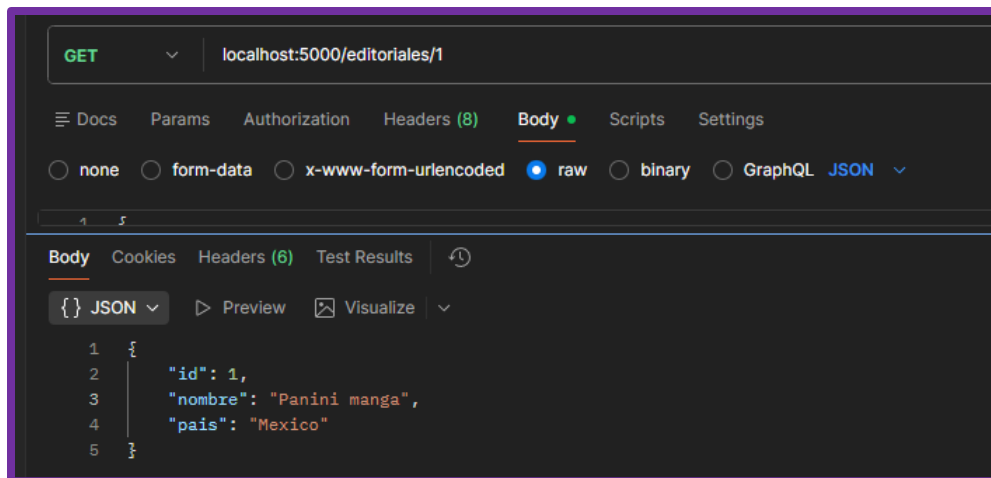
el id que ingreso el usuario, se verificara que haya habido cambios en la tabla editoriales y se mostrara un mensaje de éxito si si hubo cambios o de error en caso de que la editorial no fuera encontrada

Capturas de postman para editoriales

GET

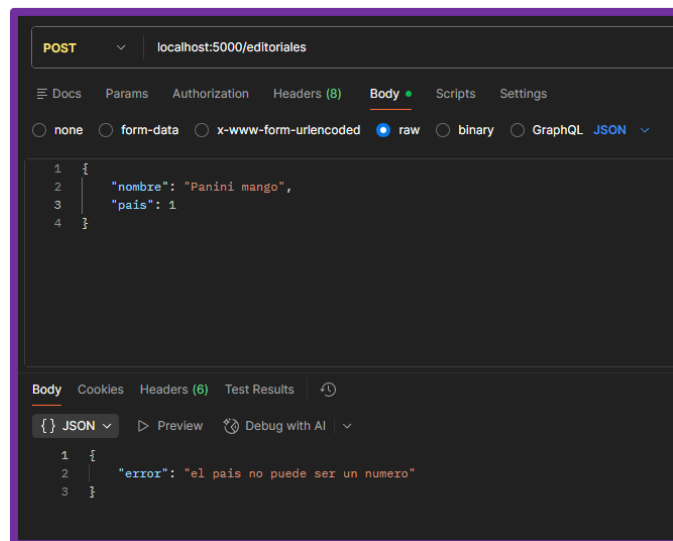


Get por id

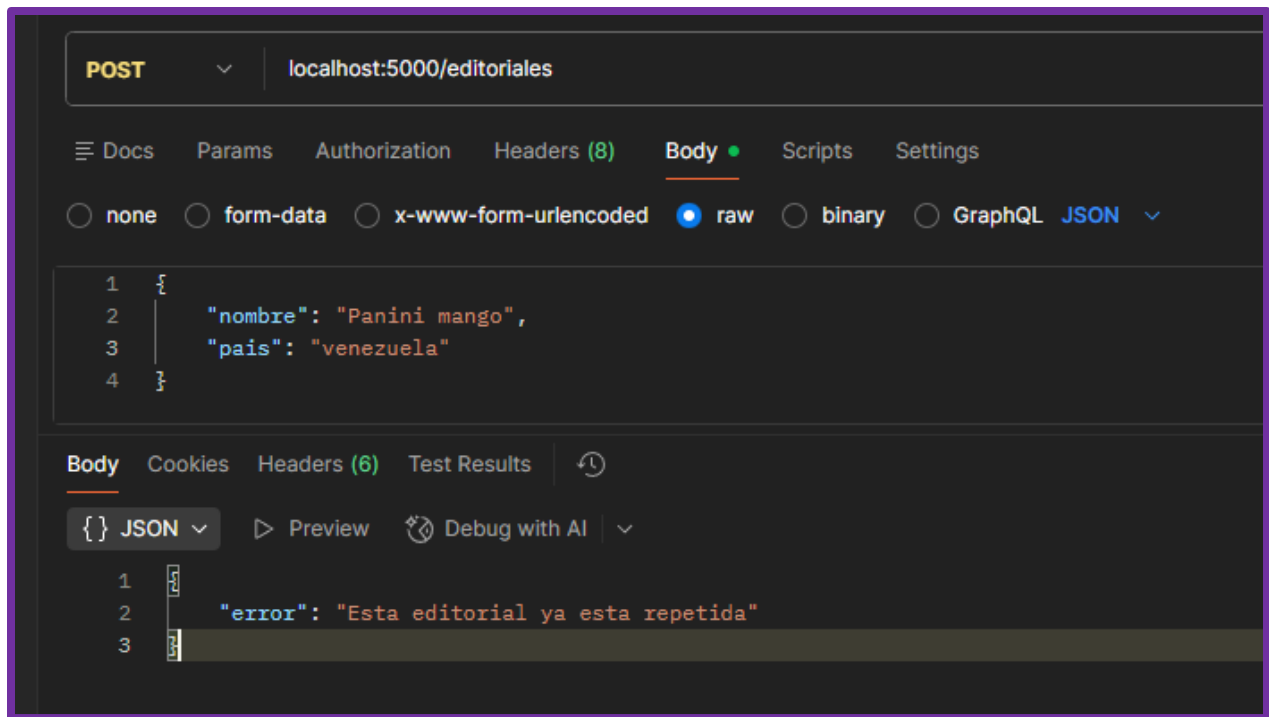


post

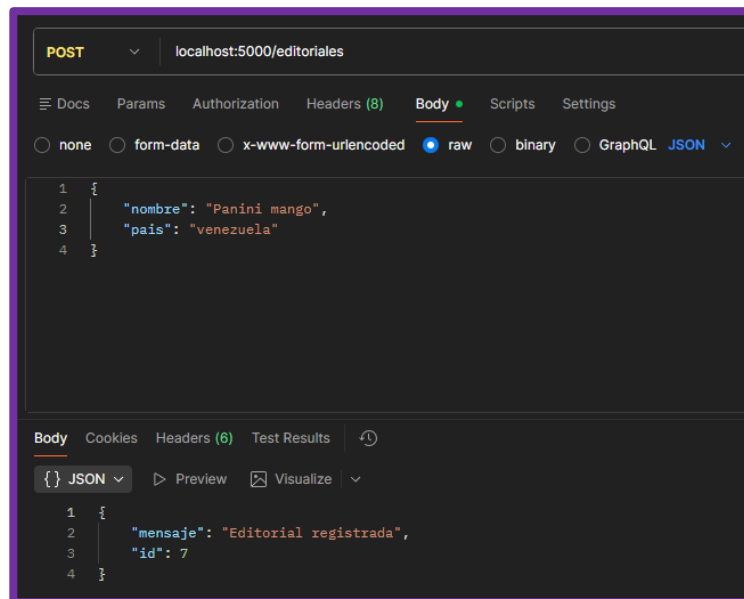
validación de que el país no puede ser un numero



Validación de que no puede haber editoriales repetidas

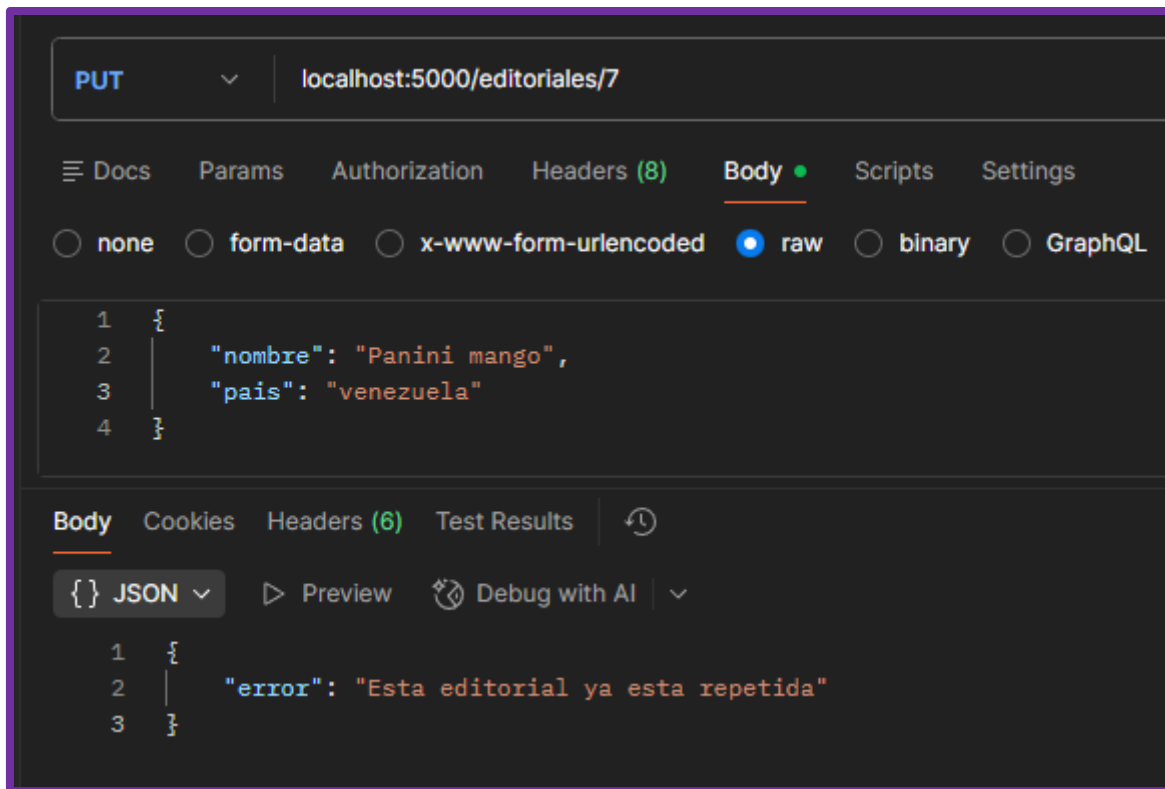


POST exitoso

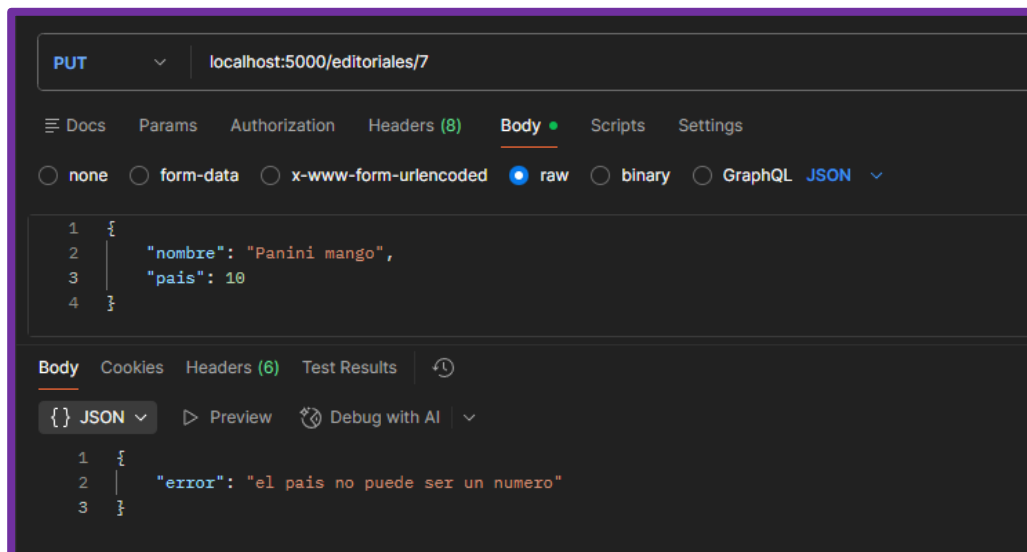


PUT

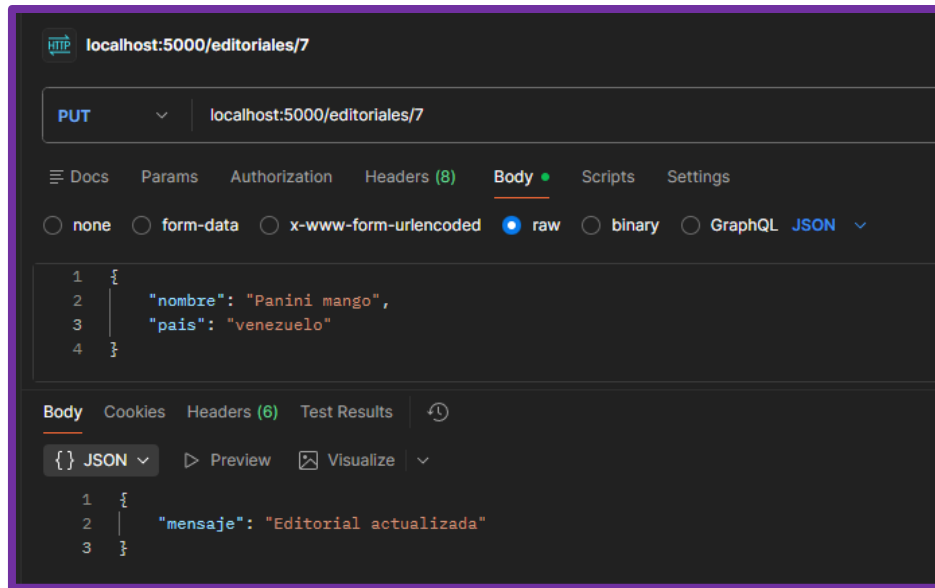
Validación de que no puede haber editoriales repetidas



Validar que el país no sea un numero

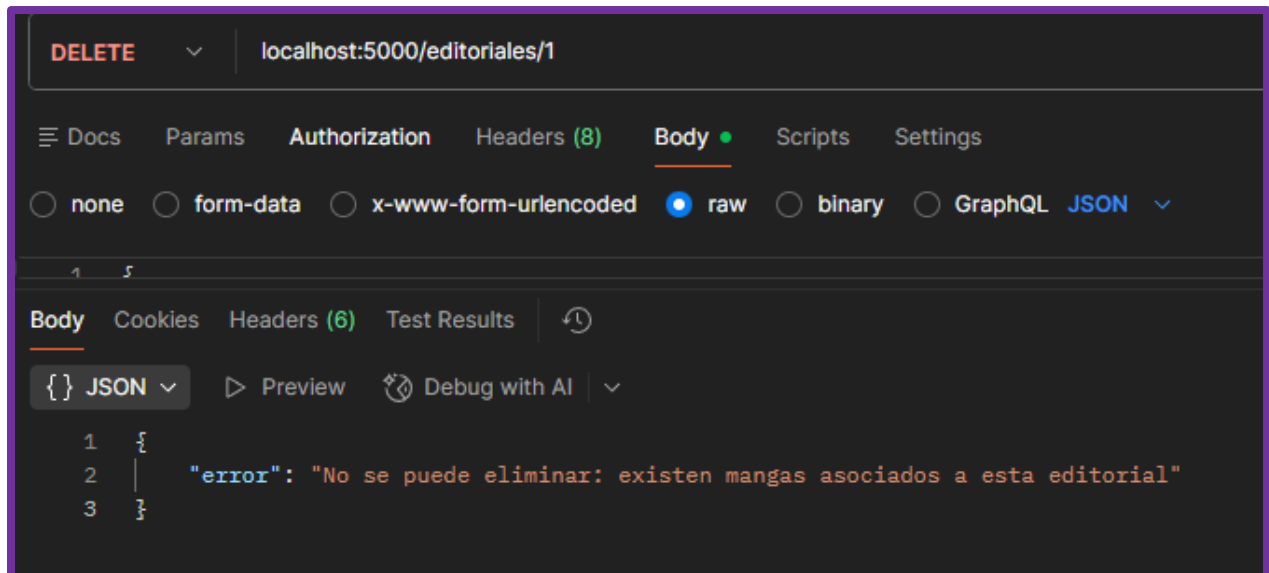


Put exitoso

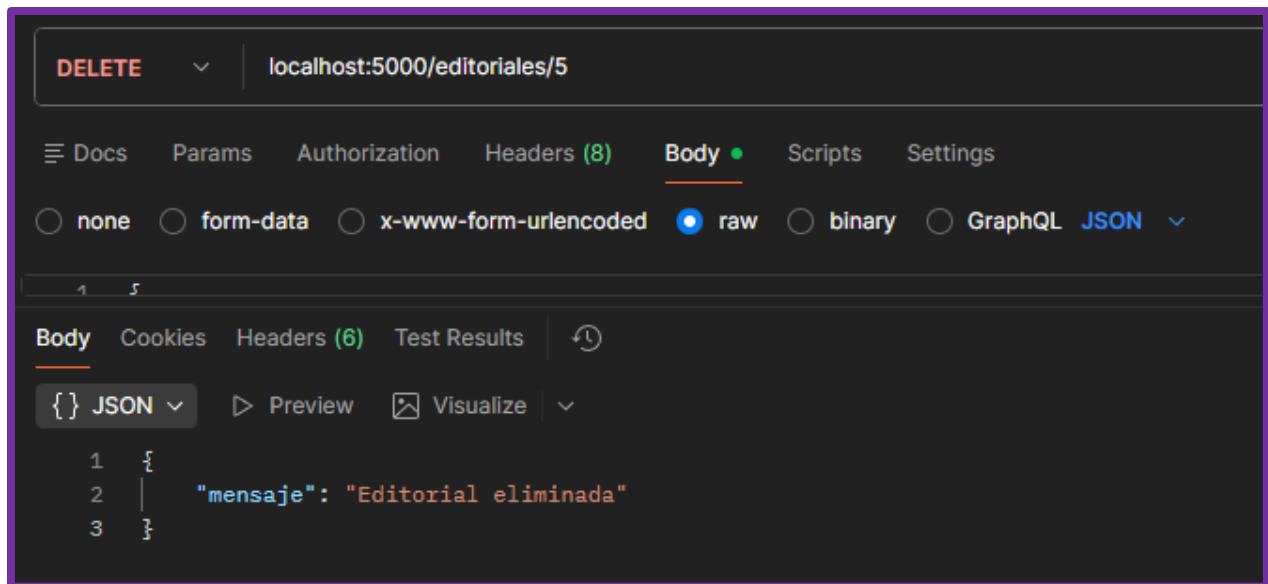


DELETE

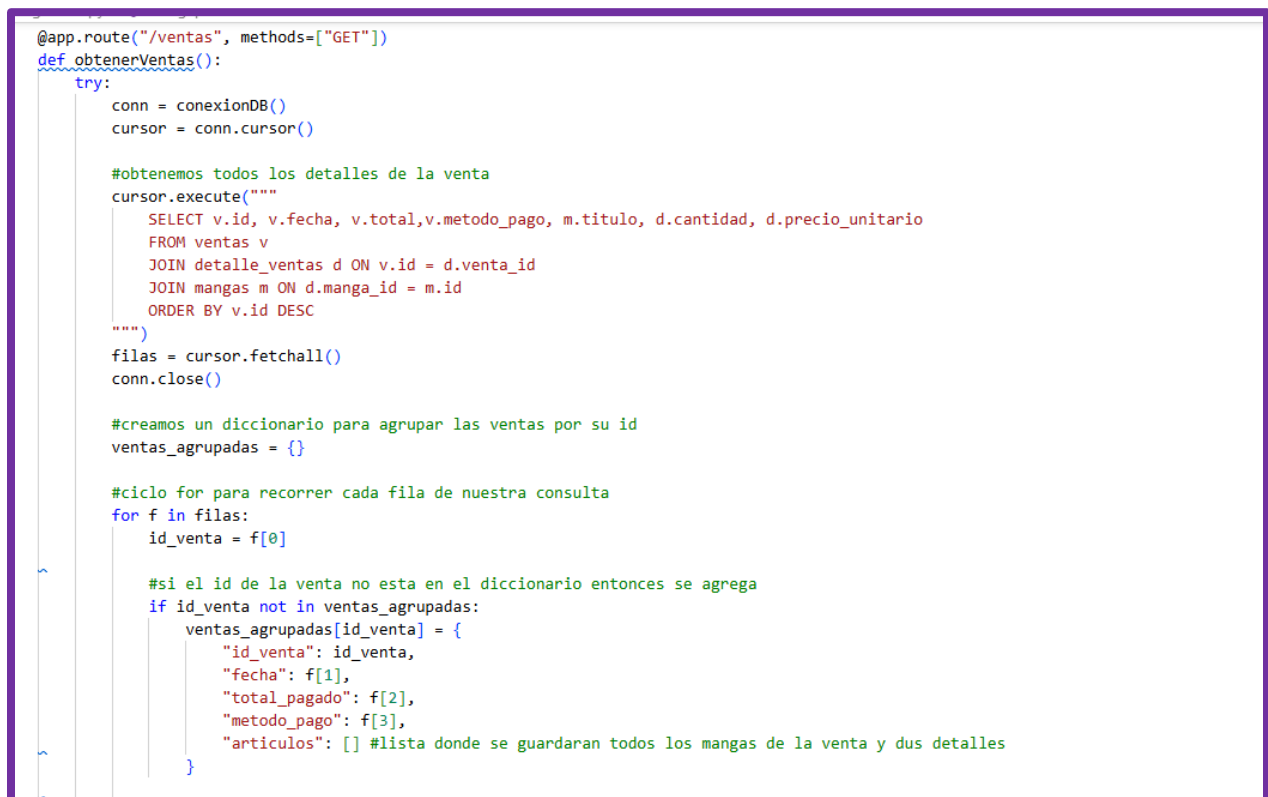
Verificar que no se puedan eliminar editoriales que tengan mangas vinculados



DELETE exitoso



METODO GET en ventas



```

# aqui agregamos los mangas y sus detalles a la lista articulos
ventas_agrupadas[id_venta]["articulos"].append({
    "manga": f[4],
    "cantidad": f[5],
    "precio_unitario": f[6],
    "subtotal": f[5] * f[6]
})

# transformamos los valores del diccionario a una lista para que nos la regrese en json
return jsonify(list(ventas_agrupadas.values())), 200

except Exception as e:
    return jsonify({"error": str(e)}), 500

```

A diferencia de los otros métodos get que hemos explicado en el documento, este es mas elaborado y detallado, pues trabajaremos en conjunto con nuestras 3 tablas de la base de datos para obtener todos los datos relevantes que conlleva una venta, por ejemplo, los datos que nos interesa de una venta son: el id de la venta, la fecha en que se llevó a cabo, el total, el método de pago, título de los mangas que se vendieron, la cantidad de dichos mangas y el precio unitario del producto, no podemos encontrar todos esos datos en la tabla “ventas”, por eso en la consulta usamos un join para obtener el resto de datos de la tabla “mangas” y “detalles_ventas”, crearemos un diccionario para agrupar cada una de las ventas por separado donde guardaremos todos los datos ya mencionados, aparte también se usara una lista dentro del diccionario para guardar los detalles de cada uno de los artículos de la ventas, luego simplemente retornamos esto al usuario en formato JSON para su fácil lectura.

Metodo GET por id en ventas

```
@app.route("/ventas/<int:id>", methods=["GET"])
def obtenerVentaPorId(id):
    try:
        conn = conexionDB()
        cursor = conn.cursor()

        #consulta que nos traer el id de la venta que coincida con la busqueda
        cursor.execute("SELECT id, fecha, total, metodo_pago FROM ventas WHERE id = ?", (id,))
        venta = cursor.fetchone()

        #si el id de la venta no se encontro entonces regresa una mensaje de venta no encontrada
        if not venta:
            conn.close()
            return jsonify({"error": "Venta no encontrada"}), 404

        #si si se encontro el id usamos una consulta para que nos traiga todos los datos de esa venta
        cursor.execute("""
            SELECT m.titulo, d.cantidad, d.precio_unitario, (d.cantidad * d.precio_unitario) as subtotal
            FROM detalle_ventas d
            JOIN mangas m ON d.manga_id = m.id
            WHERE d.venta_id = ?
            """, (id,))
        productos = cursor.fetchall()

        #acomodamos los datos obtenido de la consulta en un diccionario
        respuesta = {
            "id_venta": venta[0],
            "fecha": venta[1],
            "total_pagado": venta[2],
            "metodo_pago": venta[3],
            "articulos": [] #lista donde se guardan todos los articulos de la venta y sus detalles
        }

        #agregamos todos los articulos de la venta en diccionarios en la lista "articulos"
        for p in productos:
            respuesta["articulos"].append({
                "manga": p[0],
                "cantidad": p[1],
                "precio_unitario": p[2],
                "subtotal": p[3]
            })

        conn.close()
        return jsonify(respuesta), 200 #regresamos el diccionario con los valores de la venta

    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Este GET funciona similar al GET normal anteriormente mencionado, lo que lo diferencia es que primero aquí debemos verificar que la venta con el id que inserto el usuario si exista, si paso ese filtro entonces ya usamos una consulta que nos traiga todos los datos de esas ventas, simplemente queda acomodar esos datos en un diccionario y mostrárselo al usuario en formato json.

Metodo POST en ventas

```
577 #POST ventas
578 @app.route("/ventas", methods=["POST"])
579 def registrarVenta():
580     conn = conexionDB()
581     try:
582         data = request.json #guardamos los datos que ingreso el usuario en una variable
583
584         #si el usuario no puso ningun producto regresa un mensaje de error
585         if "productos" not in data or not data["productos"]:
586             return jsonify({"error": "debe incluir al menos un producto"}), 400
587
588         conn = conexionDB()
589         cursor = conn.cursor()
590
591         total_venta = 0 #variable con la que calcularemos el total de la venta con los articulos que inserto el usuario
592         productos = []
593
594         #ciclo for con el que leeremos cada producto que haya insertado el usuario
595         for item in data["productos"]:
596             m_id = item["manga_id"]
597             cant = item["cantidad"]
598
599             #si la cantidad de mangas es menor a cero se regresa un mensaje de error
600             if cant <= 0:
601                 return jsonify({"error": f"La cantidad para el manga ID {m_id} debe ser mayor a 0"}), 400
602             metodo_pago = data.get("metodo_pago", "Efectivo")
603             if metodo_pago not in ["Efectivo", "Tarjeta"]:
604                 return jsonify({"error": "Método de pago no válido. Debe ser 'Efectivo' o 'Tarjeta'."}), 400
605
606
607
608
609             #con una consulta nos traemos el precio, stock y el titulo del manga que coincida con el id insertado por el usuario
610             cursor.execute("SELECT precio, stock, titulo FROM mangas WHERE id = ?", (m_id,))
611             manga = cursor.fetchone() #guardamos los resultados de la consulta en una variable
612
613             #si la consulta no nos devolvio algo sale un mensaje de error que ese manga no existe
614             if not manga:
615                 return jsonify({"error": f"Manga con ID {m_id} no existe"}), 404
616
617             #acomodamos los resultados de la consulta en variables
618             precio, stock, titulo = manga
619             if stock < cant: #si el stock del manga es menor a la que solicita el usuario entonces mandamos un mensade de error
620                 return jsonify({"error": f"Stock insuficiente para {titulo}"}), 400
621
622             total_venta += (precio * cant) #calculamos y acumulamos el precio de cada manga respecto a la cantidad ingresada
623
```

```

623         #guardamos los datos de los mangas en la lista productos
624         productos.append({
625             "id": m_id,
626             "cantidad": cant,
627             "precio": precio
628         })
629         fecha_sistema = datetime.now().strftime('%Y-%m-%d %H:%M:%S') #obtenemos la fecha y hora del sistema para guardarla en la base de datos
630
631         #creamos una venta y le asignamos al campo total el valor de la variable "total_venta"
632         cursor.execute("""
633             INSERT INTO ventas (fecha, total, metodo_pago)
634             VALUES (?, ?, ?)
635             """, (fecha_sistema, total_venta, metodo_pago))
636
637         nueva_venta_id = cursor.lastrowid #obtenemos el id que le asigno a nuestra venta la base de datos
638
639         #insertamos los detalles de la venta en la tabla "detalle_ventas"
640         for prod in productos:
641             cursor.execute("""
642                 INSERT INTO detalle_ventas (venta_id, manga_id, cantidad, precio_unitario)
643                 VALUES (?, ?, ?, ?)
644                 """, (nueva_venta_id, prod["id"], prod["cantidad"], prod["precio"]))
645
646         #actualizamos el stock en la tabla "mangas"
647         cursor.execute("UPDATE mangas SET stock = stock - ? WHERE id = ?", (prod["cantidad"], prod["id"]))
648
649         conn.commit()
650         conn.close()
651
652         #mostramos un mensaje de exito de que se proceso exitosamente la venta junto con el id de la venta y el total pagado
653         return jsonify({
654             "mensaje": "venta procesada con exito",
655             "venta_id": nueva_venta_id,
656             "metodo_pago": metodo_pago,
657             "total_pagado": total_venta}), 201
658
659     except Exception as e:
660         conn.rollback() # Si algo falló en el proceso, deshacemos todo
661         conn.close()
662         return jsonify({"error": str(e)}), 500
663

```

Primero abrimos una conexión antes del bloque try, con el fin de que si algo llega a salir mal se pueda hacer un rollback en la parte de excepción y asegurarnos que la variable conn esté disponible en todo momento, ya adentro del bloque try obtenemos los datos insertados por el usuario, una vez obtenidos verificaremos que se encuentren los campos necesarios para hacer el POST, una vez ya verificado se abre la conexión a la base de datos y crearemos una variable para calcular el total de los productos que decidió insertar el usuario, después se vienen una validaciones que se deben cumplir para insertar una venta, como que la cantidad de producto debe ser mayor a cero, que el manga que queremos registrar en la venta si exista y que el stock sea mayor a la cantidad que solicita el cliente, una vez ya verificados todos esos datos empezamos a calcular el precio total de la venta, una vez que se realiza todo esto se guardan los datos del manga en la lista "productos" creada anteriormente, insertamos la nueva venta en la tabla "ventas", al igual que insertamos los detalles de venta en la tabla "detalle_ventas", y también actualizamos el stock del manga debido a que ahora hay menor cantidad gracias a la venta, si no hubo ningún error en todo el proceso se mostrara un mensaje de que se insertó la venta correctamente y si no lanzara la excepción de que hubo algún problema en algo

Método DELETE en ventas

```
556 #DELETE ventas
557 @app.route("/ventas/<int:id>", methods=["DELETE"])
558 def eliminarVenta(id):
559     try:
560         conn = conexionDB()
561         cursor = conn.cursor()
562
563         #seleccionamos el id del manga y la cantidad que coincidan con el id de la venta
564         cursor.execute("SELECT manga_id, cantidad FROM detalle_ventas WHERE venta_id = ?", (id,))
565         detalles = cursor.fetchall() #guardamos los resultados de la consulta en una variable
566
567         #si la consulta no nos dio un resultado entonces mandamos un mensaje de error
568         if not detalles:
569             conn.close()
570             return jsonify({"error": "Venta no encontrada"}), 404
571
572         # actualizamos el stock sumando la cantidad que obtuvimos de la consulta anterior
573         for d in detalles:
574             m_id, cant = d
575             cursor.execute("UPDATE mangas SET stock = stock + ? WHERE id = ?", (cant, m_id))
576
577         #eliminamos todos los registros de la tabla "detalle_ventas" donde este el id de la tabla "ventas"
578         cursor.execute("DELETE FROM detalle_ventas WHERE venta_id = ?", (id,))
579         #eliminamos el registro de la tabla "ventas" que tenga el id ingresado por el usuario
580         cursor.execute("DELETE FROM ventas WHERE id = ?", (id,))
581
582         conn.commit()
583         conn.close()
584         return jsonify({"mensaje": "Venta eliminada o cancelada y stock restaurado"}), 200 #mensaje de la eliminacion se llevo a cabo exitosamente
585     except Exception as e:
586         return jsonify({"error": str(e)}), 500
587
```

Este método funciona básicamente igual que la mayoría de los métodos DELETE ya comentados anteriormente, lo primero es verificar que exista la venta con el id que ingreso el usuario, esto mediante una consulta que nos traiga la venta con dicho id en la base de datos, en caso de que no se haya obtenido un resultado lanza un mensaje de error de que la venta con ese id no existe, en caso de que si exista entonces antes de borrar la venta se debe actualizar el stock sumando la cantidad de productos que tenía la venta, eliminamos todos los registros de la tabla “detalle_ventas” que coincida con el id de la venta que se desea eliminar y ahora si ya eliminamos la venta de la tabla “ventas”.

Método PUT en ventas

```
698 def actualizarVenta(id):
700     data = request.json #obtenemos los datos ingresados por el usuario
701
702     if "productos" not in data or len(data["productos"]) == 0: #si no se encuentra nada en la consulta del usuario lanza un mensaje de error
703         return jsonify({
704             "error": "La venta debe tener al menos un producto."
705         }), 400
706     nuevo_metodo_pago = data.get("metodo_pago")
707     conn = conexionDB()
708     cursor = conn.cursor()
709
710     # Verificar que la venta existe y obtener método actual
711     cursor.execute("SELECT metodo_pago FROM ventas WHERE id = ?", (id,))
712     venta_actual = cursor.fetchone()
713     if not venta_actual:
714         conn.close()
715         return jsonify({"error": "Venta no encontrada"}), 404
716     if nuevo_metodo_pago not in ["Efectivo", "Tarjeta"]:
717         return jsonify({"error": "Método de pago no válido. Debe ser 'Efectivo' o 'Tarjeta'."}), 400
718
719     # Si no enviaron un nuevo método, nos quedamos con el que ya tenía la base de datos
720     if not nuevo_metodo_pago:
721         nuevo_metodo_pago = venta_actual[0]
722
723     #seleccionamos el id de la venta que coincida con el id que ingresó el usuario
724     cursor.execute("SELECT id FROM ventas WHERE id = ?", (id,))
725     if not cursor.fetchone(): #si no hubo resultado lanza un mensaje de error de que no se encontró la venta
726         conn.close()
727         return jsonify({"error": "Venta no encontrada"}), 404
728
729     #seleccionamos el id del manga junto con la cantidad para restablecer el stock antes de que se realizara la venta
730     cursor.execute("SELECT manga_id, cantidad FROM detalle_ventas WHERE venta_id = ?", (id,))
731     detalles_anteriores = cursor.fetchall()
732     for detalle in detalles_anteriores:
733         m_id_viejo, cant_vieja = detalle
734         cursor.execute("UPDATE mangas SET stock = stock + ? WHERE id = ?", (cant_vieja, m_id_viejo)) #aquí actualizamos el stock a como estaba antes de la venta
735
736     #aquí empezaremos a actualizar la venta con los nuevos valores
737     total_nuevo = 0
738     productos_para_insertar = []
739
740     #ciclo para validar los nuevos cambios
741     for item in data["productos"]:
742         m_id = item.get("manga_id")
743         cant = item.get("cantidad")
744
745         #si la nueva cantidad es menor a cero o nula lanza un mensaje de error
746         if cant is None or cant <= 0:
```

```
743         #si la nueva cantidad es menor a cero o nula lanza un mensaje de error
744         if cant is None or cant <= 0:
745             conn.rollback() #este rollback es para que regrese la tabla a su estado original antes de actualizar en caso de que la cantidad haya sido menor a cero o nula
746             conn.close()
747             return jsonify({"error": f"La cantidad para el manga ID {m_id} debe ser mayor a 0"}), 400
748
749         #seleccionamos precio, stock y títulos del manga del id ingresado
750         cursor.execute("SELECT precio, stock, titulo FROM mangas WHERE id = ?", (m_id,))
751         manga = cursor.fetchone()
752
753         #si no hubo coincidencia en la búsqueda lanza mensaje de error y rollback
754         if not manga:
755             conn.rollback()
756             conn.close()
757             return jsonify({"error": f"El manga con ID {m_id} no existe"}), 404
758
759         precio, stock_actual, titulo = manga
760
761         #si el stock es menor a la cantidad solicitada lanza mensaje de error y rollback
762         if stock_actual < cant:
763             conn.rollback()
764             conn.close()
765             return jsonify({"error": f"Stock insuficiente para '{titulo}'. Disponible: {stock_actual}"}), 400
766
767         total_nuevo += (precio * cant) #calculamos el nuevo total respecto al precio y la nueva cantidad
768         productos_para_insertar.append({
769             "id": m_id,
770             "cantidad": cant,
771             "precio": precio
772         })
773
774     cursor.execute("UPDATE ventas SET total = ?, metodo_pago = ? WHERE id = ?", (total_nuevo, nuevo_metodo_pago, id)) #actualizamos la tabla "ventas" con el nuevo total y método
775
776     cursor.execute("DELETE FROM detalle_ventas WHERE venta_id = ?", (id,)) #eliminamos los detalles anteriores para reemplazarlos por los nuevos
777
778     for prod in productos_para_insertar:
779         #insertamos los nuevos detalles
780         cursor.execute("""
781             INSERT INTO detalle_ventas (venta_id, manga_id, cantidad, precio_unitario)
782             VALUES (?, ?, ?, ?)
783             """, (id, prod["id"], prod["cantidad"], prod["precio"]))
784
785     cursor.execute("UPDATE mangas SET stock = stock - ? WHERE id = ?", (prod["cantidad"], prod["id"])) #descontamos el nuevo stock
786
787     conn.commit()
```

```

787         conn.commit()
788         conn.close()
789
790         #mensaje de actualizacion realizada con exito
791         return jsonify({
792             "mensaje": "Venta actualizada correctamente",
793             "venta_id": id,
794             "nuevo_total": total_nuevo,
795             "nuevo_metodo_pago": nuevo_metodo_pago
796         }), 200
797
798     except Exception as e:
799         return jsonify({"error": str(e)}), 500
800

```

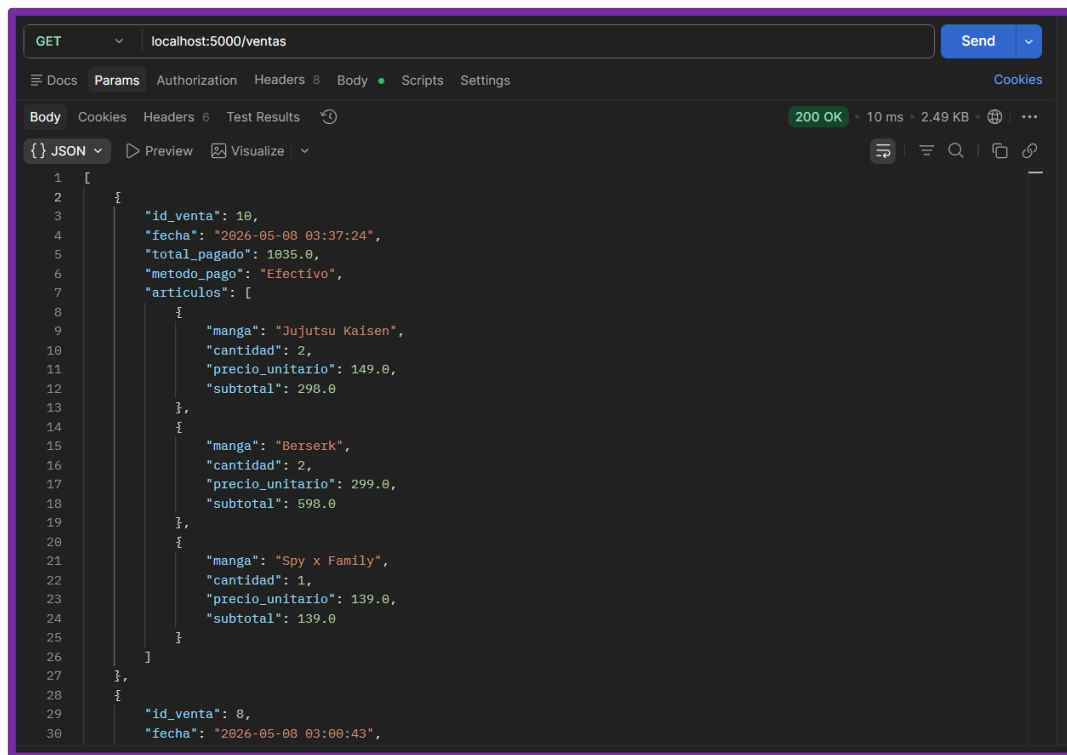
Este método PUT es más complejo, pues requiere de varias validaciones y rollbacks en caso de que algo salga mal, primero empezamos validando que el usuario haya escrito y puesto los campos obligatorios correctamente, luego abriremos una conexión a la base de datos donde confirmaremos que el id de la venta que se quiere actualizar exista, una vez verificado eso vamos a seleccionar el id de los mangas de la venta original junto con sus cantidades para restablecer el stock a como estaba antes de que se realizara la venta, esto con el fin de recalcular el stock con las nuevas cantidades que insertara el usuario, crearemos una nueva variable para calcular el nuevo total y una nueva lista donde insertaremos los nuevos detalles de los productos, ahora toca validar los nuevos datos insertados por el usuario, si la nueva cantidad que ingreso el usuario es menor a cero o nula entonces lanzara un error y se hará un rollback para restablecer la base de datos a como estaba antes de actualizarla, después seleccionaremos los detalles del id del manga ingresado y verificaremos que exista, si no existe se hará un rollback para restablecer la base de datos, si si existe tocara verificar el stock, donde si la nueva cantidad de la venta supera la cantidad que hay en stock se lanza u mensaje de error y un rollback.

Una vez que hayamos pasado todas las verificaciones entonces simplemente calculamos el total de la nueva venta y los detalles a la nueva lista, actualizaremos la tabla “ventas” ventas con el nuevo total, eliminamos los detalles anteriores de la tabla “detalle_ventas” e insertamos los nuevos, actualizamos el stock de los mangas y si todo salió bien

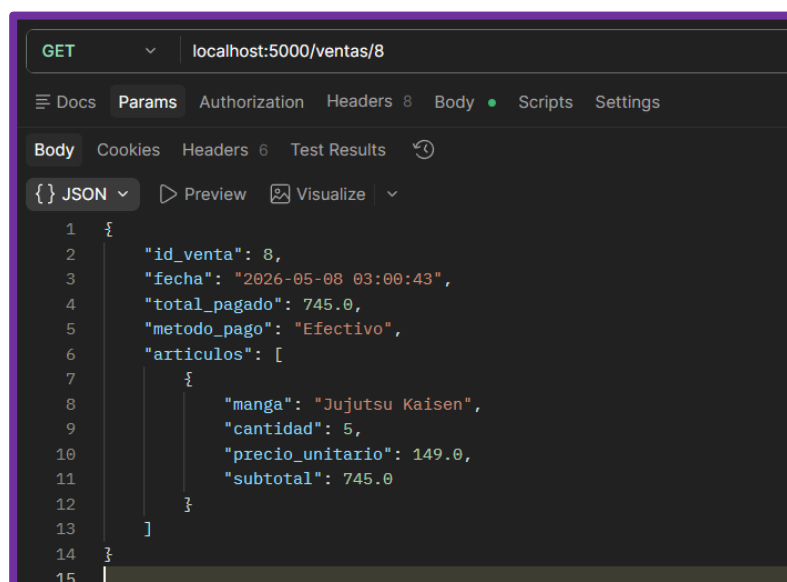
mostramos un mensaje de éxito, y si algo fallo se lanza la excepción con mensaje de error.

Capturas de postman para ventas

GET

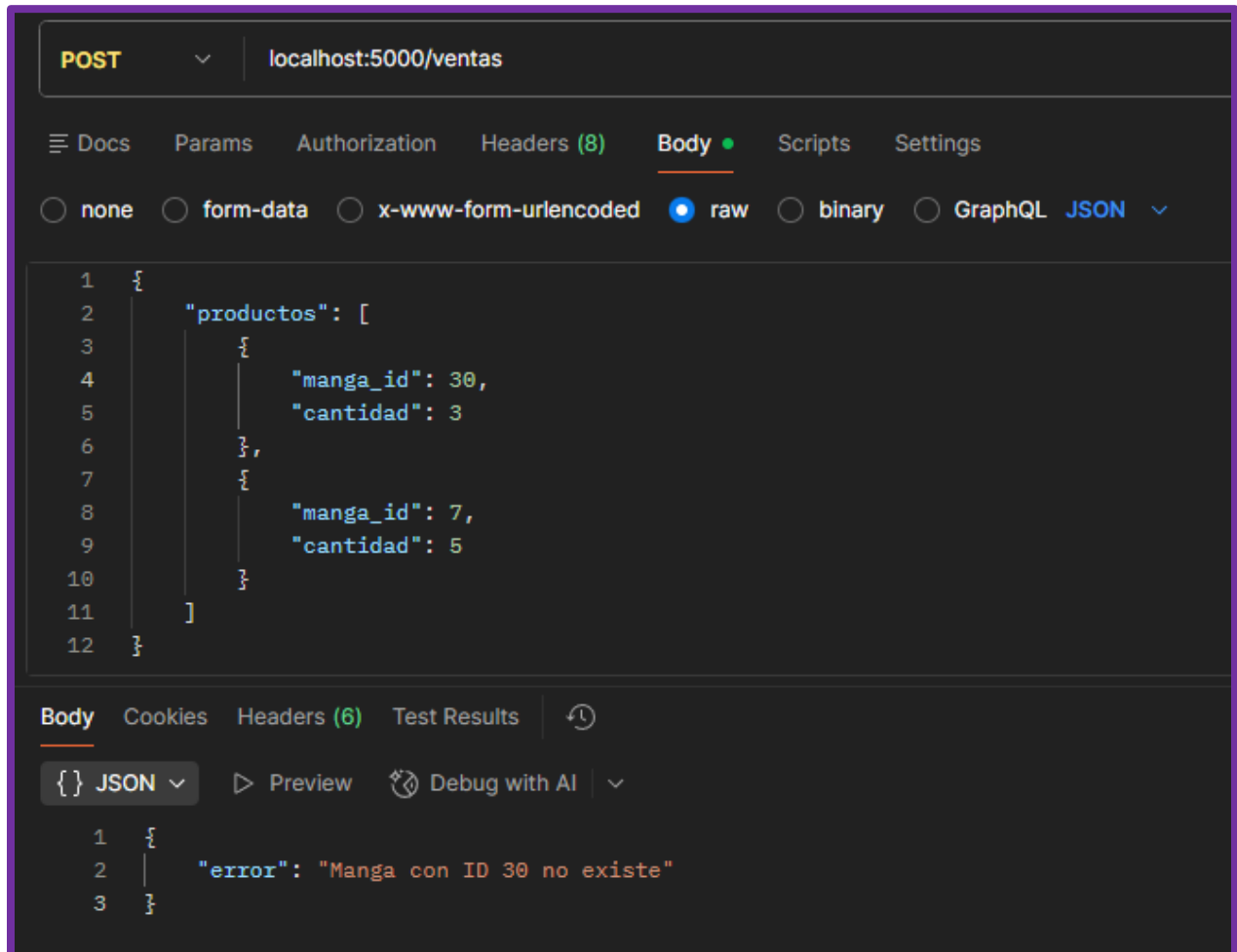


GET por id

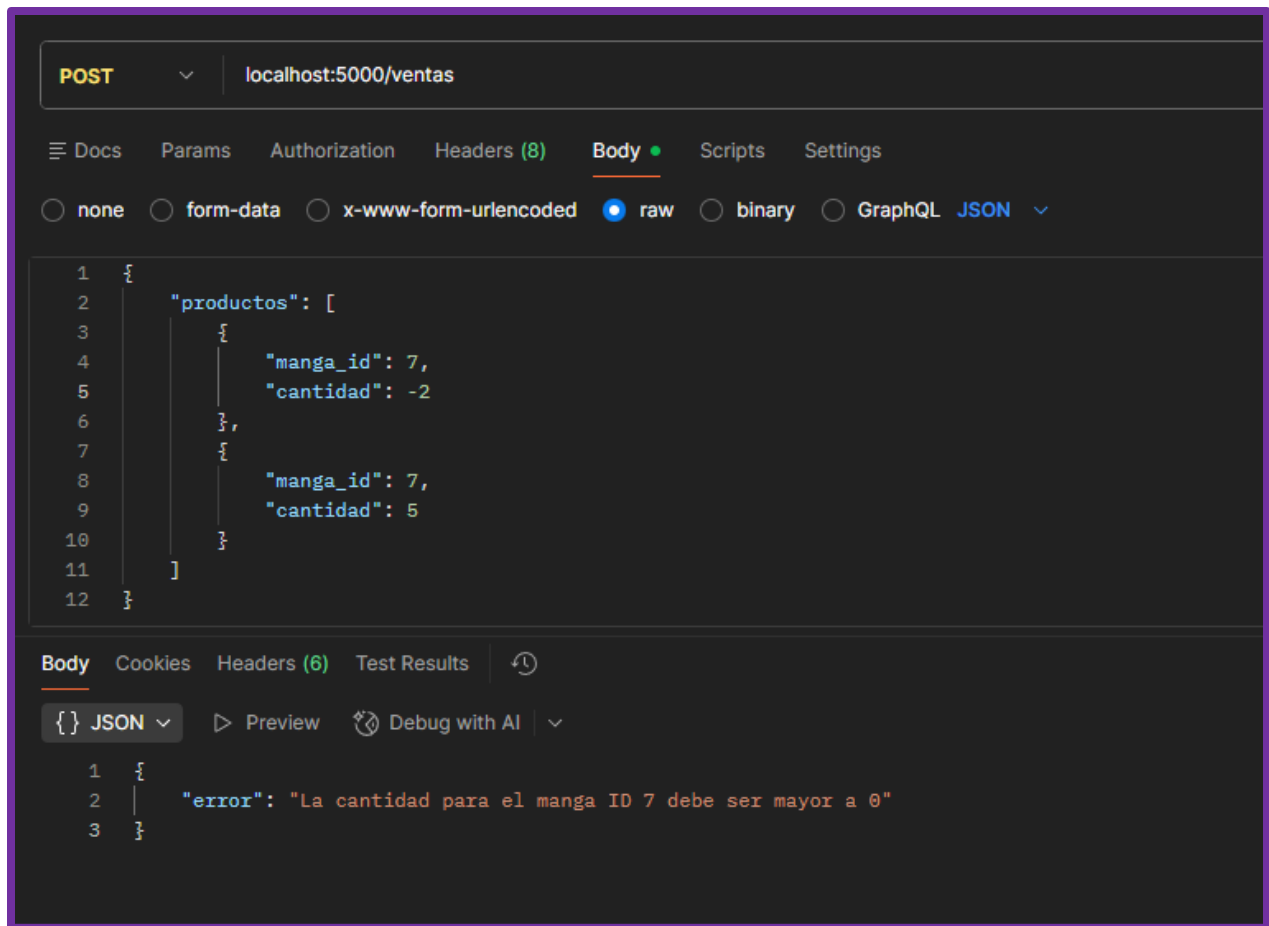


POST

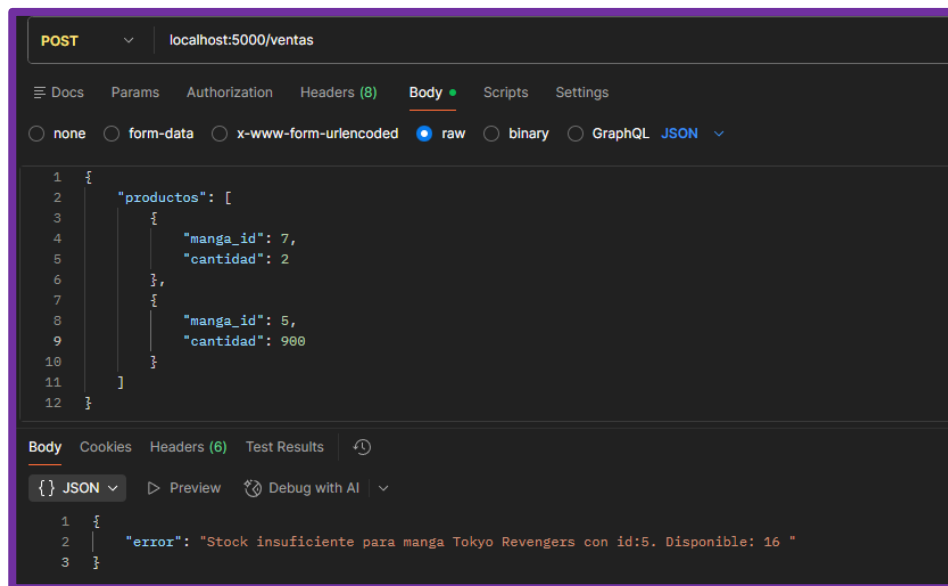
Validar que el id si exista



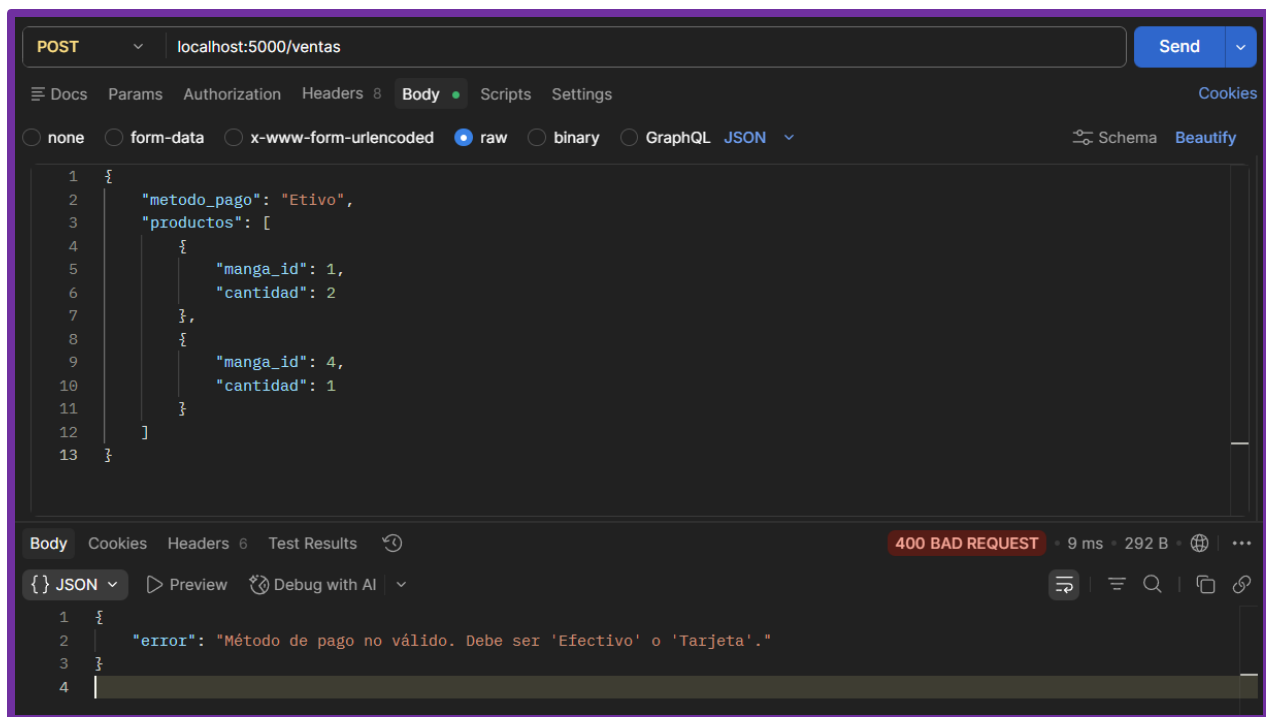
Validar que la cantidad sea mayor a 0



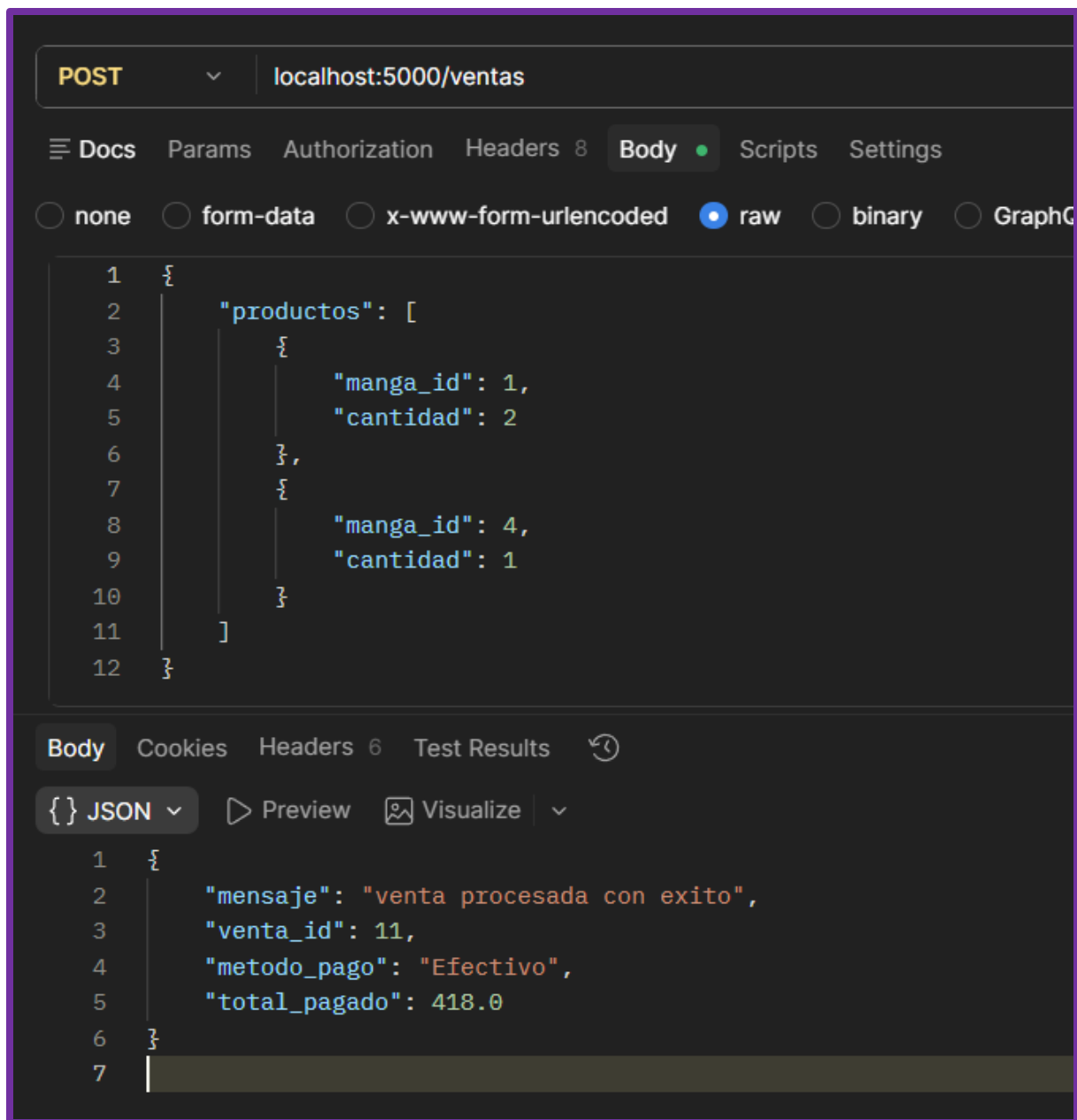
Validar que exista stock suficiente



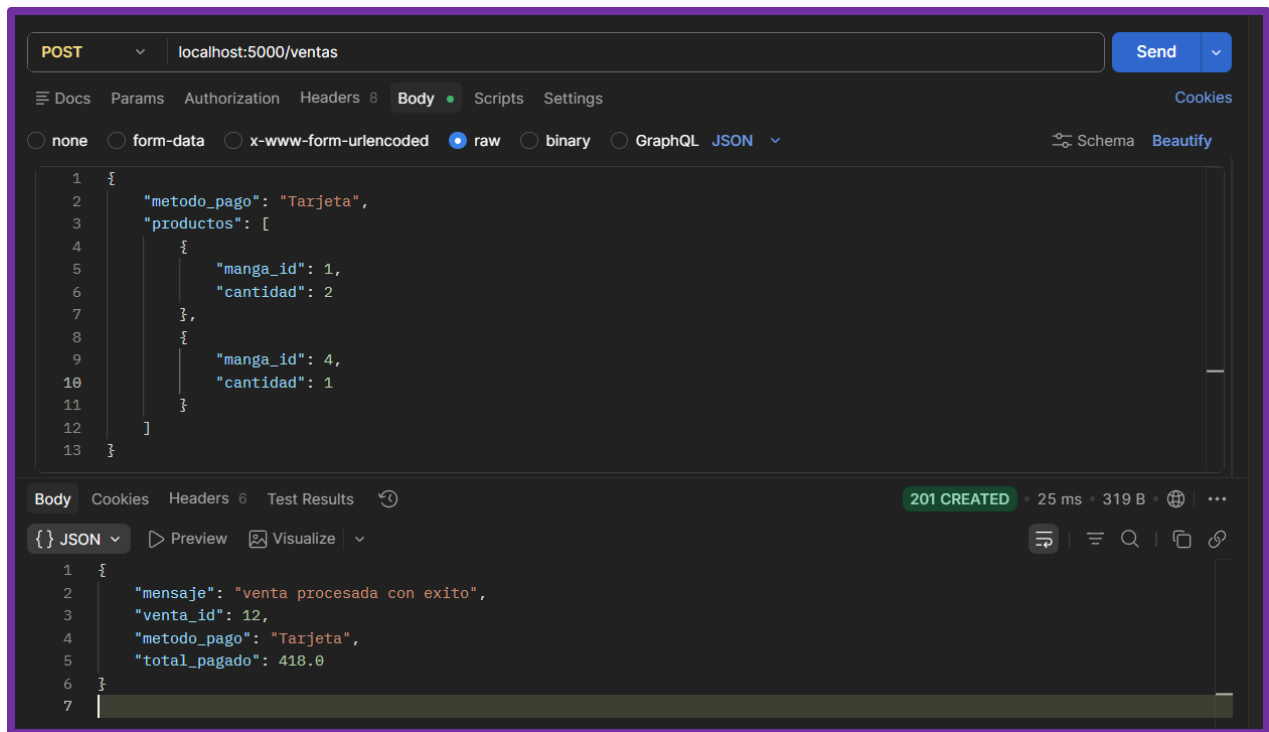
Validar que ingresen el método de pago correctamente



En caso de que no ingrese un método tomara uno por defecto

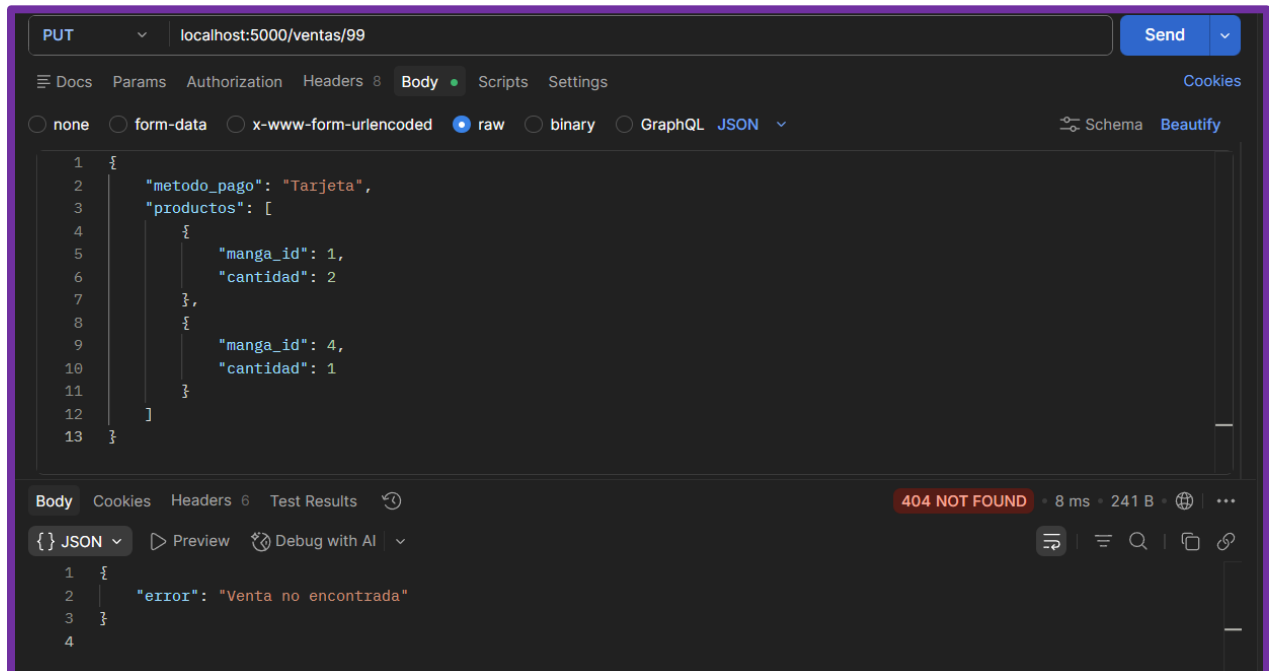


Post exitoso

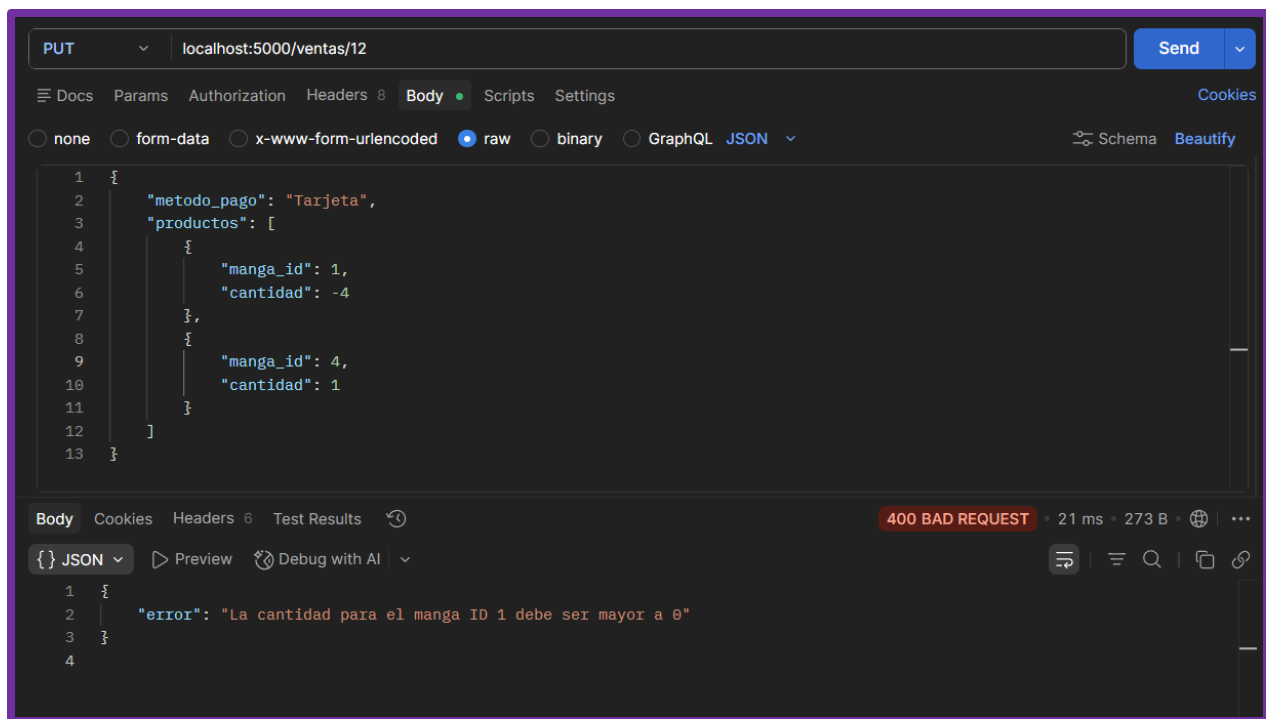


PUT

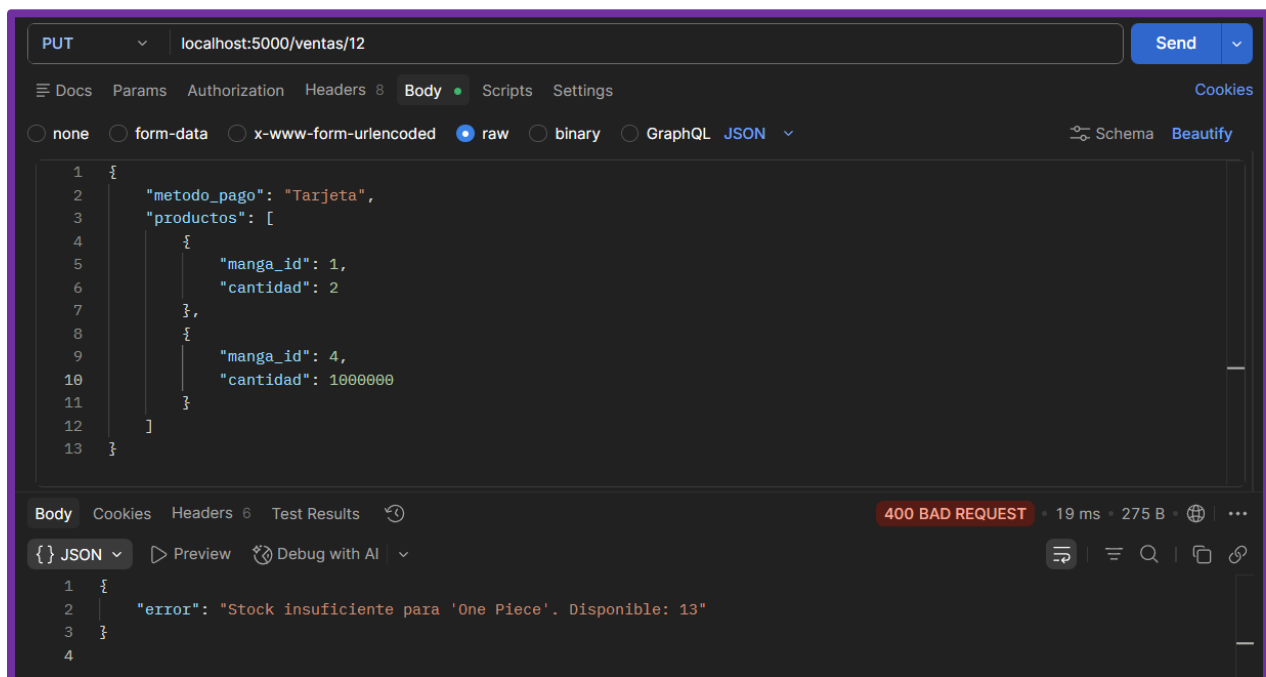
Validar que el id exista



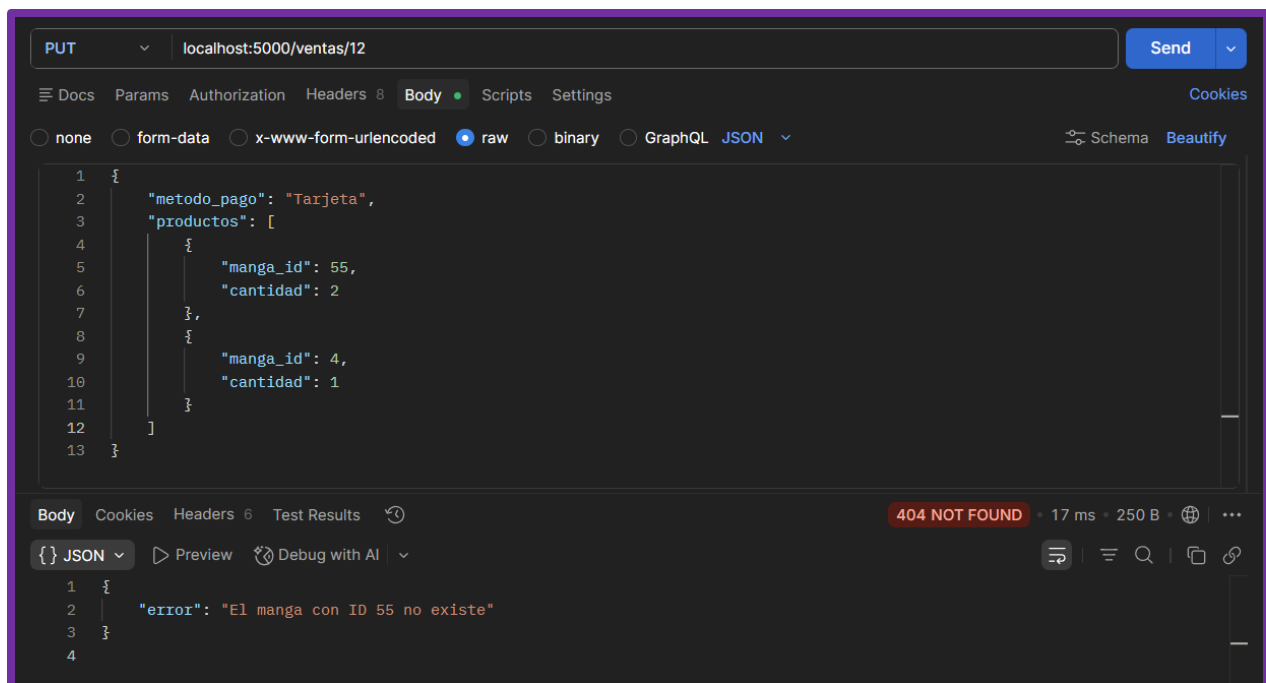
Validar que la cantidad sea mayor a o



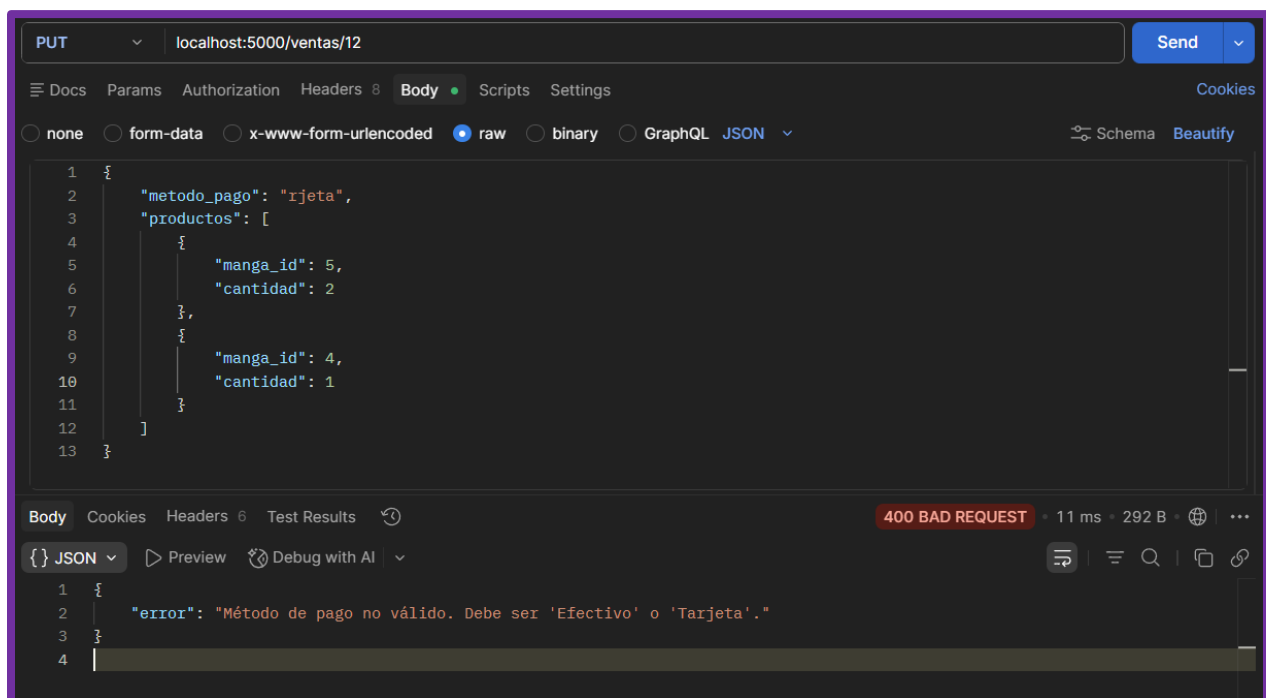
Validar que haya stock suficiente



Validar el ID del manga



Validar el método de pago



PUT exitoso

PUT localhost:5000/ventas/12 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "metodo_pago": "Efectivo",
3   "productos": [
4     {
5       "manga_id": 5,
6       "cantidad": 2
7     },
8     {
9       "manga_id": 4,
10      "cantidad": 1
11    }
12  ]
13 }
```

Body Cookies Headers 6 Test Results 200 OK • 20 ms • 326 B

{ JSON Preview Visualize

```
1 {
2   "mensaje": "Venta actualizada correctamente",
3   "venta_id": 12,
4   "nuevo_total": 440.0,
5   "nuevo_metodo_pago": "Efectivo"
6 }
7
```

DELETE

DELETE localhost:5000/ventas/12 Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "metodo_pago": "Efectivo",
3   "productos": [
4     {
5       "manga_id": 5,
6       "cantidad": 2
7     },
8     {
9       "manga_id": 4,
10      "cantidad": 1
11    }
12  ]
13 }
```

Body Cookies Headers 6 Test Results 200 OK • 22 ms • 262 B

{ JSON Preview Visualize

```
1 {
2   "mensaje": "Venta eliminada o cancelada y stock restaurado"
3 }
4
```

Método GET para las estadísticas del panel

```
#ENDPOINTS CREADOS PARA FILTRAR INFORMACION PARA LO VISUAL
# Nuevo endpoint para estadísticas del panel

@app.route("/stats/diarias", methods=["GET"])
def obtenerStatsDiarias():
    try:
        # Obtenemos el día actual del sistema local
        hoy = datetime.now().strftime('%Y-%m-%d')

        conn = conexionDB()
        cursor = conn.cursor()

        # Cantidad total de ventas del día
        cursor.execute("SELECT COUNT(*) FROM ventas WHERE fecha LIKE ?", (f"{hoy}%",))
        total_ventas = cursor.fetchone()[0] or 0

        # Suma de ventas en EFECTIVO hoy
        cursor.execute("""
            SELECT SUM(total) FROM ventas
            WHERE fecha LIKE ? AND metodo_pago = 'Efectivo'
            """, (f"{hoy}%",))
        efectivo = cursor.fetchone()[0] or 0

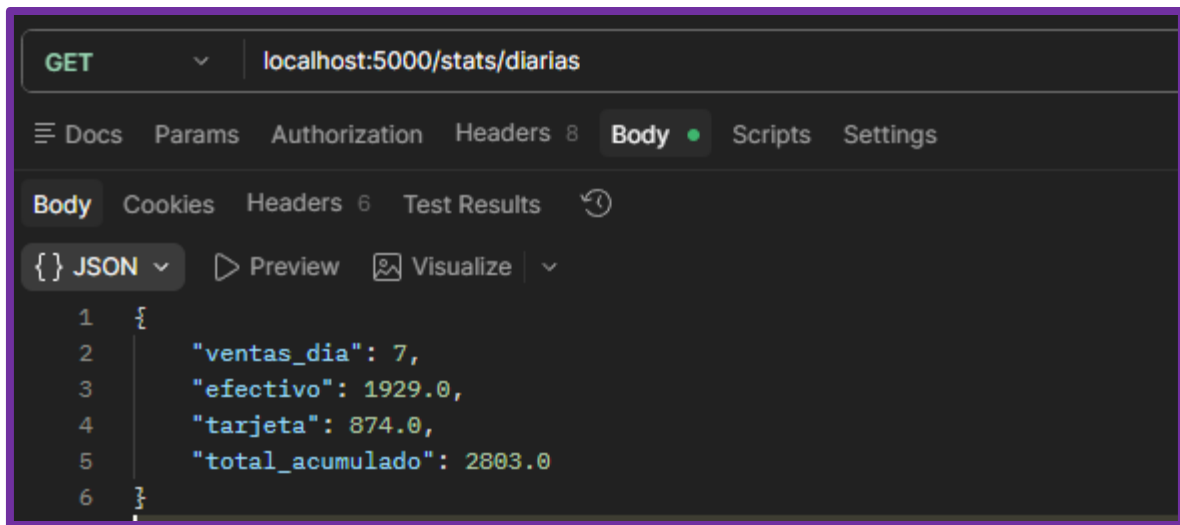
        # Suma de ventas con TARJETA hoy
        cursor.execute("""
            SELECT SUM(total) FROM ventas
            WHERE fecha LIKE ? AND metodo_pago = 'Tarjeta'
            """, (f"{hoy}%",))
        tarjeta = cursor.fetchone()[0] or 0

        conn.close()

        # Devolvemos todo desglosado y el total acumulado (efectivo + tarjeta)
        return jsonify({
            "ventas_dia": total_ventas,
            "efectivo": efectivo,
            "tarjeta": tarjeta,
            "total_acumulado": efectivo + tarjeta
        }), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Este método nos permite filtrar la información necesaria para mostrarla en el panel de control de nuestra página de mangas. Primero almacenamos la fecha actual en el sistema donde se esté ejecutando el programa, luego filtramos la información necesaria mediante las consultas, siendo estas las ventas totales del momento actual, monto vendido en efectivo, monto vendido con tarjetas. Posteriormente mandamos en un **jsonify** la información recolectada añadiendo el campo de **total_acumulado** que se calcula mediante la suma del monto de efectivo y tarjeta obtenidos anteriormente.

Prueba en postman de stats



Método Get para la tabla manga_imagenes

```
# GET: Obtener la imagen de todos los mangas
@app.route("/mangas/imagenes/todas", methods=["GET"])
def obtenerImagenMangatodas():
    try:
        conn = conexionDB()
        cursor = conn.cursor()
        cursor.execute("SELECT url_imagen FROM manga_imagenes")
        filas = cursor.fetchall()
        conn.close()

        if filas:
            return jsonify({"imagenes": [fila[0] for fila in filas]}), 200
        return jsonify({"mensaje": "No se encontraron imágenes para los mangas"}), 404
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Ejecuta la consulta para visualizar todas las imágenes de mangas actualmente registradas.

Método POST para manga_imagenes

```
@app.route("/mangas/imagenes", methods=["POST"])
def asociarImagen():
    try:
        datos = request.get_json()
        conn = conexionDB()
        cursor = conn.cursor()
        #si manda una lista de objetos JSON para asociar varias imagenes a varios mangas
        if isinstance(datos, list):
            for item in datos:
                cursor.execute("""
                    INSERT INTO manga_imagenes (manga_id, url_imagen)
                    VALUES (?, ?)
                    """, (item['manga_id'], item['url_imagen']))
        else:
            # Si manda solo un objeto JSON
            cursor.execute("""
                INSERT INTO manga_imagenes (manga_id, url_imagen)
                VALUES (?, ?)
                """, (datos['manga_id'], datos['url_imagen']))

        conn.commit()
        conn.close()
        return jsonify({"mensaje": "Imagen(es) asociada(s) con éxito"}), 201
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

Tiene como objetivo actualizar la relación entre un producto y su imagen. Recibe el **manga_id** y la **url_imagen** para vincular el archivo de imagen con el registro correspondiente en la base de datos. Está diseñado para procesar tanto una asociación individual (un solo objeto **json**) como múltiples asociaciones simultáneas (una lista de objetos). Asegura que la ruta física del archivo quede registrada y disponible.

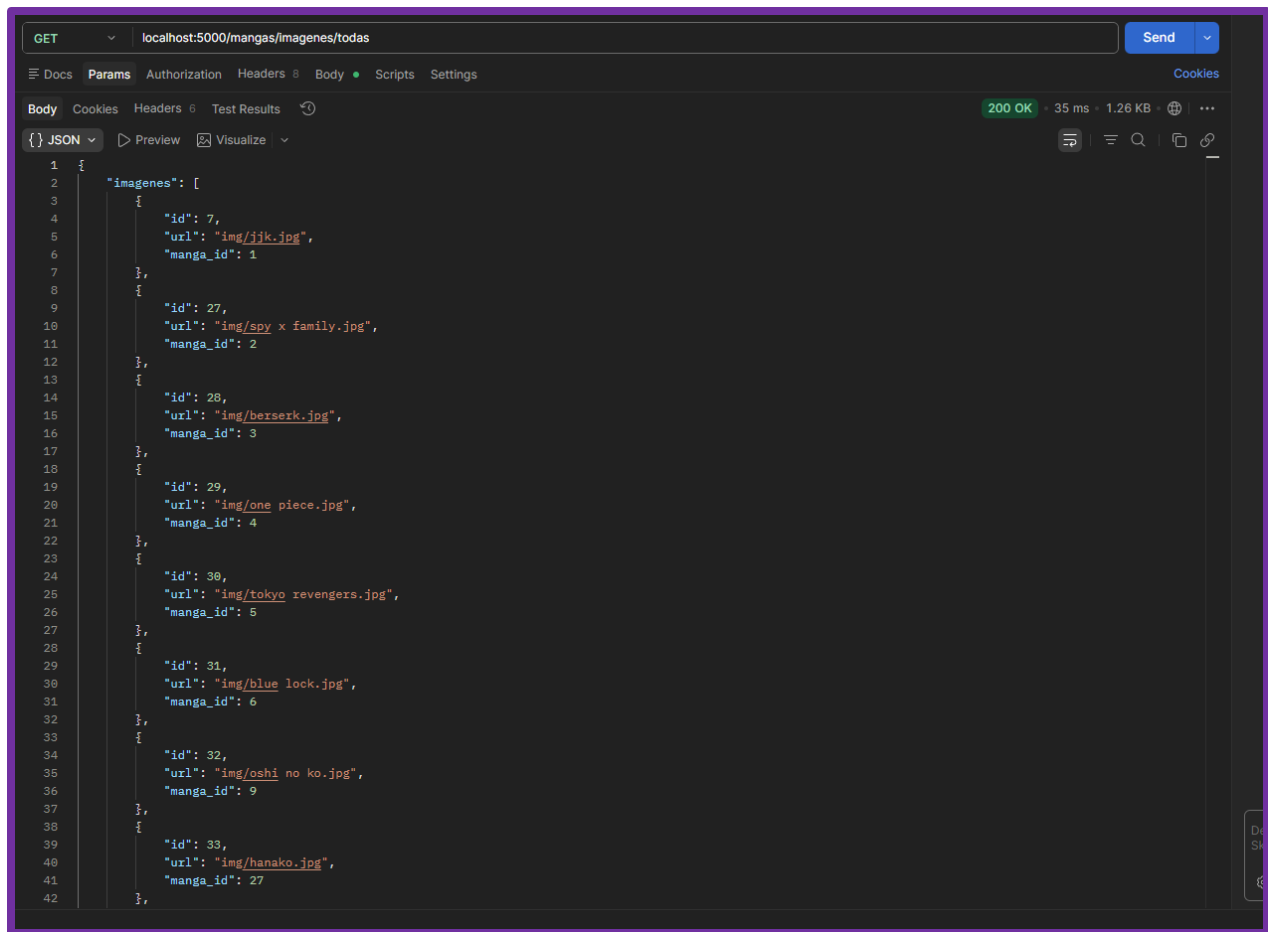
Método DELETE para manga_imagenes

```
# Eliminar la relación de imagen
@app.route("/mangas/imagenes/<int:manga_id>", methods=["DELETE"])
def eliminarImagenManga(manga_id):
    try:
        conn = conexionDB()
        cursor = conn.cursor()
        cursor.execute("DELETE FROM manga_imagenes WHERE manga_id = ?", (manga_id,))
        conn.commit()
        conn.close()
        return jsonify({"mensaje": "Relación de imagen eliminada"}), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
```

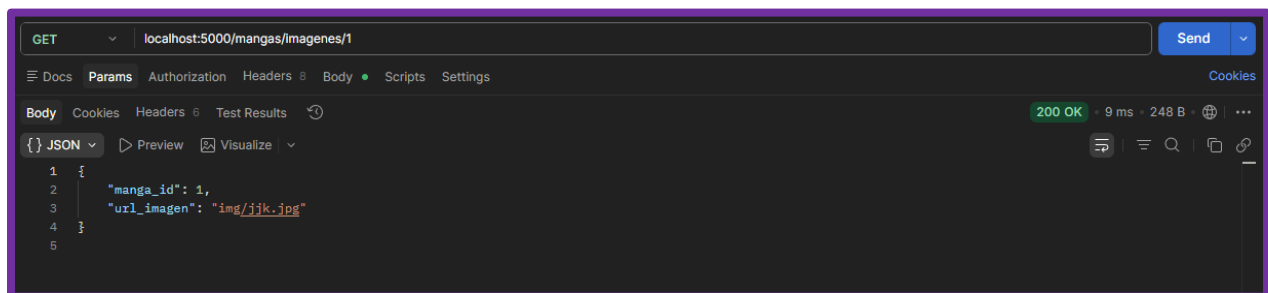
Funciona igual que el resto de delete, obtiene el id seleccionado, lo inserta en la query y si lo encuentra, elimina el registro y regresa un mensaje de éxito, en caso de no encontrar el registro le notifica al cliente que ese registro no existe.

Prueba en Postman para manga_Imagenes

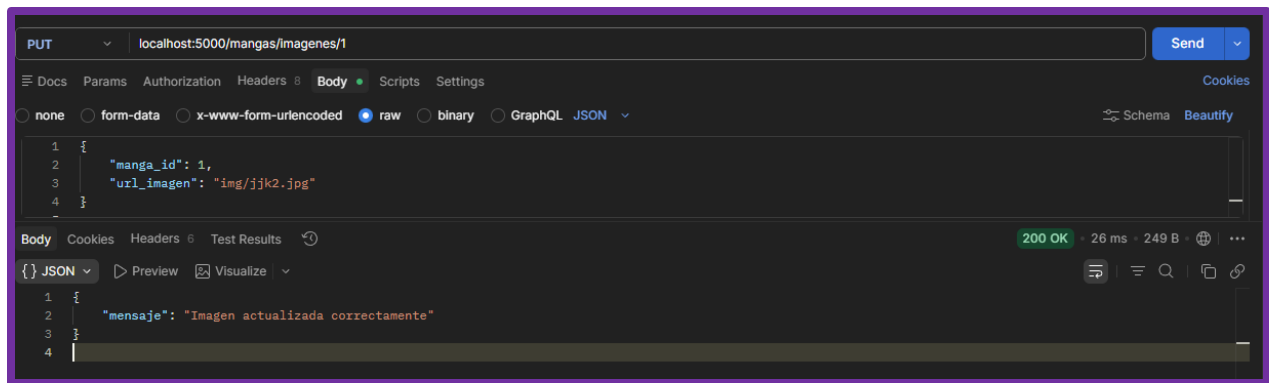
GET



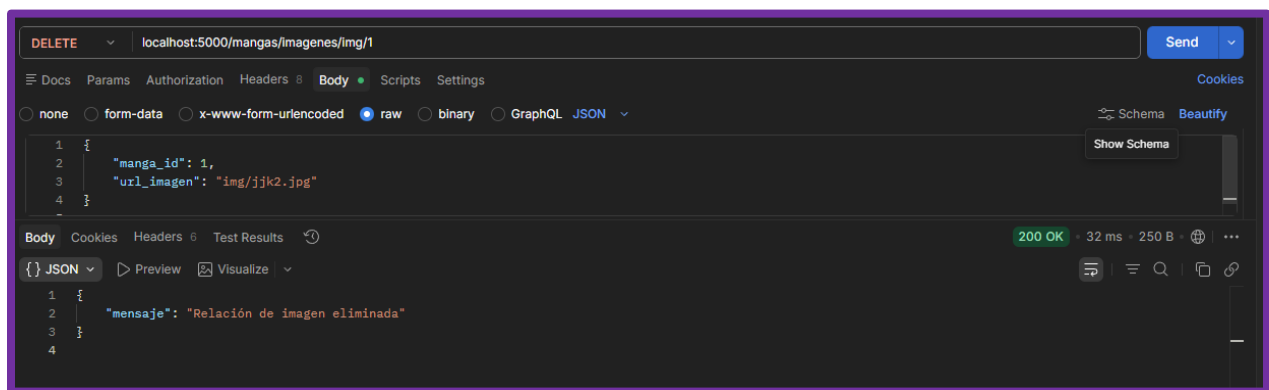
Get por id de un manga



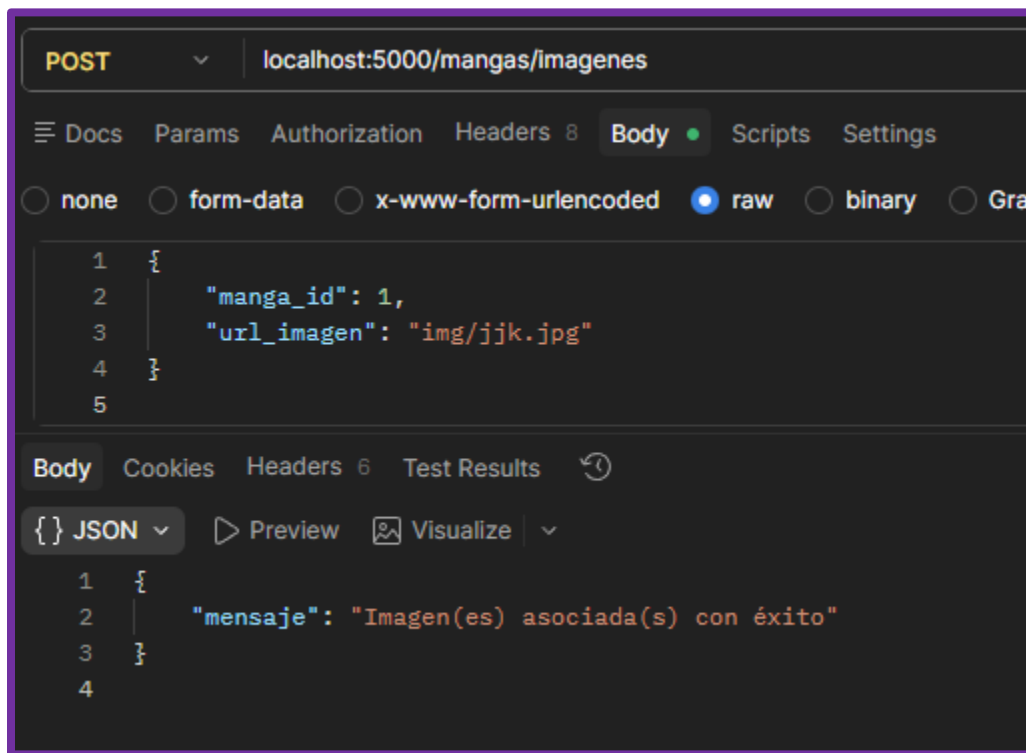
PUT



DELETE



POST



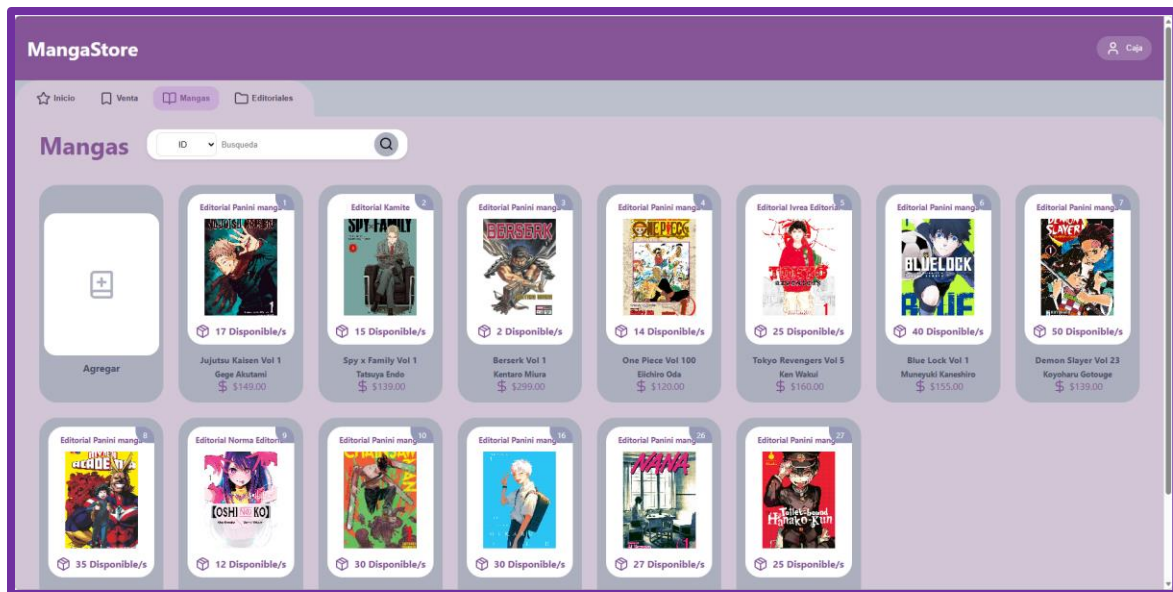
Carpeta Catalogo Mangas

Manga.html:

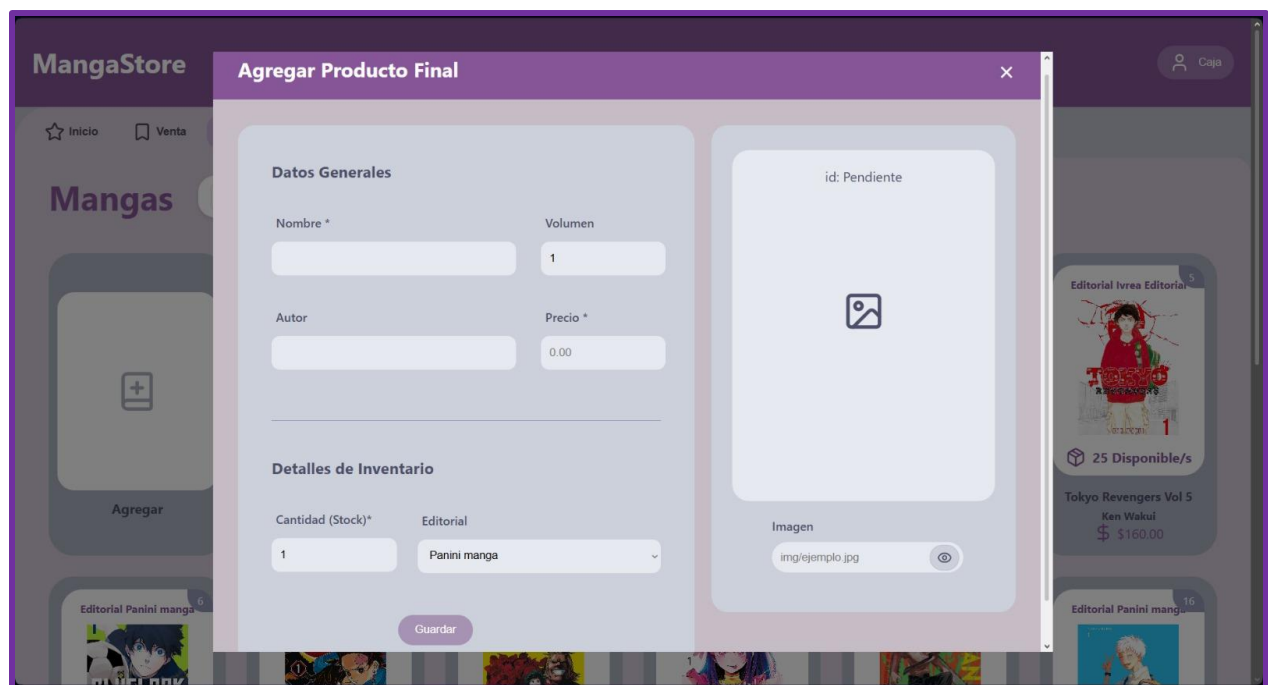
Estructura de la página donde se visualiza el inventario de mangas.

Manga.css:

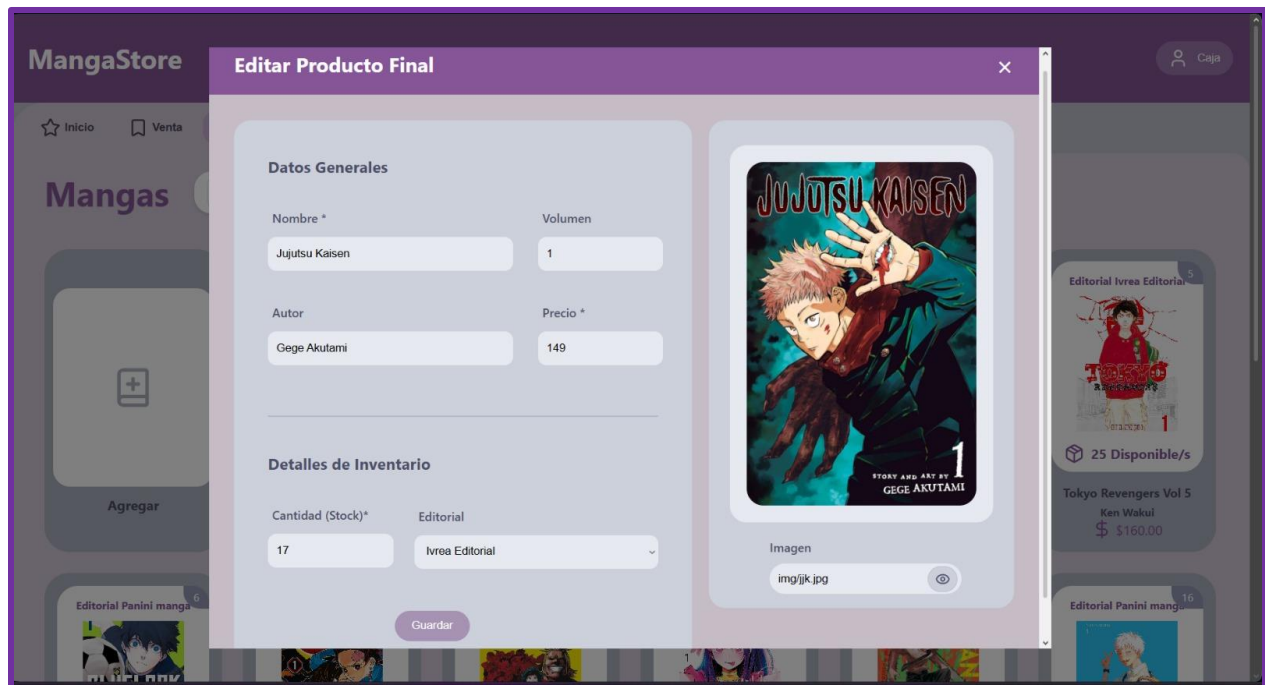
Se encarga de la presentación visual y la ergonomía del sitio.



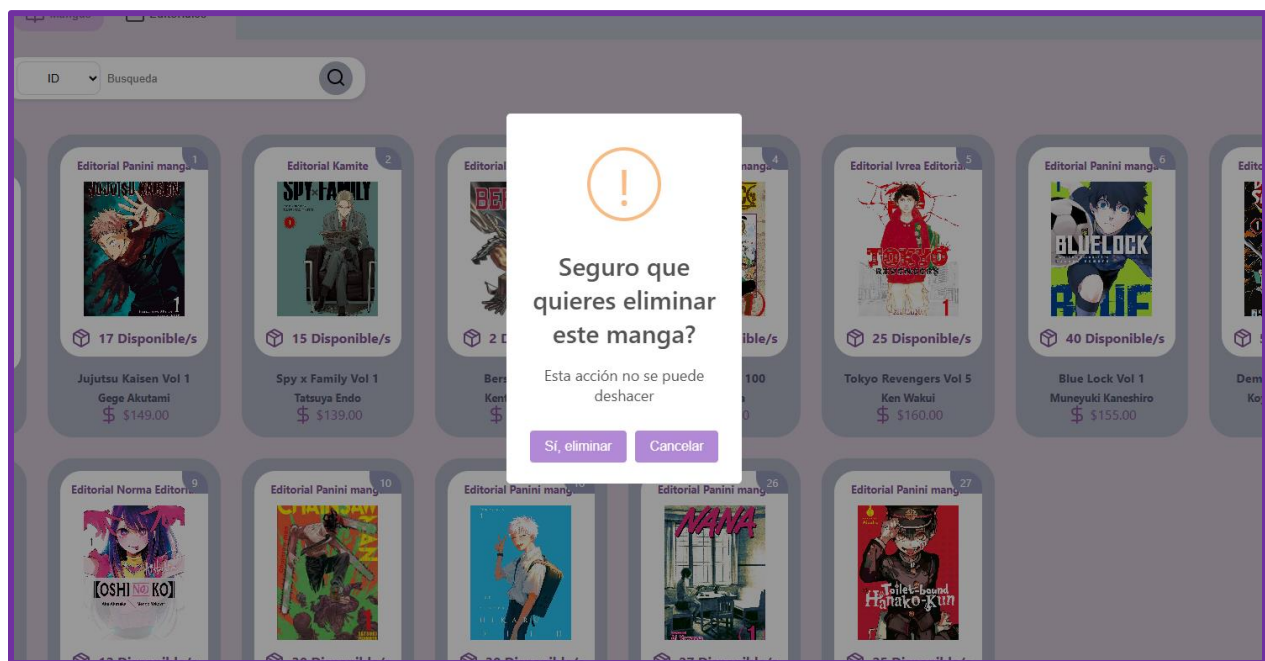
Modal para Agregar Mangas



Modal para editar Mangas



Eliminar un Manga Alert



Manga.js :

El archivo `mangas.js` actúa como el controlador de la interfaz de usuario para el catálogo de productos. Su función principal es gestionar el ciclo de vida de los datos de los mangas en el frontend, actuando como puente entre la API (Backend) y el DOM (interfaz visual).

Se encarga de la persistencia de datos mediante peticiones asíncronas y de mantener una experiencia de usuario dinámica sin necesidad de recargar la página para cada consulta.

Gestión Dinámica del Inventario (Operaciones CRUD):

Lectura (GET): Recupera la lista completa de mangas desde el servidor y los transforma en tarjetas visuales interactivas.

```
28 async function cargarMangas() {
29   try {
30     // Llamada al endpoint que ya incluye el LEFT JOIN con manga_imagenes
31     const response = await fetch('http://127.0.0.1:5000/mangas');
32     const mangas = await response.json();
33
34     const container = document.getElementById('mangas-container');
35
36     // Mantenemos solo el botón de "Agregar" y limpiamos el resto
37     const btnAddHtml = `
38       <div class="card-add" id="btn-abrir-modal-add">
39         <div class="inner-add" style="width: 190px; height: 235px; display: flex; align-items: center; justify-content: center; margin-top: 1px;">
40           <i data-lucide="book-plus" style="width: 50px; height: 50px; color: #ac0000;"></i>
41         </div>
42         <span style="margin-top: 10px; font-weight: bold;">Agregar</span>
43       </div>
44     `;
45     container.innerHTML = btnAddHtml;
46
47     // Generar las tarjetas dinámicamente
48     mangas.forEach(m => {
49       const card = document.createElement('div');
50       card.className = 'manga-card';
51
52       //si la BD no tiene url_imagen, usa una por defecto
53       const rutaPortada = m.url_imagen ? m.url_imagen : 'img/default.png';
54
55       // Construir el HTML de la tarjeta con los datos del manga
56       card.innerHTML = `
57         <div class="card-top">
58           <span class="id-badge">${m.id}</span>
59           <span class="editorial-label">Editorial ${m.id_editorial}</span>
60           
65           <span class="stock-label"><i data-lucide="package"></i> ${m.stock} Disponible/s </span>
66
67           <button class="btn-edit-card" onclick="montareditarManga(${m.id},event)">
68             <i data-lucide="pencil"></i>
69           </button>
70           <button class="btn-delete-card" onclick="eliminarManga(${m.id})">
71             <i data-lucide="trash-2"></i>
72           </button>
73         </div>
74         <div class="manga-info">
75           <h3>${m.titulo} Vol. ${m.volumen}</h3>
```

```

73     </div>
74     <div class="manga-info">
75         <h3>${m.titulo} Vol ${m.volumen}</h3>
76         <p>${m.autor}</p>
77         <div class="price-tag">
78             <i data-lucide="dollar-sign"></i>
79             <span>${m.precio.toFixed(2)}</span>
80         </div>
81     </div>
82     `;
83     container.appendChild(card);
84 });
85
86     // Refrescar iconos para las nuevas tarjetas inyectadas
87     lucide.createIcons();
88
89 } catch (error) {
90     console.error("Error al conectar con la API de MangaStore:", error);
91 }
92 }
93

```

Escritura (POST): Captura los datos de nuevos ejemplares desde formularios modales y los envía a la base de datos, manejando de forma separada la información general y la asociación de imágenes.

```

268 // Enviar a la base de datos al dar click en "Guardar" la informacion que hay en los inputs del modal
269 //METODO DE CREAR MANGA (POST)
270 document.getElementById('btn-confirmar-guardar').addEventListener('click', async () => {
271     const mangaData = {
272         titulo: document.getElementById('nombreCrear').value,
273         autor: document.getElementById('nuevo-autor').value,
274         volumen: parseFloat(document.getElementById('nuevo-volumen').value),
275         precio: parseFloat(document.getElementById('nuevo-precio').value),
276         stock: parseInt(document.getElementById('nuevo-stock').value),
277         id_editorial: parseInt(document.getElementById('select-editorial').value)
278     };
279
280     try {
281         const response = await fetch('http://127.0.0.1:5000/mangas', {
282             method: 'POST',
283             headers: { 'Content-Type': 'application/json' },
284             body: JSON.stringify(mangaData)
285         });
286
287         const result = await response.json();
288
289         if (response.ok) {
290             // Mostrar el ID que devolvió la BD
291             console.log("ID del manga creado:", result.id);
292             const urlImg = document.getElementById('nuevo-url-img').value;
293             if (urlImg) {
294                 await fetch('http://127.0.0.1:5000/mangas/imagenes', {
295                     method: 'POST',
296                     headers: { 'Content-Type': 'application/json' },
297                     body: JSON.stringify({ manga_id: result.id, url_imagen: urlImg })
298                 });
299             }
300
301             await Swal.fire({
302                 title: '¡Manga Agregado!',
303                 text: 'El registro se guardó correctamente',
304                 icon: 'success',
305                 width: '280px',
306                 confirmButtonText: 'ok',
307                 confirmButtonColor: '#b189d7'
308             });
309             // Reemplaza 'modalCrear' por el ID real del modal
310             const modalElement = document.getElementById('modalCrear');
311             if (modalElement) {
312                 const modalBootstrap = bootstrap.Modal.getInstance(modalElement);
313                 if (modalBootstrap) {
314                     modalBootstrap.hide();
315                 }
316             }
317         }
318     } catch (error) {
319         console.error('Error al crear manga:', error);
320     }
321 }

```

Actualización (PUT): Implementa una lógica de "Upsert" para las imágenes; verifica si un manga ya cuenta con una portada para decidir si debe actualizar el registro existente o crear uno nuevo.

```

452 // Función para confirmar edición (PUT a la API)
453 document.getElementById('btn-confirmar-editar').addEventListener('click', async () => {
454     const mangaId = document.getElementById('edit-manga-id').value;
455     const urlImg = document.getElementById('editar-url-img').value; // Definimos la variable urlImg
456
457     if (!mangaId) {
458         Swal.fire('Error', 'No se encontró el ID', 'error');
459         return;
460     }
461
462     const mangaData = {
463         titulo: document.getElementById('nombreEditar').value,
464         autor: document.getElementById('editar-autor').value,
465         volumen: parseFloat(document.getElementById('editar-volumen').value),
466         precio: parseFloat(document.getElementById('editar-precio').value),
467         stock: parseInt(document.getElementById('editar-stock').value),
468         id_editorial: parseInt(document.getElementById('select-editorial-editar').value)
469     };
470
471     try {
472         //Actualizar datos generales del Manga
473         let resManga = await fetch(`http://127.0.0.1:5000/mangas/${mangaId}`, {
474             method: 'PUT',
475             headers: { 'Content-Type': 'application/json' },
476             body: JSON.stringify(mangaData)
477         });
478
479         if (resManga.ok) {
480             // Verificar si la imagen ya existe en la DB
481             // Hacemos un GET simple.
482             let resVerificar = await fetch(`http://127.0.0.1:5000/mangas/imagenes/${mangaId}`);
483
484             let resImg;
485
486             if (resVerificar.ok) {
487                 // Si el GET fue exitoso (200), la imagen EXISTE -> Usamos PUT
488                 console.log("La imagen existe, actualizando...");
489                 resImg = await fetch(`http://127.0.0.1:5000/mangas/imagenes/${mangaId}`, {
490                     method: 'PUT',
491                     headers: { 'Content-Type': 'application/json' },
492                     body: JSON.stringify({ url_imagen: urlImg })
493                 });
494             } else if (resVerificar.status === 404) {
495                 // Si el GET devolvió 404, la imagen NO EXISTE -> Usamos POST
496                 console.log("La imagen no existe, creando nuevo registro...");
497                 resImg = await fetch(`http://127.0.0.1:5000/mangas/imagenes`, {
498                     method: 'POST',

```

Eliminación (DELETE): Gestiona la baja de registros del sistema mediante confirmaciones de seguridad para evitar pérdida de datos accidental.

```

390 async function eliminarManga(id) {
391     const result = await Swal.fire({
392         title: 'Seguro que quieres eliminar este manga?',
393         text: 'Esta acción no se puede deshacer',
394         icon: 'warning',
395         width: '280px',
396         showCancelButton: true,
397         cancelButtonText: 'Cancelar',
398         cancelButtonColor: '#b189d7',
399         confirmButtonText: 'Sí, eliminar',
400         confirmButtonColor: '#b189d7'
401     });
402     if (result.isConfirmed) {
403         try {
404             const response = await fetch(`http://127.0.0.1:5000/mangas/${id}`, {
405                 method: 'DELETE'
406             });
407
408             if (response.ok) {
409                 await Swal.fire({
410                     title: '¡Manga Eliminado!',
411                     text: 'El registro se eliminó correctamente',
412                     icon: 'success',
413                     width: '280px',
414                     confirmButtonText: 'ok',
415                     confirmButtonColor: '#b189d7'
416                 });
417                 location.reload();
418             } else {
419                 alert("Error al eliminar el manga");
420             }
421         } catch (error) {
422             console.error("Error al eliminar:", error);
423         }
424     } else {
425         await Swal.fire({
426             title: '¡Cancelado!',
427             text: 'El manga no fue eliminado',
428             icon: 'error',
429             width: '280px',
430             confirmButtonText: 'ok',
431             confirmButtonColor: '#b189d7'
432         });
433     }
434 }
435
436 }

```

Motor de Búsqueda y Filtrado:

Implementa una lógica de consulta parametrizada que permite al usuario filtrar el catálogo por **ID, Nombre o Autor**, optimizando la localización de productos específicos en el inventario.

[illegible]

```

159         class="manga-img"
160         alt="Portada de ${m.titulo}"
161         onerror="this.src='img/default.png'"
162         <span class="stock-label"> <i data-lucide="package"></i> ${m.stock} Disponible/s </span>
163
164         <button class="btn-edit-card" onclick="montareeditarManga(${m.id}, event)">
165             <i data-lucide="pencil"></i>
166         </button>
167         <button class="btn-delete-card" onclick="eliminarManga(${m.id})">
168             <i data-lucide="trash-2"></i>
169         </button>
170     </div>
171     <div class="manga-info">
172         <h3>${m.titulo} Vol ${m.volumen}</h3>
173         <p>${m.autor}</p>
174         <div class="price-tag">
175             <i data-lucide="dollar-sign"></i>
176             <span>${m.precio.toFixed(2)}</span>
177         </div>
178     </div>
179     `;
180     container.appendChild(card);
181 });
182
183 // Renderizar iconos
184 if (typeof lucide !== 'undefined') {
185     lucide.createIcons();
186 }
187 } else {
188     container.innerHTML = `<p class="no-results">No se encontraron resultados para "${valor}"</p>`;
189 }
190 } else {
191     container.innerHTML = `<p class="no-results">No se encontraron resultados para "${valor}"</p>`;
192 }
193 } catch (error) {
194     console.error("Error al buscar:", error);
195     Swal.fire('Error', 'No se pudo realizar la búsqueda', 'error');
196 }
197 }

```

Control de Interfaz y Validación:

Validación de Entradas: Asegura que campos críticos como stock, volumen y precio no contengan valores negativos antes de ser enviados al servidor.

```

250 //no permitir que volumen, precio o stock sean negativos
251 document.getElementById('nuevo-volumen').addEventListener('input', (e) => {
252     if (e.target.value <= 0) e.target.value = 1;
253 });
254 document.getElementById('nuevo-precio').addEventListener('input', (e) => {
255     if (e.target.value < 0) e.target.value = 0;
256 });
257 document.getElementById('nuevo-stock').addEventListener('input', (e) => {
258     if (e.target.value <= 0) e.target.value = 1;
259 });
260

```

Gestión de Modales: Controla la apertura, cierre y limpieza de las ventanas emergentes para creación y edición.

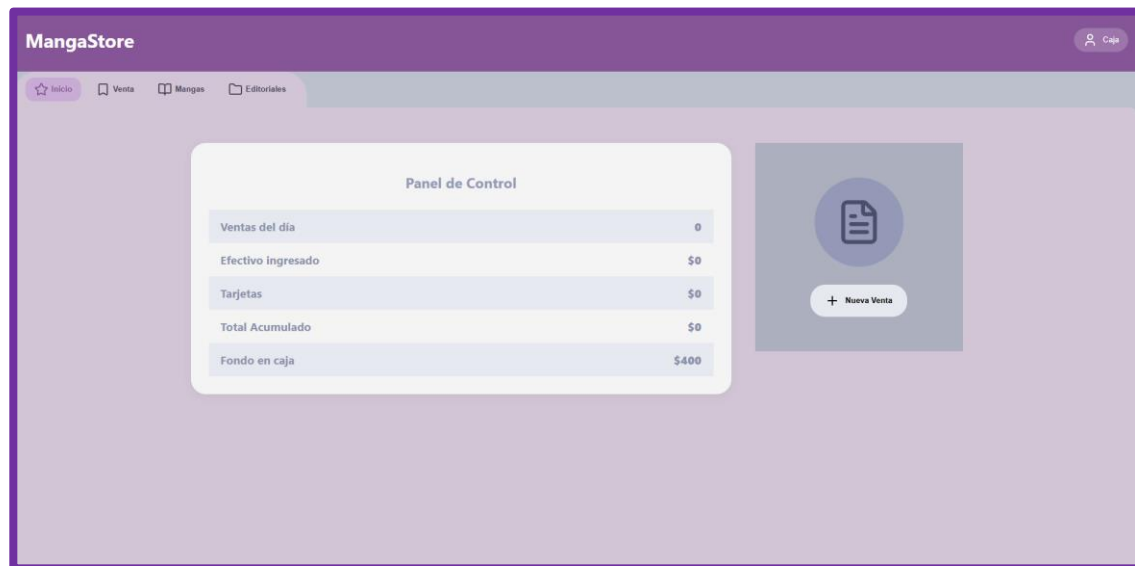
Carpeta PP (Página Principal)

pp.html:

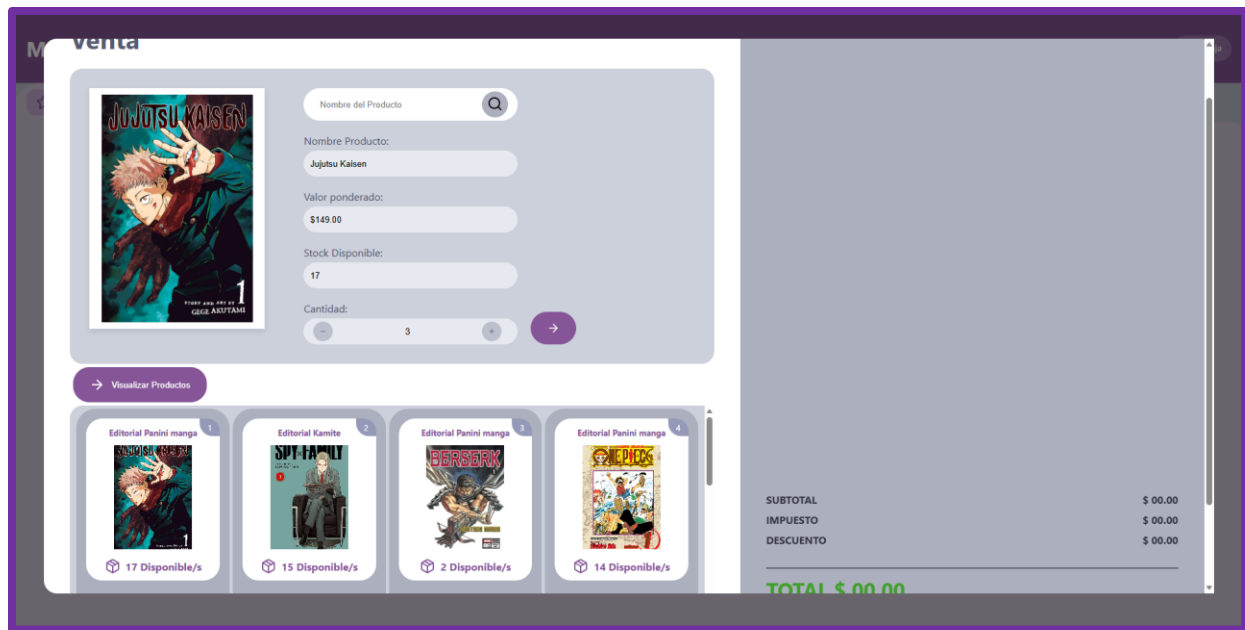
Estructura de la página donde se visualiza las stats diarias de ventas y donde se realiza el proceso principal de las ventas.

pp.css:

Se encarga de la presentación visual y la ergonomía del sitio.



Modal de Venta



Pp.js

Su función es gestionar el flujo desde que llega un cliente hasta que se procesa su pago. Se encarga de la interacción en tiempo real para buscar productos, gestionar un carrito de compras temporal, calcular impuestos y totales, y finalmente comunicar la transacción al servidor para actualizar el inventario y las finanzas.

Al cargar la página (DOMContentLoaded), el script dispara `cargarEstadisticas()`, que consulta al servidor las ventas del día, el total acumulado y el desglose por método de pago (Efectivo/Tarjeta) para mostrarlos en el Dashboard principal.

```
//----- Carga de iconos y estadísticas al iniciar la página-----

// Llamar a la función cuando cargue la página
document.addEventListener('DOMContentLoaded', () => {
  lucide.createIcons();
  cargarEstadisticas();
});
```

```
//----- Función para obtener las estadísticas desde el servidor
async function cargarEstadisticas() {
  try {
    const respuesta = await fetch('http://127.0.0.1:5000/stats/diarias');
    const data = await respuesta.json();

    if (respuesta.ok) {
      // Actualiza los en HTML usando los IDs
      document.getElementById('ventas-hoy-count').textContent = data.ventas_dia;

      const elEfectivo = document.getElementById('efectivo-hoy');
      const elTarjeta = document.getElementById('tarjeta-hoy');
      const elTotal = document.getElementById('total-acumulado');

      if (elTotal) elTotal.textContent = `$$${data.total_acumulado}`;
      if (elEfectivo) elEfectivo.textContent = `$$${data.efectivo}`;
      if (elTarjeta) elTarjeta.textContent = `$$${data.tarjeta}`;
    }
  } catch (error) {
    console.error("Error:", error);
  }
}
```

Realiza peticiones asíncronas para buscar mangas específicos. Si el manga existe, rellena automáticamente los campos de precio, stock y muestra la portada; si no, reinicia los campos y muestra una imagen por defecto.

```

98 async function buscarMangaPorNombre() {
99   console.log("Buscando manga...");
100   const nombreBusqueda = buscarManga.value.trim();
101   console.log('Nombre ingresado:', nombreBusqueda);
102
103   try {
104     const respuesta = await fetch('http://127.0.0.1:5000/mangas/nombre/${nombreBusqueda}');
105     const manga = await respuesta.json();
106     console.log('Respuesta del servidor:', manga);
107     const rutaPortada = manga[0].url_imagen ? manga[0].url_imagen : 'img/default.png';
108
109     if (Array.isArray(manga) && manga.length > 0) {
110       CampoNombre.value = manga[0].titulo;
111       console.log('Titulo del manga:', manga[0].titulo);
112       CampoPrecio.value = `$$${manga[0].precio}`;
113       CampoStock.value = manga[0].stock;
114       imagenManga.innerHTML = `<img src=../catalogomangas/${rutaPortada}" alt="${manga[0].titulo}" style="width: 100%; height: 100%; object-fit: cover; border-radius: 20px;`
115     } else {
116       imagenManga.innerHTML = `<img src=../catalogomangas/img/default.png" alt="Manga no encontrado" style="width: 100%; height: 100%; object-fit: cover; border-radius: 20px;`
117       CampoNombre.value = "No encontrado";
118       CampoPrecio.value = "$0.00";
119       CampoStock.value = "0";
120     }
121   } catch (error) {
122     console.error("Error:", error);
123     return null;
124   }
125 }
126
127 // Evento para presionar Enter
128 buscarManga.addEventListener('keypress', (e) => {
129   if (e.key === 'Enter') {
130     buscarMangaPorNombre();
131   }
132 });
133
134 // Evento para clic en la lupa
135 btnBuscarManga.addEventListener('click', buscarMangaPorNombre);
136
137
138
139
140

```

Permite cargar una lista de "cards" interactivas. Al hacer clic en una tarjeta, los datos del manga se transfieren automáticamente al formulario de venta.

Cargar cards:

```

145 async function cargarMangas() {
146   try {
147     // Llamada al endpoint que ya incluye el LEFT JOIN con manga_imagenes
148     const response = await fetch('http://127.0.0.1:5000/mangas');
149     const mangas = await response.json();
150
151     const container = document.getElementById('mangas-list-container');
152
153     // Generar las tarjetas dinámicamente
154     mangas.forEach(m => {
155       const card = document.createElement('div');
156       card.className = 'manga-card';
157
158       //si la BD no tiene url_imagen, usa una por defecto
159       const rutaPortada = m.url_imagen ? m.url_imagen : 'img/default.png';
160
161
162       // Construir el HTML de la tarjeta con los datos del manga
163       card.innerHTML = `
164         <div class="card-top">
165           <span class="id-badge">${m.id}</span>
166           <span class="editorial-label">Editorial ${m.id_editorial}</span>
167           
171           <span class="stock-label"> <i data-lucide="package"></i> ${m.stock} Disponible/s </span>
172         </div>
173         <div class="manga-info">
174           <h3>${m.titulo}</h3>
175           <p>${m.autor}</p>
176           <div class="price-tag">
177             <i data-lucide="dollar-sign"></i>
178             <span>${m.precio.toFixed(2)}</span>
179           </div>
180         </div>
181       `;
182     });
183
184     // - Evento para llenar los campos al hacer clic ---
185     card.addEventListener('click', () => {
186       // Llenar Nombre del Producto (Título + Volumen)
187       document.getElementById('display-nombre').value = `${m.titulo}`;
188
189       // Llenar Valor Ponderado (Precio)
190       document.getElementById('display-precio').value = `${m.precio.toFixed(2)}`;
191     });

```

```

189 // Llenar Valor Ponderado (Precio)
190 document.getElementById('display-precio').value = `$$${m.precio.toFixed(2)}`;
191
192 // Llenar Stock Disponible
193 document.getElementById('display-stock').value = m.stock;
194
195 // Actualizar la imagen del placeholder
196 const previewImg = document.querySelector('.image-placeholder');
197 previewImg.innerHTML = ``;
200
201 // Reseteo cantidad a 1 al seleccionar nuevo producto
202 document.getElementById('display-cantidad').value = 1;
203 });
204 container.appendChild(card);
205 });
206
207 // Refrescar iconos para las nuevas tarjetas inyectadas
208 lucide.createIcons();
209
210 } catch (error) {
211     console.error("Error al conectar con la API de MangaStore:", error);
212 }
213 }
214 // Evento para cargar mangas al hacer clic en el botón
215 VisualizarMangas.addEventListener('click', cargarMangas);
216

```

Evento de selección de las cards

```

239 //Agregar un manga al wrapper desde una card y llenar los fields con los datos del manga seleccionado
240 //Obtener el id
241 async function obtenerId(nombre) {
242     try {
243         const respuesta = await fetch(`http://127.0.0.1:5000/mangas/${nombre}`);
244         if (!respuesta.ok) {
245             console.log('Manga no encontrado');
246             return null;
247         }
248         const manga = await respuesta.json();
249         if (Array.isArray(manga) && manga.length > 0) {
250             console.log('ID del primer manga:', manga[0].id);
251             return manga[0].id;
252         }
253     } catch (error) {
254         console.error("Error:", error);
255         return null;
256     }
257 }
258

```

Incluye lógica para validar que la cantidad a vender no sea menor a uno.

```
// Evento para aumentar
btnPlus.addEventListener('click', () => {
  let valorActual = parseInt(inputCantidad.value);
  inputCantidad.value = valorActual + 1;
});

// Evento para disminuir
btnMinus.addEventListener('click', () => {
  let valorActual = parseInt(inputCantidad.value);
  // Evitamos que sea menor a 1
  if (valorActual > 1) {
    inputCantidad.value = valorActual - 1;
  }
});
```

Antes de agregar un producto a la tabla de venta, el script verifica si hay existencias suficientes en la base de datos y si la cantidad acumulada en el carrito no supera el límite físico del inventario.

```

    });
    // Verificar si la cantidad total solicitada (en tabla + nueva) supera el stock
    const totalSolicitado = cantidadEnTabla + cantidad;

    if (totalSolicitado > stock) {
        Swal.fire('Error', `Stock insuficiente. Disponible: ${stock - cantidadEnTabla}`, 'error');
        return;
    }
    const bloque = {
        id, nombre, cantidad, precio
    }
    console.log(bloque);
    const fila = document.createElement('tr');

    fila.innerHTML = `
        <td>${bloque.id}</td>
        <td>${bloque.nombre}</td>
        <td>${bloque.cantidad}</td>
        <td>${bloque.precio}</td>
        <td>
            <span class="btn-eliminar-tabla">&times;</span>
        </td>
    `;

    const btnEliminar = fila.querySelector('.btn-eliminar-tabla');
    btnEliminar.addEventListener('click', () => {
        console.log(btnEliminar)
        fila.remove();

        calculoTotales();
    });

    tbody.appendChild(fila);
    calculoTotales();

    watchProductos.addEventListener('click', cargarDatosTabla);

```

Calcula en base a los productos ingresados el monto sin impuestos, los impuestos y el total a pagar.

```

// Función para calcular totales, impuestos y descuentos
function calculoTotales() {
  console.log('Holi');
  const filas = document.querySelectorAll('#cart-items tr');
  let subtotal = 0;

  filas.forEach(fila => {
    // Obtenemos precio y cantidad de las celdas (ajusta el índice si es necesario)
    const cantidad = parseFloat(fila.cells[2].innerText) || 0;
    const precioText = fila.cells[3].innerText;
    const precio = parseFloat(
      precioText.replace('$', '').trim()
    ) || 0;
    subtotal += cantidad * precio;
    console.log(cantidad, precio);
  });

  const impuestoReal = subtotal * 0.16;
  const impuesto = impuestoReal; // Ejemplo 16%
  const subtotalFinal = subtotal - impuestoReal;
  const descuento = subtotal > 1000 ? 50 : 0; // Ejemplo: $50 si es > 1000
  const total = subtotal - descuento;

  // Actualizar el HTML
  document.querySelector('.totals-section p:nth-child(1) span').innerText = `$ ${subtotalFinal.toFixed(2)}`;
  document.querySelector('.totals-section p:nth-child(2) span').innerText = `$ ${impuesto.toFixed(2)}`;
  document.querySelector('.totals-section p:nth-child(3) span').innerText = `$ ${descuento.toFixed(2)}`;
  document.querySelector('.final-total span').innerText = `$ ${total.toFixed(2)}`;

  return { subtotal, impuesto, descuento, total };
}
// Función para agregar producto a la tabla

```

Una vez confirmado el pago, envía un objeto JSON al endpoint /ventas con el detalle de los productos y el método de pago. Tras la respuesta exitosa del servidor, utiliza *SweetAlert2* para confirmar al usuario, limpia el carrito y refresca las estadísticas del panel.

```

async function procesarVenta(metodoPago) {
    const productos = [];
    // Recolectar productos
    filas.forEach(fila => {
        productos.push({
            manga_id: parseInt(fila.cells[0].innerText),
            cantidad: parseInt(fila.cells[2].innerText)
        });
    });

    // Datos de venta
    const datosVenta = {
        productos: productos,
        metodo_pago: metodoPago
    };
    try {
        const response = await fetch('http://127.0.0.1:5000/ventas', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json'
            },
            body: JSON.stringify(datosVenta)
        });
        const resultado = await response.json();
        if (response.ok) {
            Swal.fire({
                title: 'Éxito',
                text: 'Pago procesado correctamente',
                icon: 'success'
            }).then(() => {
                cerrarModalPago();

                // Limpiar tabla
                limpiarTabla();
                // Limpiar tabla
                limpiarForm();
            });
        } else {
            Swal.fire('Error', `Error:${resultado.error}`, 'error');
        }
    } catch (error) {
        console.error("Error en la conexión:", error);
    }
}

```

Carpeta historialVentas

Esta carpeta corresponde a la ventana de ventas donde se visualizarán todas las ventas que se realicen dentro de la página.

Ventas.html

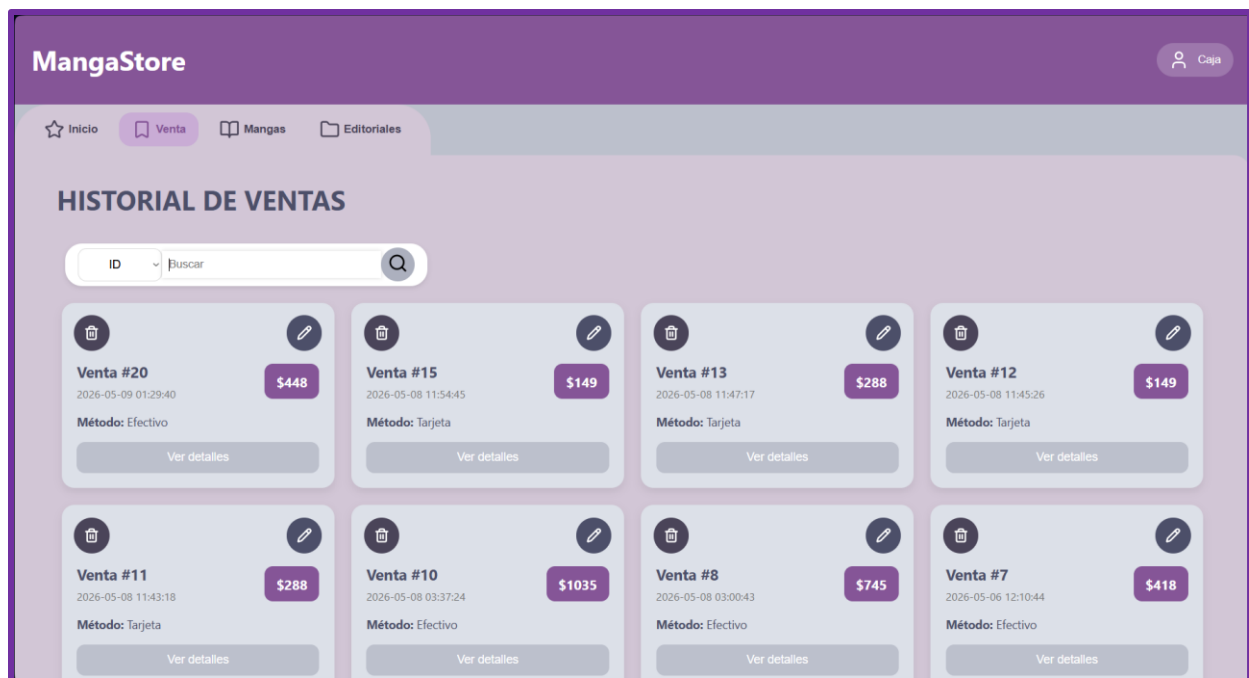
Se trata de la estructura que tendrá la ventana, donde se alojan los componentes que constituyen la pagina correspondiente a ventas señalizados por medio de etiquetas.

En este archivo en la parte superior, se están mandando llamar las librerías para importar el estilo de los iconos, alertas y el estilo general de la página.

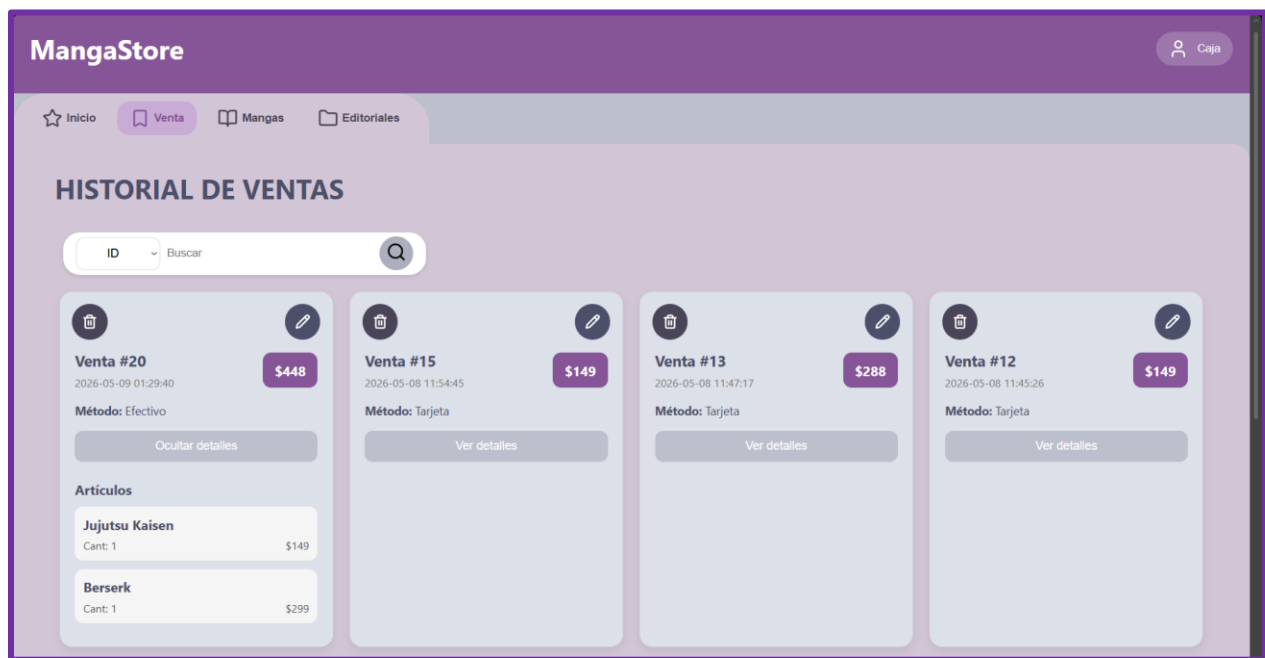
```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ventas</title>
  <link rel="stylesheet" href="ventas.css">
  <script src="https://unpkg.com/lucide@latest"></script>
  <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script> <!--Libreria para las alertas-->
</head>
```

Ventas.css

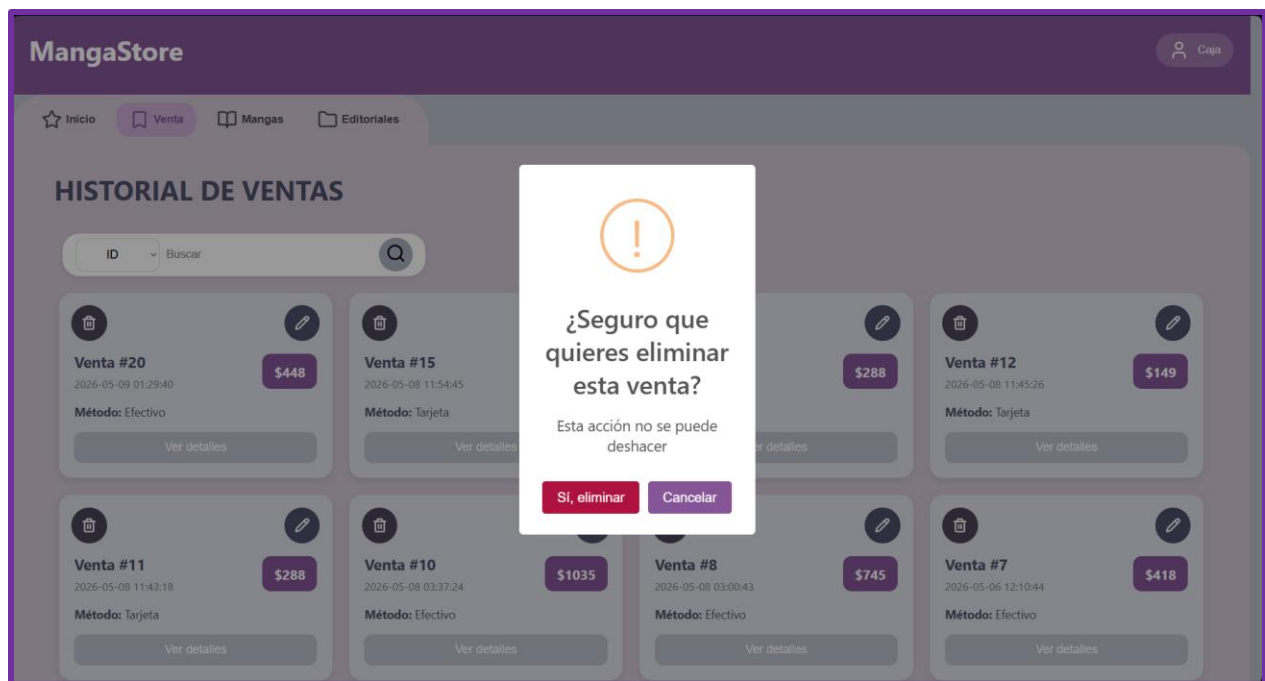
Se encarga del apartado visual y define la interactividad básica, permitiendo que elementos como los botones cambien de color o forma al interactuar con ellos.



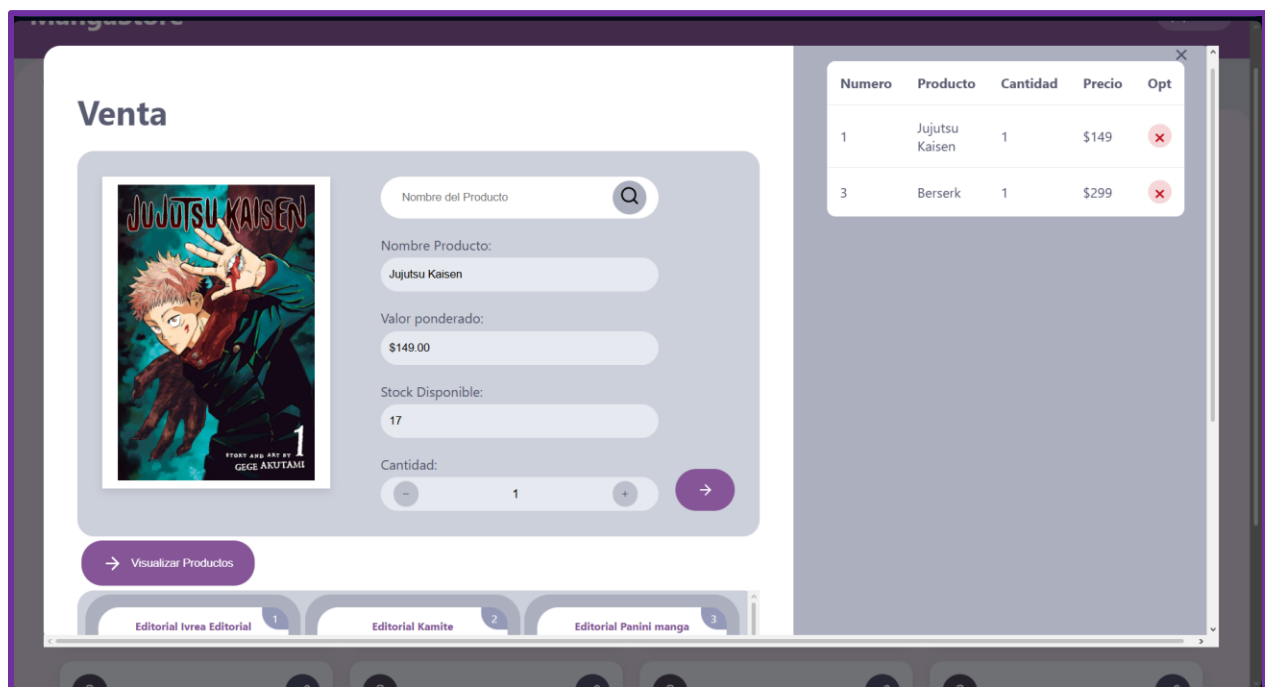
Ventana principal al desplegar el “Ver detalles” de una card



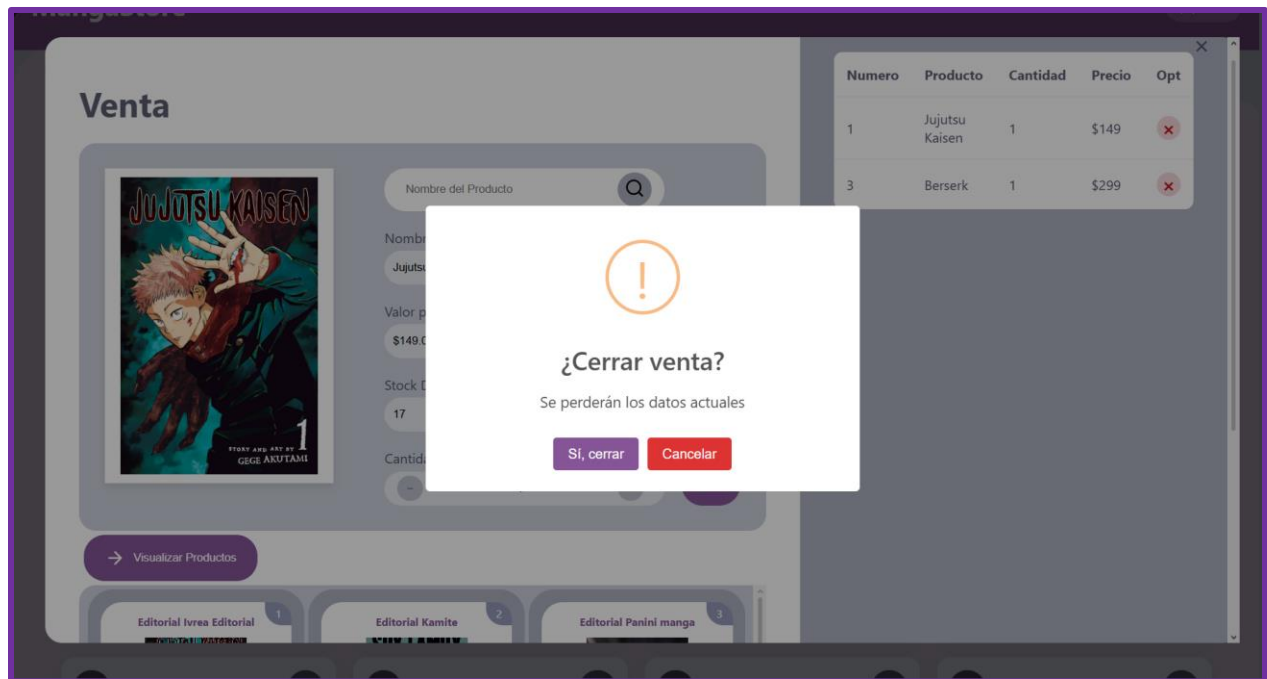
Eliminar una venta Alerta



Modal actualizar venta



Alerta al cerrar la edición venta



Ventas.js

El archivo JavaScript actúa como el motor dinámico de la interfaz, gestionando la interactividad de la ventana y sincronizando la información recolectada en cada proceso con la API. A continuación, se detallan los métodos críticos para su funcionamiento, organizados por categorías:

Mapeo y visualización

MapearVentas()

Esta función, es la que se encarga de transformar los datos de las ventas en elementos visuales dentro de la interfaz de usuario. Consiste en recibir un conjunto de datos (ya sea una venta individual o un listado), limpiar el contenedor HTML correspondiente para evitar duplicados y recorrer cada registro para generar dinámicamente tarjetas. Dentro de cada tarjeta, la función procesa de forma anidada la lista de artículos vendidos, convirtiéndolos en una estructura de lista HTML que muestra detalles clave como el nombre del manga, la cantidad adquirida y el subtotal de cada producto.

```

38
39  /*Funcion correspondiente al mapeo de las ventas , aqui se encuentra el card que se genera dinamicamente con la disposcion adecuada
40 para mostrar en el formato incial datos : num venta , total ,fecha y metodo de pago*/
41
42  /*NOTA : Tiene un paramatro puesto que mas abajo hay dos metodos uno para cargar todas las ventas que utiliza el edpoint
43 de get y otro que se trata de un metodo filtro para buscar una venta especifica por id*/
44  function mapearVentas(ventas) {
45
46      const contenedor = document.getElementById('contenedor-ventas');
47      contenedor.innerHTML = '';
48      if (!ventas) return;
49      // SI ES UNA SOLA VENTA
50      //Valida que sea una array lo que le esta pasando
51      const listaVentas = Array.isArray(ventas)
52          ? ventas
53          : [ventas];
54      listaVentas.forEach(venta => {
55          const card = document.createElement('div');
56          card.className = 'venta-card';
57          const listaArticulos = venta.articulos || [];
58          //Listado con los articulos que se seleccionaron
59          const articulosHTML = listaArticulos.map(art => `
60              <li class="articulo-item">
61                  <div>
62                      <strong>${art.manga}</strong>
63                  </div>
64                  <div class="articulo-info">
65                      <span>Cant: ${art.cantidad}</span>
66                      <span>${art.subtotal}</span>
67                  </div>
68              </li>
69          `).join('');

```

```

70 //Card
71 card.innerHTML = `
72     <!-- BOTÓN ELIMINAR -->
73     <button
74         class="btn-eliminar"
75         title="Eliminar venta">
76         <i data-lucide="trash-2"></i>
77     </button>
78     <!-- BOTÓN ACTUALIZAR -->
79     <button
80         class="btn-actualizar"
81         title="Actualizar venta">
82         <i data-lucide="pencil"></i>
83     </button>
84     <div class="venta-header">
85         <div>
86             <h3>Venta #${venta.id_venta}</h3>
87             <p>${venta.fecha}</p>
88         </div>
89         <div class="venta-total">
90             ${venta.total_pagado}
91         </div>
92     </div>
93     <div class="venta-body">
94         <p>
95             <strong>Método:</strong> ${venta.metodo_pago}
96         </p>
97     </div>
98     <button class="btn-leer-mas">
99         Ver detalles
100    </button>
101    <div class="detalles">
102        <h4>Artículos</h4>
103        <ul>
104            ${articulosHTML}
105        </ul>
106    </div>
107 `;
108

```

Muestra la construcción de la estructura para la tarjeta de venta individual, utilizando **template literals** para insertar datos específicos. La estructura incluye botones interactivos para **eliminar y actualizar** registros, un encabezado que muestra el **ID de la venta**, la **fecha** y el **monto total pagado**, y un cuerpo que detalla el **método de pago**. Finalmente, incorpora un botón de "Ver detalles" que despliega una sección oculta donde se inyecta dinámicamente, mediante la variable **articulosHTML**, la lista de productos asociados a dicha transacción.

```

// BOTÓN VER DETALLES
//Boton para ver los detalles que son la lista antes mencionada con los mangas que llevo el cliente
const botonDetalles = card.querySelector('.btn-leer-mas');
const detalles = card.querySelector('.detalles');

botonDetalles.addEventListener('click', () => {
  detalles.classList.toggle('mostrar');
  botonDetalles.textContent =
    detalles.classList.contains('mostrar')
      ? 'Ocultar detalles'
      : 'Ver detalles';
});

// BOTÓN ELIMINAR
//Boton para eliminar la venta
const btnEliminar = card.querySelector('.btn-eliminar');

btnEliminar.addEventListener('click', () => {
  eliminarVenta(venta.id_venta);
});

// BOTÓN ACTUALIZAR
//Boton para actualizar la venta
const btnActualizar = card.querySelector('.btn-actualizar');
btnActualizar.addEventListener('click', () => {
  actualizarVenta(venta);
});
contenedor.appendChild(card);
});
// ACTIVAR ICONOS LUCIDE
lucide.createIcons();
}

```

Finalmente asigna interactividad a los elementos de cada tarjeta mediante eventos. En primer lugar, implementa un mecanismo de cambio para el botón de detalles, el cual cambia su texto entre "Ver detalles" y "Ocultar detalles" mientras muestra o esconde la lista de artículos. Posteriormente, se configuran los eventos para los botones de **Eliminar** y **Actualizar**, vinculándolos con las funciones `eliminarVenta`, utilizando el ID de la transacción, y `actualizarVenta`, pasando el objeto completo. Por último, la tarjeta terminada se añade al contenedor principal en el DOM y se ejecuta `lucide.createIcons()` para asegurar que los iconos visuales se rendericen correctamente.

cargarVentas() – cargarMangas()

```
async function cargarVentas() {  
  try {  
    const respuesta = await fetch('http://127.0.0.1:5000/ventas');  
    const ventas = await respuesta.json();  
    mapearVentas(ventas);  
  } catch (error) {  
    console.error("Error al obtener las ventas:", error);  
  }  
}  
  
document.addEventListener('DOMContentLoaded', () => {  
  cargarVentas();  
});
```

Utiliza la API **fetch** para realizar una petición GET al endpoint de ventas y, una vez recibida la respuesta, la convierte a formato **JSON** para enviarla como parámetro a la función `mapearVentas`, encargada de renderizarla visual. El código incluye un bloque **try-catch** para gestionar posibles errores de conexión en la consola, y se asegura de que el proceso inicie automáticamente al cargar la página mediante el evento `DOMContentLoaded`.

```

async function cargarMangas() {
  try {
    // Llamada al endpoint que ya incluye el LEFT JOIN con manga_imagenes
    const response = await fetch('http://127.0.0.1:5000/mangas');
    const mangas = await response.json();

    const container = document.getElementById('mangas-list-container');

    // Generar las tarjetas dinámicamente
    mangas.forEach(m => {
      const card = document.createElement('div');
      card.className = 'manga-card';

      // si la BD no tiene url_imagen, usa una por defecto
      const rutaPortada = m.url_imagen ? m.url_imagen : 'img/default.png';

      // Construir el HTML de la tarjeta con los datos del manga
      card.innerHTML = `
        <div class="card-top">
          <span class="id-badge">${m.id}</span>
          <span class="editorial-label">Editorial ${m.id_editorial}</span>
          
          <span class="stock-label"> <i data-lucide="package"></i> ${m.stock} Disponible/s </span>
        </div>
        <div class="manga-info">
          <h3>${m.titulo}</h3>
          <p>${m.autor}</p>
          <div class="price-tag">
            <i data-lucide="dollar-sign"></i>
            <span>${m.precio.toFixed(2)}</span>
          </div>
        </div>
      `;
    });
  }
};

```

Realiza una petición fetch al endpoint del servidor para obtener la lista de mangas en formato JSON. Posteriormente, recorre cada objeto del arreglo recibido mediante un `forEach` para crear dinámicamente elementos de tipo `div` con la clase `manga-card`. Dentro de este ciclo, se construye la estructura interna de la tarjeta utilizando *template literals*, donde se inyectan datos como el ID, la editorial, el título, el autor, el precio y el stock disponible, incluyendo además una lógica para mostrar una imagen por defecto en caso de que no exista una ruta de portada válida.

```

// --- NUEVO: Evento para llenar los campos al hacer clic ---
card.addEventListener('click', () => {
  // Llenar Nombre del Producto (Título + Volumen)
  document.getElementById('display-nombre').value = `${m.titulo} `;

  // Llenar Valor Ponderado (Precio)
  document.getElementById('display-precio').value = `$$${m.precio.toFixed(2)}`;

  // Llenar Stock Disponible
  document.getElementById('display-stock').value = m.stock;

  // Actualizar la imagen del placeholder
  const previewImg = document.querySelector('.image-placeholder');
  previewImg.innerHTML = ``;

  // 5. Resetear cantidad a 1 al seleccionar nuevo producto
  document.getElementById('display-cantidad').value = 1;
});
container.appendChild(card);
});

// Refrescar iconos para las nuevas tarjetas inyectadas
lucide.createIcons();

} catch (error) {
  console.error("Error al conectar con la API de MangaStore:", error);
}
}

```

Se añade un evento para el clic, el cual, al activarse, envía automáticamente la información del manga seleccionado (nombre, precio y stock) a campos de tipo input específicos del formulario principal. Además, actualiza la imagen de vista previa en un contenedor, resetea el campo de cantidad a uno y finalmente añade la tarjeta completa al contenedor principal del DOM.

Gestión del Carrito (Modal de Actualización)

cargarDatosTabla()

```
489 async function cargarDatosTabla() {  
490   console.log('Agregando producto a la tabla...');  
491   const nombre = document.getElementById('display-nombre').value.trim();  
492   const id = await obtenerId(nombre);  
493   const precio = document.getElementById('display-precio').value;  
494   const cantidad = parseInt(  
495     document.getElementById('display-cantidad').value  
496   );  
497   const stock = parseInt(  
498     document.getElementById('display-stock').value  
499   );  
500   const tbody = document.getElementById('cart-items');  
501   if (!id) {  
502     Swal.fire('Advertencia', 'Debe seleccionar un producto', 'warning');  
503     return;  
504   }  
505   if (!cantidad || cantidad <= 0) {  
506     Swal.fire('Advertencia', 'Ingrese una cantidad valida', 'warning');  
507     return;  
508   }  
509   if (cantidad > stock) {  
510     Swal.fire('Error', 'No hay stock suficiente', 'error');  
511     return;  
512   }  
513   let cantidadEnTabla = 0;  
514   const filas = document.querySelectorAll('#cart-items tr');  
515   filas.forEach(fila => {  
516     const idTabla = parseInt(fila.cells[0].innerText);  
517     const cantidadTabla = parseInt(fila.cells[2].innerText);  
518     // Si es el mismo manga  
519     if (idTabla === id) {  
520       cantidadEnTabla += cantidadTabla;  
521     }  
522   });  
523 }
```

Este método se encarga de recolectar los datos del producto (nombre, ID, precio, cantidad y stock) directamente desde los elementos de la interfaz. Incluye una serie de validaciones mediante alertas de *SweetAlert2* para asegurar que se haya seleccionado un producto, que la cantidad sea válida y que no supere el stock disponible. Además, recorre las filas actuales del carrito para sumar la cantidad de un mismo producto ya existente y así evitar que el total solicitado exceda el inventario real.

```

525     const totalSolicitado = cantidadEnTabla + cantidad;
526     if (totalSolicitado > stock) {
527
528         Swal.fire('Error', `Stock insuficiente. Disponible: ${stock - cantidadEnTabla}`, 'error');
529
530         return;
531     }
532     const bloque = {
533         id, nombre, cantidad, precio
534     }
535     console.log(bloque);
536     const fila = document.createElement('tr');
537
538     fila.innerHTML = `
539         <td>${bloque.id}</td>
540         <td>${bloque.nombre}</td>
541         <td>${bloque.cantidad}</td>
542         <td>${bloque.precio}</td>
543         <td>
544             <span class="btn-eliminar-tabla">&times;</span>
545         </td>
546     `;
547     //Evento para eliminar la fila del producto agregado a la tabla del carrito
548     const btnEliminar = fila.querySelector('.btn-eliminar-tabla');
549     btnEliminar.addEventListener('click', () => {
550         console.log(btnEliminar)
551         fila.remove();
552
553         calculoTotales();
554     });
555
556     tbody.appendChild(fila);
557     calculoTotales();
558
559
560
561 }

```

El método procede a la creación dinámica de la fila (tr) para la tabla del carrito si todas las validaciones previas fueron exitosas. Utiliza un objeto llamado bloque para organizar la información e inyectarla en el HTML de la fila, incluyendo un botón de eliminación. Finalmente, define un evento de clic para dicho botón que remueve la fila y recalcula los costos, concluyendo con la inserción de la nueva fila en el cuerpo de la tabla (tbody) y la actualización del total general de la venta.

calculoTotales()

```
function calculoTotales() {  
  const filas = document.querySelectorAll('#cart-items tr');  
  let subtotal = 0;  
  filas.forEach(fila => {  
    // Obtenemos precio y cantidad de las celdas (ajusta el índice si es necesario)  
    const cantidad = parseFloat(fila.cells[2].innerText) || 0;  
    const precioText = fila.cells[3].innerText;  
    const precio = parseFloat(  
      precioText.replace('$', '').trim()  
    ) || 0;  
    subtotal += cantidad * precio;  
    console.log(cantidad, precio);  
  });  
  const impuestoReal = subtotal * 0.16;  
  const impuesto = impuestoReal; // Ejemplo 16%  
  const subtotalFinal = subtotal - impuestoReal;  
  const descuento = subtotal > 1000 ? 50 : 0; // Ejemplo: $50 si es > 1000  
  const total = subtotal - descuento;  
  // Actualizar el HTML que pasaste  
  document.querySelector('.totals-section p:nth-child(1) span').innerText = ` $ ${subtotalFinal.toFixed(2)} `;  
  document.querySelector('.totals-section p:nth-child(2) span').innerText = ` $ ${impuesto.toFixed(2)} `;  
  document.querySelector('.totals-section p:nth-child(3) span').innerText = ` $ ${descuento.toFixed(2)} `;  
  document.querySelector('.final-total span').innerText = ` $ ${total.toFixed(2)} `;  
  return { subtotal, impuesto, descuento, total };  
}
```

El método recorre las filas de la tabla para obtener la cantidad y el precio de cada producto, calculando un subtotal acumulado. A partir de este valor, la función determina automáticamente un impuesto del 16%, y actualiza dinámicamente los valores en la interfaz del usuario, finalizando con el retorno de un objeto que contiene el desglose completo de la operación.

NOTA: El impuesto es un desglose y no un valor agregado, es decir el total que se muestra al final, ya incluye el impuesto, entonces el programa lo que hace es extraer ese impuesto mediante un cálculo para visualizarlo en pantalla.

Operaciones CRUD y Persistencia

procesarVenta(metodoPago)

```
async function procesarVenta(metodoPago) {
  const modal = document.getElementById('modal-venta');
  const id = modal?.dataset?.ventaId;
  const filas = document.querySelectorAll('#cart-items tr');
  if (filas.length === 0) {
    Swal.fire('Error', 'No hay productos en la venta', 'error');
    return;
  }
  const productos = [];
  filas.forEach(fila => {
    const manga_id = parseInt(fila.cells[0].innerText);
    const cantidad = parseInt(fila.cells[2].innerText);
    if (!manga_id || !cantidad) return;
    productos.push({ manga_id, cantidad });
  });

  if (productos.length === 0) {
    Swal.fire('Error', 'Productos inválidos', 'error');
    return;
  }
  const datos = {
    productos,
    metodo_pago: metodoPago
  };

  try {
    const res = await fetch(`http://127.0.0.1:5000/ventas/${id}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(datos)
    });
  }
};
```

El método obtiene el ID de la venta desde el modal y extrae los productos y cantidades directamente de las filas de la tabla. Tras validar que existan los artículos, organiza la información junto al método de pago en un objeto JSON y realiza una petición de tipo PUT al servidor para actualizar el registro.

```
const data = await res.json().catch(() => ({}));

if (!res.ok) {
  Swal.fire('Error', data.error || 'Error al actualizar venta', 'error');
  return;
}
await Swal.fire({
  title: 'Éxito',
  text: 'Pago procesado correctamente',
  icon: 'success'
});
// Limpieza
cerrarModalPago();
limpiarTabla();
limpiarForm();
location.reload();

} catch (error) {
  console.error("Error PUT venta:", error);
  Swal.fire('Error', 'Error de conexión con el servidor', 'error');
}
}
```

Si la operación falla, muestra una alerta de error, pero si es exitosa, lanza un mensaje de confirmación, todo mediante *SweetAlert2*. Por último, ejecuta una serie de funciones de limpieza para resetear los formularios, cierra el modal de pago y recarga la página para reflejar los cambios en la base de datos.

eliminarVenta(id)

```
//FUNCION PARA CREAR DINAMICAMENTE Y CARGAR EL FORM DE ELIMINA
async function eliminarVenta(id) { //Metodo para eliminar una venta desde un boton que se encuentra en las cards
  const result = await Swal.fire({ //Formato para avisar al usuario si esta seguro de eliminar la venta
    title: '¿Seguro que quieres eliminar esta venta?',
    text: 'Esta acción no se puede deshacer',
    icon: 'warning',
    width: '280px',
    showCancelButton: true,
    confirmButtonText: 'Sí, eliminar',
    confirmButtonColor: '#b0123f',
    cancelButtonText: 'Cancelar',
    cancelButtonColor: '#855597'
  });

  if (result.isConfirmed) { //Si es correctp

    try {
      const response = await fetch(
        `http://127.0.0.1:5000/ventas/${id}`, //Hace la solicitud por medio del endpoint de eliminar venta
        {
          method: 'DELETE' //En el metodo de delete
        }
      );
    }
    if (response.ok) { //Si la operacion se realizo con exito
      await Swal.fire({ //Devuelve una alerta de que la venta se borro correctamente
        title: '¡Venta eliminada!',
        text: 'La venta se eliminó correctamente',
        icon: 'success',
        width: '280px',

        confirmButtonText: 'OK',
        confirmButtonColor: '#855597'
      });
      location.reload(); //Recarga la pagina
    }
  }
}
```

Solicita confirmación al usuario mediante una alerta de *SweetAlert2* antes de continuar con el borrado. Si se confirma la acción, se realiza una petición al servidor utilizando el método DELETE hacia un endpoint que incluye el ID de la venta. En caso de éxito, se muestra un mensaje de confirmación y se recarga la página para actualizar la interfaz.

```

    } else { //Si no es exitoso devuelve una alerta avisando al usuario que no se pudo eliminar la venta
      await Swal.fire({
        title: 'Error',
        text: 'No se pudo eliminar la venta',
        icon: 'error',
        width: '280px',

        confirmButtonText: 'OK',
        confirmButtonColor: '#855597'
      });
    }
  } catch (error) {

    console.error('Error al eliminar venta:', error);

    await Swal.fire({
      title: 'Error',
      text: 'Ocurrió un error al eliminar la venta',
      icon: 'error',
      width: '280px',
      confirmButtonText: 'OK',
      confirmButtonColor: '#855597'
    });
  }
} else {
  await Swal.fire({
    title: 'Cancelado',
    text: 'La venta no fue eliminada',
    icon: 'info',
    width: '280px',
    confirmButtonText: 'OK',
    confirmButtonColor: '#855597'
  });
}
}
}

```

En esta parte se considera casos como, la cancelación del proceso o que este no se pueda llevar a cabo. Incluye bloques para mostrar alertas de error si la respuesta del servidor no es satisfactoria o si ocurre una falla en la conexión durante el bloque catch. Finalmente, si el usuario decide no confirmar la eliminación inicial, se despliega una alerta informativa indicando que la venta no fue eliminada.

Búsqueda y Filtrado

buscarVentas()

```

//Metodo para hacer la busqueda de las ventas por id
async function buscarVentas() {
  const tipoFiltro = //De un seleccion obtiene la opcion , en este caso solo es id
    document.getElementById('tipo-filtro').value;
  const valor =
    document.getElementById('input-busqueda')
      .value
      .trim(); //Obtiene el valor que se ingreso en el input
  if (valor === '') { //Cuando el input esta vacio se van a cargar todas las ventas que haya en la bd
    cargarVentas();
    return;
  }
  let url = '';
  switch (tipoFiltro) {
    case 'id':
      const valorBusqueda = parseInt(valor);
      console.log(valorBusqueda)
      url = `http://127.0.0.1:5000/ventas/${valorBusqueda}`;
      //En este segun lo que el cliente ingreso en el input lo coloca para mandar llamar el endpoint
      break;
  }
  try {
    const respuesta = await fetch(url); //pide la informacion al servidor
    if (!respuesta.ok) { //si el servidor no obtiene nada va colocar un mensaje en la pantalla de que el id no fue encontrado

      const contenedor =
        document.getElementById('contenedor-ventas');

      contenedor.innerHTML = `
        <p class="mensaje-error">
          No se encontró ninguna venta con ese ID
        </p>
      `;
      return;
    }
    const ventas = await respuesta.json(); //si obtiene la informacion correctamente
    mapearVentas(ventas) //Mandara llamar el metodo para que mapee la venta encontrada
  } catch (error) {
    console.error("Error en búsqueda:", error);
  }
}

```

Permite filtrar registros mediante su ID. El método captura el valor de búsqueda desde el input de la interfaz, si el campo está vacío, restaura la lista completa por medio de `cargarVentas()`; de lo contrario, realiza una petición fetch al servidor utilizando el ID proporcionado. Finalmente, si el registro existe, se procesa a través de `mapearVentas(ventas)` para mostrarlo, pero si no es encontrado, inyecta dinámicamente un mensaje de error en el contenedor visual para informar al usuario.

Carpeta editorial

Esta carpeta corresponde a la ventana de editoriales donde se visualizarán todas las editoriales que se existan dentro de la página.

Editorial.html

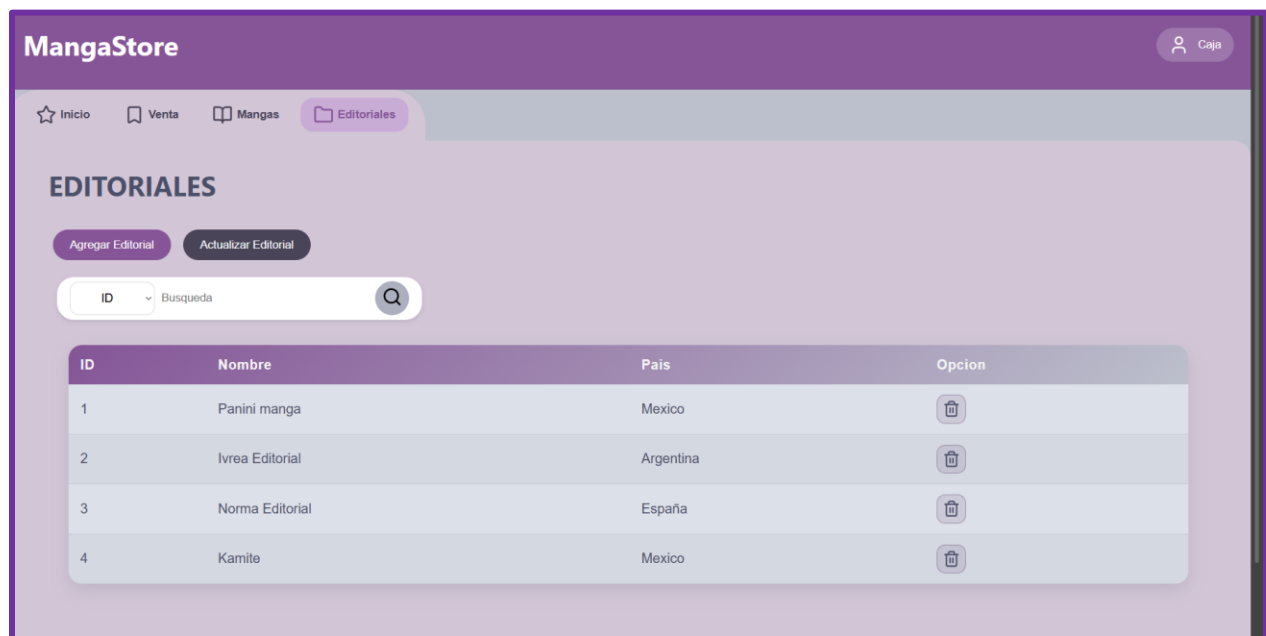
Se trata de la estructura que tendrá la ventana, donde se alojan los componentes que constituyen la página correspondiente a editoriales señalizados por medio de etiquetas.

En este archivo en la parte superior, se están mandando llamar las librerías para importar el estilo de los iconos, alertas y el estilo general de la página.

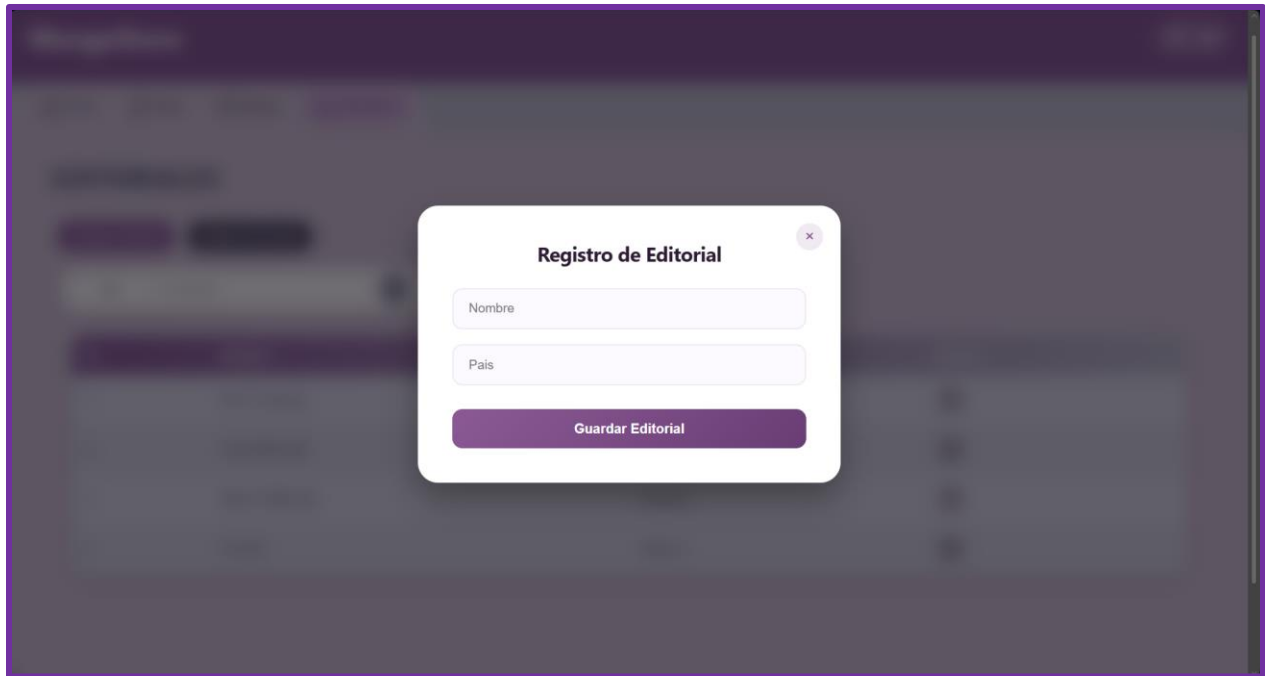
```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Editoriales</title>
  <link rel="stylesheet" href="styleEditorial.css">
  <script src="https://unpkg.com/lucide@latest"></script>
  <script src="https://cdn.jsdelivr.net/npm/sweetalert2@11"></script> <!--Libreria para las alertas-->
</head>
```

styleEditorial.css

Se encarga del apartado visual y define la interactividad básica, permitiendo que elementos como los botones cambien de color o forma al interactuar con ellos.



Formulario de registro de editoriales



A screenshot of a web application interface showing a modal form titled "Registro de Editorial". The modal is centered on a blurred background of a table. It contains two input fields: "Nombre" and "Pais", followed by a purple "Guardar Editorial" button. A close button (X) is in the top right corner of the modal.

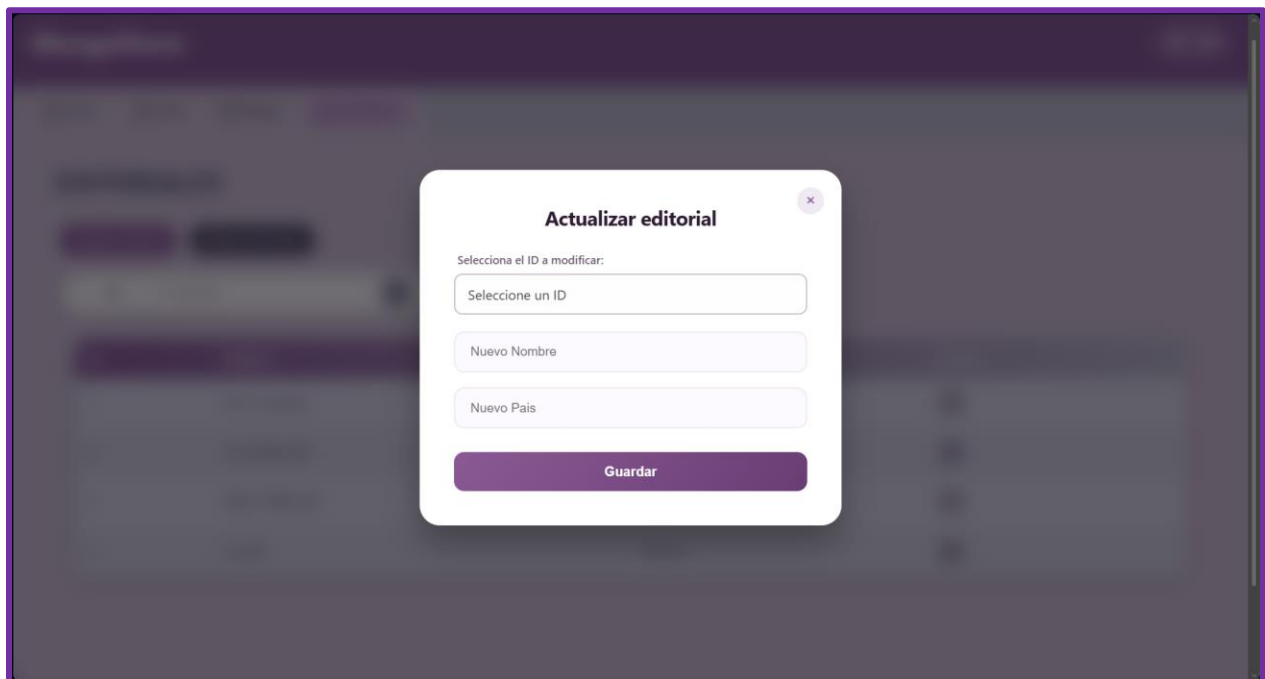
Registro de Editorial

Nombre

Pais

Guardar Editorial

Formulario de Edición



A screenshot of a web application interface showing a modal form titled "Actualizar editorial". The modal is centered on a blurred background of a table. It contains a dropdown menu labeled "Selecciona el ID a modificar:" with the placeholder "Seleccione un ID", followed by two input fields: "Nuevo Nombre" and "Nuevo Pais", and a purple "Guardar" button. A close button (X) is in the top right corner of the modal.

Actualizar editorial

Selecciona el ID a modificar:

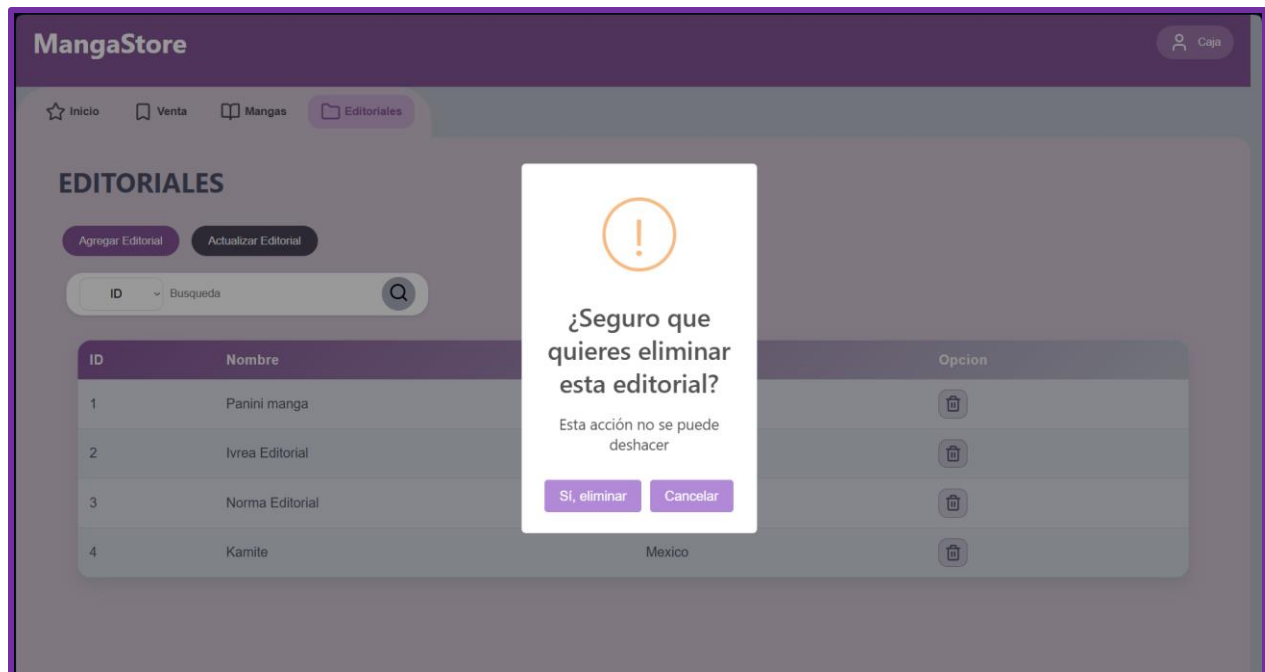
Seleccione un ID

Nuevo Nombre

Nuevo Pais

Guardar

Alerta de eliminacion



Editorial.js

Metodos CRUD

cargarEdit(editoriales)

```
function cargarEdit(editoriales) {  
  const cuerpoTabla = document.querySelector('#tablaEditorial tbody');  
  cuerpoTabla.innerHTML = ''; // Limpiamos la tabla antes de llenarla  
  
  // Recorremos el JSON y creamos una fila por cada manga  
  editoriales.forEach(editorial => {  
    const fila = `<tr>  
      <td>${editorial.id}</td>  
      <td>${editorial.nombre}</td>  
      <td>${editorial.pais}</td>  
      <td>  
        <button class="btn-eliminar" onclick="eliminarEditorial(${editorial.id})">  
          <i data-lucide="trash-2"></i>  
        </button>  
      </td>  
    </tr>`;  
    cuerpoTabla.innerHTML += fila;  
  });  
  
  lucide.createIcons();  
}
```

Se encarga de mapear dinámicamente una tabla HTML con los datos de los editoriales recibidos en formato JSON. El proceso consiste en básicamente limpiar el contenido

previo del cuerpo de la tabla, recorrer cada objeto para generar filas con sus respectivos botones de eliminación y, finalmente, reinicializar la librería Lucide para que los iconos de basura se visualicen correctamente en los elementos recién creados.

cargarEditorial()

```
// Función para obtener y mostrar los mangas
async function cargarEditorial() {
  // Solo ejecuta la lógica si el elemento existe en esta página
  const urlAPIEdt = 'http://127.0.0.1:5000/editoriales';
  try {
    // Hacemos la petición GET a la API
    const respuesta = await fetch(urlAPIEdt);
    const editoriales = await respuesta.json();
    cargarEdit(editoriales);
  } catch (error) {
    console.error("Error al cargar las editoriales:", error);
  }
}
```

Esta función actúa como el puente de comunicación entre el servidor y la interfaz. Su tarea es realizar una petición **HTTP GET** a la API mediante fetch, esperar la respuesta para convertirla en un objeto **JSON** y, finalmente, enviar esos datos a la función de cargarEdit para mapear la tabla, todo esto protegido por un bloque try-catch para capturar cualquier error de conexión.

eliminarEditorial(id)

```
80 });
81 // Función para eliminar una editorial
82 async function eliminarEditorial(id) {
83   const result = await Swal.fire({
84     title: '¿Seguro que quieres eliminar esta editorial?',
85     text: 'Esta acción no se puede deshacer',
86     icon: 'warning',
87     width: '280px',
88     showCancelButton: true,
89     cancelButtonText: 'Cancelar',
90     cancelButtonColor: '#b189d7',
91     confirmButtonText: 'Sí, eliminar',
92     confirmButtonColor: '#b189d7'
93   });
94
95   if (result.isConfirmed) {
96     try {
97       const response = await fetch(`http://127.0.0.1:5000/editoriales/${id}`, {
98         method: 'DELETE'
99       });
100       if (response.ok) {
101         await Swal.fire({
102           title: '¡Editorial eliminada!',
103           text: 'El registro se eliminó correctamente',
104           icon: 'success',
105           width: '280px',
106           confirmButtonText: 'ok',
107           confirmButtonColor: '#b189d7'
108         });
109         location.reload();
110       } else {
111         await Swal.fire({
112           title: 'Error',
113           text: 'No se pudo eliminar la editorial',
114           icon: 'error',
115           confirmButtonColor: '#b189d7'
116         });
117       }
118     } catch (error) {
119       console.error("Error al eliminar:", error);
120     }
121   }
122 }
```

Se detalla el flujo principal para la eliminación de una editorial: utiliza **SweetAlert2** para desplegar una ventana de confirmación y, si el usuario acepta, ejecuta una petición al servidor con el método **DELETE** hacia la API; finalmente, muestra un mensaje de éxito y recarga la página para actualizar la lista.

```

    } else {
      await Swal.fire({
        title: 'Cancelado',
        text: 'La editorial no fue eliminada',
        icon: 'error',
        width: '280px',
        confirmButtonText: 'ok',
        confirmButtonColor: '#b189d7'
      });
    }
  }
}

```

Aquí es donde se salta una alerta de cancelación si el usuario decide no proceder con el borrado, asegurando que el sistema notifique que el registro no sufrió alteraciones.

mapForm(tipo)

```

208 //Formulario para agregar una nueva editorial
209 function mapForm(tipo) {
210   const contenedor = document.getElementById('Form');
211   if (tipo === 'agregar') {
212     contenedor.style.display = 'flex';
213     contenedor.innerHTML = `
214       <form id="form-agregar" class="forms">
215         <!-- Botón de cerrar integrado -->
216         <button type="button" class="btn-cerrar-dinamico" id="cerrar-form-agregar">
217           &times;
218         </button>
219         <h3>Registro de Editorial</h3>
220         <input type="text" name="nombre" placeholder="Nombre" required autofocus>
221         <input type="text" name="pais" placeholder="Pais" required autofocus>
222         <button type="submit">Guardar Editorial</button>
223       </form>
224     `;
225     const btnCerrarPost = document.getElementById('cerrar-form-agregar');
226     btnCerrarPost.addEventListener('click', () => {
227       contenedor.style.display = 'none';
228     });
229   }
230 }

```

Se trata del mapeo del forms en este caso es para **agregar** una editorial, inyectando un formulario limpio mediante innerHTML en un contenedor oculto, activando su visibilidad con flex y configurando un evento de cierre para que el usuario pueda descartar la ventana fácilmente.

```

    } else if (tipo === 'actualizar') {
      contenedor.style.display = 'flex';
      contenedor.innerHTML = `
        <form id="form-actualizar" class="forms">
          <!-- Botón de cerrar integrado -->
          <button type="button" class="btn-cerrar-dinamico" id ="cerrar-form-actualizar">
            &times;
          </button>
          <h3>Actualizar editorial</h3>

          <label>Selecciona el ID a modificar:</label>
          <select name="id_editorial" id="select-ids" class="select-int">
            <!-- Aquí se cargarán los IDs dinámicamente -->
            <option value="">Cargando...</option>
          </select>

          <input type="text" placeholder="Nuevo Nombre" name="nombre" id="inp-Nombre">
          <input type="text" placeholder="Nuevo Pais" name="pais" id="inp-Pais">
          <button type="submit">Guardar</button>
        </form>
      `;

      // Llamamos a la función para cargar el combo de id
      cargarIdCombo();
      const btnCerrarUpdate = document.getElementById('cerrar-form-actualizar');
      btnCerrarUpdate.addEventListener('click', () => {
        contenedor.style.display = 'none';
      });
    }
  }
}

```

Se muestra la opción de **actualizar**, la cual inyecta un formulario más complejo que incluye un elemento select para elegir el ID, además, manda llamar la función cargarIdCombo para llenar las opciones con los datos existentes y establece un botón para cerrar la interfaz de edición.

cargarIdCombo()

```
261 // Función para cargar el combo de IDs en el formulario de actualización
262 async function cargarIdCombo() {
263   const select = document.getElementById('select-ids');
264   const nom = document.getElementById('inp-Nombre');
265   const pais = document.getElementById('inp-Pais');
266
267   try {
268     const respuesta = await fetch('http://127.0.0.1:5000/editoriales'); // Ruta que trae todas
269     const editoriales = await respuesta.json();
270
271     select.innerHTML = '<option value="">Seleccione un ID</option>';
272
273     // Creamos una opción por cada editorial
274     editoriales.forEach(ed => {
275       const opcion = document.createElement('option');
276       opcion.value = ed.id; // El valor que se envía a Python
277       opcion.textContent = `ID: ${ed.id} - ${ed.nombre}`; // Lo que ve el usuario
278       select.appendChild(opcion);
279     });
280
281     // Agregamos el evento para detectar el cambio de selección y que cargue los datos asociados al id en los inputs
282     select.addEventListener('change', (e) => {
283       const idSeleccionado = e.target.value;
284
285       // Buscamos la editorial que coincide con el ID seleccionado
286       const editorialEncontrada = editoriales.find(ed => ed.id == idSeleccionado);
287
288       if (editorialEncontrada) {
289         // Llenamos los inputs con los datos encontrados
290         nom.value = editorialEncontrada.nombre;
291         pais.value = editorialEncontrada.pais;
292       } else {
293         // Limpiamos si no hay selección válida
294         nom.value = "";
295         pais.value = "";
296       }
297     });
298   } catch (error) {
299     console.error("Error al cargar IDs:", error);
300   }
301 }
```

Esta función gestiona la sincronización de datos para el formulario de actualización. Su propósito es llenar dinámicamente una lista desplegable con los IDs de las editoriales obtenidas de la API y configurar un evento de escucha que busca automáticamente la información del registro seleccionado para rellenar los campos de input, permitiendo así que el usuario vea y edite los valores actuales de forma inmediata.

buscarEditoriales()

```

135  async function buscarEditoriales() {
136      const tipoFiltro =
137          document.getElementById('tipo-filtro').value;
138      const valor =
139          document.getElementById('input-busqueda')
140              .value
141              .trim();
142
143      if (valor === '') {
144          cargarEditorial();
145          return;
146      }
147      let url = '';
148      let tipo = '';
149      let valorBusqueda = '';
150      if (tipoFiltro == 'id') {
151          valorBusqueda = parseInt(valor);
152          tipo = 'Id';
153          url = `http://127.0.0.1:5000/editoriales/${valorBusqueda}`;
154
155      } else if (tipoFiltro == 'nombre') {
156          valorBusqueda = valor.trim();
157          tipo = 'Nombre';
158
159          url = `http://127.0.0.1:5000/editoriales/nombre/${valorBusqueda}`;
160
161      } else if (tipoFiltro == 'pais') {
162          valorBusqueda = valor.trim();
163          tipo = 'Pais';
164
165          url = `http://127.0.0.1:5000/editoriales/pais/${valorBusqueda}`;
166      }

```

Se muestra la **consulta**, donde la función extrae el tipo de filtro (**ID, nombre o país**) y el valor ingresado en el input por el usuario para generar automáticamente la URL específica hacia la API, incluyendo una validación inicial que recarga la lista completa si el campo de búsqueda está vacío.

```

try {
  const respuesta = await fetch(url);
  if (!respuesta.ok) {

    const cuerpoTabla =
      document.querySelector('#tablaEditorial tbody');
    cuerpoTabla.innerHTML = `
      <tr>
        <td colspan="3" class="mensaje-error">
          No se encontró ninguna editorial con ese ${tipo} , ${valorBusqueda}
        </td>
      </tr>
    `;

    return;
  }
  const editoriales = await respuesta.json();
  if (Array.isArray(editoriales)) {
    cargarEdit(editoriales);
  } else {
    cargarEdit([editoriales]);
  }
} catch (error) {
  console.error("Error en búsqueda:", error);
}
}

```

Se trata del procesamiento de la respuesta, la cual realiza la petición y maneja dos situaciones: si no hay resultados, inyecta un mensaje de error directamente en la tabla; si la búsqueda es exitosa, normaliza los datos (asegurándose de que sean un arreglo) y llama a la función de mapeo para actualizar la vista.

CONCLUSIONES

Este proyecto destaca por la implementación de una arquitectura cliente-servidor robusta que hace uso de **Python (Flask)** en el backend y tecnologías web estándar (**HTML, CSS, JavaScript**) en el frontend.

En el siguiente apartado, se detallarán las conclusiones partiendo de diversos puntos.

1. ESTRUCTURA

El proyecto está diseñado bajo una estructura basada en vistas, lo que facilita el mantenimiento y la escalabilidad del código. La organización por carpetas (como catalogomangas, editorial, historialVentas y pp) permite que cada módulo funcional

maneje de forma independiente su estructura (HTML), lo visual (CSS) y la funcionalidad (JavaScript). El backend se encuentra centralizado en mangaStore.py, que actúa como un puente eficiente entre la base de datos y la interfaz, estandarizando la comunicación mediante el formato JSON.

2. DISEÑO DE DATOS

El sistema utiliza **SQLite3** con un esquema relacional bien definido que garantiza la integridad de la información mediante el uso de llaves foráneas y restricciones.

- **Gestión de Inventario:** Las tablas de mangas y editoriales están relacionadas de forma que se evita la redundancia y se asegura que cada producto pertenezca a una entidad editorial válida.
- **Control de Ventas:** El modelo de ventas, emplea una tabla intermedia (detalles_ventas) para manejar la relación de muchos a muchos entre ventas y productos, lo que permite desglosar cada transacción con precisión.
- **Imágenes de Mangas:** La separación de las imágenes en la tabla manga_imagenes optimiza el rendimiento de la tabla principal de productos al no sobrecargarla con datos extensos de URLs. **Nota:** Las imágenes se almacenan dentro de una carpeta local en el programa, lo cual permite que su mapeo dentro de la interfaz no sea tardado, ya que, de lo contrario, si se almacenaran las imágenes dentro de la bd el proceso sería tardado y se consumirían más recursos.

3. CRUD

La API implementa una serie de operaciones **CRUD** (Crear, Leer, Actualizar, Borrar):

- **Validaciones Críticas:** Antes de realizar cualquier inserción o actualización (POST/PUT), el sistema verifica que no existan duplicados, que los campos obligatorios estén presentes y que los valores numéricos (precio, stock) sean positivos.

- **Protección de Datos:** En el módulo de editoriales, el método DELETE incluye una lógica de seguridad que impide eliminar una editorial si esta tiene mangas vinculados.
- **Gestión de Stock:** Al procesar una venta, el sistema no solo registra la transacción, sino que actualiza automáticamente los niveles de inventario, incluyendo mecanismos de rollback en procesos complejos (como la actualización de ventas) para restaurar la base de datos si ocurre un error.

4. INTERFAZ VISUAL Y EXPERIENCIA DEL USUARIO

El frontend utiliza peticiones fetch para actualizar el DOM sin necesidad de recargar la página completa, lo que mejora la fluidez de la administración.

- **Panel de Control (Dashboard):** Incluye un módulo de estadísticas que permite al administrador visualizar en tiempo real el rendimiento del día, desglosando ventas por método de pago (efectivo/tarjeta) y calculando totales acumulados.
- **Ventana de Inventario:** En la ventana del módulo de gestión de inventario se cuentan con múltiples interacciones, desde el mapeo dinámico de cards que almacenan la información importante los mangas, así como una imagen ilustrativa, también hace la experiencia visual más cómoda al ocultar los botones de eliminar y actualizar a menos que el usuario decida pasar el cursor por encima de la card.
- **Ventana de Ventas:** El módulo de gestión de ventas también busca brindar una experiencia visual cómoda al mapear mini cards que en primera instancia arrojan los datos más relevantes en relación con la venta, para posteriormente si el usuario desea un desglose mas completo de los productos, este tenga la opción de clicar en los detalles de la venta, para observar esa información.
- **Interfaz Interactiva:** El uso de bibliotecas como **SweetAlert2** para confirmaciones y alertas, junto con un sistema de búsqueda flexible (usando la cláusula LIKE e insensibilidad a mayúsculas/minúsculas), proporciona una herramienta fácil de usar

CONCLUSION GENERAL

En resumen, "MangaStore" no es solo un sistema de inventario y venta digital, sino una solución administrativa que digitaliza flujos de trabajo tradicionalmente propensos a errores. La integración exitosa de validaciones rigurosas en el backend con una interfaz dinámica en el frontend demuestra un desarrollo en la seguridad de los datos y la usabilidad del sistema, cumpliendo con el objetivo de optimizar la eficiencia operativa del negocio.